Programming Paradigms Introduction

Learning Outcomes

At the end of this discussion you should be able to:

1. Explain what a programming paradigm is.
2. Identify the four main programming paradigms
3. Explain why programming languages are shifting to multi-paradigmness

The title of this course

Let's start by talking about the title of the course name because it does sound like some buzz phrase. You will start saying this phrase soon so let's get the definition out of the way.

"Programming paradigms".

You're probably familiar what half of this phrase means, I mean, I hope you are, otherwise I don't know what to do.

Paradigm

Let's focus first on the non-obvious part, the word paradigm. This word comes up often in academia. You probably heard of the term **paradigm shift** somewhere, it describes some form of fundamental change in the way we think within scientific disciplines often characterizing a scientific revolution. One notable example of a paradigm is the shift from Ptolemaic or Geocentric cosmology to Copernican or Heliocentric cosmology.

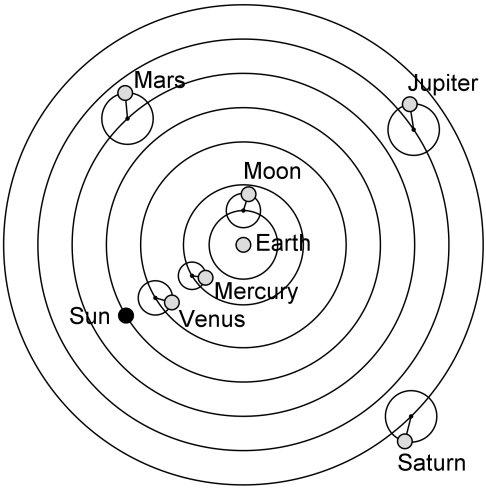


Figure 1: Ptolemaic Geocentric model of the solar system

Based on this context you can kind of formulate what the word paradigm means. This one connotes a similar meaning in the context of programming:

A paradigm is "*a framework containing the basic assumptions, ways of thinking, and methodology that are commonly accepted by members of a scientific community.*" (*PARADIGM Definition & Meaning Dictionary.com*,

n.d.). It is a set of ideas and concepts that describe some **way of thinking**.

If we go back to the geocentric vs heliocentric paradigms in astronomy, you can't really definitively say that the heliocentric model is the only correct model of the solar system, you can still reconcile the geocentric model's perspective of putting the earth in the center by modelling heavenly bodies' movement with epicycles. After all, whether the earth or the sun is the center is a matter of perspective. It just so happens that placing the sun in the center provided science with a more natural way of describing planetary movement. The heliocentric paradigm ended up uprooting geocentric paradigm as the dominant worldview, providing science with ideas that we still accept as truth until now, the earth is not the center. The earth is just one of the 9 planets, is not that special, gravity and inertia works in way which causes planets to move.

A paradigm shift like this is actually happening in programming language design, well talk about that some time later.

Programming

Now that you understand what paradigm means, let's talk about the first word, programming. As second year CS students, what is your definition of **programming**? It's a strange question to ask in a second year course, but I want you all to think about what the definition of programming is. The way you answer this question may actually tell you which perspective or programming **PARADIGM** you follow. When you say you are programming some kind of mechanism or behavior *what are you actually doing*? How do you define what a program is, and what is its relationship to a computer?

Here's one definition from the internet:

> Computer programming is the process that professionals use to write code that instructs how a computer, application or software

> program performs. At its most basic, computer programming is a set of instructions to facilitate specific actions.(Cote 2025)

That is correct. Let me simplify that definition to this.

Programming is when you tell a computer what to do. When you write programs you're writing **instructions** for your computer.

1. Ask the user for a number
2. Store that number to a variable called x.
3. While x is greater than 7 do step 4 otherwise proceed
4. Subtract 7 to x and store the difference to x
5. Show the user the value of x

That is a good definition of programming. It gives you an understanding on how you write programs that work. All you need to do is to write correct instructions that the computer understands, and you'll have a perfectly working program. A **programming language** is a medium that describes how to write instructions to communicate to your computer. If you learn to do that then you can go ahead and program away.

It is a correct definition, but is it the *only* correct definition? It defines programming under the paradigm **imperative programming**. I will not begrudge you if this is the only definition you know since there is a huge likelihood that the only paradigm you've been exposed to has been imperative programming.

Taxonomy of Programming Paradigms

For someone who has been exposed to C, C++ and nothing else, you might feel that the natural way to code is the *imperative way* when in fact there are alternatives.

The diagram (MovGP0 2013) here represents the alternative schools of thoughts describing how to program. This diagram taxonomizes programming languages by identifying which paradigms they are under. Most of these paradigms are either not pragmatic, not popular enough

or not unique enough to be studied in this course. Instead, we will be focusing on four major programming paradigms:

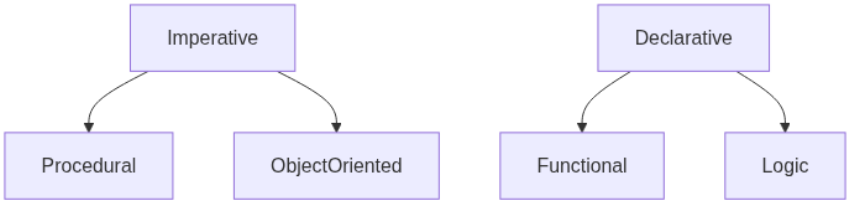


Figure 2: Programming Paradigms

Under the **imperative paradigm** family, we have procedural programming and objective oriented programming. Languages under the imperative paradigm create programs that go through different *states*. The *initial state* of an imperative program is the problem statement. From here the program follows ordered steps, with respect to the current state. Each step in the program can also constitute a *state change*. The solution is found at the *final state* of the program.

Under the **declarative family**, we have functional programming and logic programming. Declarative programs are not broken into ordered steps. Instead, declarative programs solve a problem by *defining a problem set* according to the language. For example, problem can be defined as a *composition of functions*, or a *logical relation*. The program then searches for valid solutions with respect to the definitions declared in the program.

This course will give you an overview on these programming paradigms. Each of these are built upon the foundation of some mathematical formalism. We will explore the advantages and disadvantages of each paradigm while we take a tour through these four. Studying the disciplines upheld by these paradigms will also teach us good programming practices for designing elegant programs that transcend any programming paradigm.

Multi-paradigm programming languages

The way it used to be was that a programming language would be written with features adhering to the concepts of a particular paradigm. Sometimes, a language is written with fresh features that follow a different mathematical formalism that births its own programming paradigm. Back then paradigms worked like programming language **classifications**. The programming language C for example is a strong follower of procedural programming. Therefore, you can think of C as classified under procedural programming.

But as time passed by classifying a newer programming language under one paradigm became harder and harder. A programming language like python for example is mostly procedural, object-oriented, but has functional programming features.

Modern programming languages evolved to become **multi-paradigm**. This inevitably happened because, as programming languages evolve, more **features** are added to it. These features are sometimes *borrowed from other paradigms* to solve a problem in a better way. This is the reason why established and mainstream programming languages like Java, C++, or Python tend to be multi-paradigm.

The multi-paradigmness of programming languages tend to be the reason why some programming language designers have abandoned the notion of building based on a strict paradigm. Instead, a language designer would *choose specific features* that they want to be supported on their programming language and implement it, regardless of its paradigm origins.

Instead of thinking of paradigms as classifications, you should think of paradigms as *different ways to solve problems*. You can solve one problem by applying the concepts and philosophies of imperative paradigm, and solve another problem by applying the concepts and philosophies of functional paradigm. You can even come up with a solution that combines the concepts and philosophies of different paradigms.

Imperative Programming Paradigm

Introduction

Imperative programming has turned out to be the *natural* paradigm of programming languages. The members of the imperative programming family have been dominating the market share of programming languages throughout the years with titans like BASIC, Pascal, C, Java and many more.

Learning Outcomes

At the end of this discussion you should be able to

1. Explain how imperative programming became the natural paradigm
2. Explain the concept of state in the context of imperative programming
3. Explain how the assignment statement enables the progression of states

4. Create structure programs to represent algorithms
5. Differentiate the subparadigms procedural programming and object-oriented programming

Quick Note on Imperative Programming and Procedural Programming

People usually use the terms Imperative programming and procedural programming interchangeably. Procedural programming is a subparadigm of imperative programming family, but some people refer to procedural programming as imperative programming. That's because other imperative paradigms like object-oriented programming is derived from procedural programming. You can think procedural programming as the ancestor if other imperative paradigms.

In this lecture I refer to Imperative paradigm as a whole, but I will focus on the main ideas that are common between other imperative paradigms. Ideas from object-oriented programming paradigm can be found on a separate lecture.

Popularity of the Imperative Paradigm

Imperative programming has turned out to be the *natural* paradigm of programming languages. The members of the imperative programming family have been dominating the market share of programming languages throughout the years with titans like BASIC, Pascal, C, Java and many more.

If you think about it, this is not that surprising. This paradigm's dominance could be attributed to most computer scientists' preference towards **pragmatic** and **efficient** programming languages. Especially since the most straightforward way of communicating to computer hardware is through the explicit manipulation of CPU memory and registers.

If you want the computer to do something for you, then you *communicate* to the computer that you want this and that to be done.

And if you manage to give the computer correct and comprehensive **instructions** then you'll end up getting what you want.

If we rewind back to the dawn of programming languages you'll see that early programming languages were built to **communicate to computer hardware**. As a result of this, programming languages naturally adopted syntax with **imperative** moods.

Assembly programs for example was mostly built from sequences of executable instructions which was patterned from imperative statements from natural language.

```
INC ITER
MOV AH,7
ADD AH, AL
```

For example the assembly instruction, `INC ITER`, tells the computer to increment the memory variable called `ITER`. The instruction MOV AH, 7, tells the computer to move the value 7 to the AH register. The instruction ADD AH, AL tells the computer to add the contents of the AH register to the AL register.

As time went by, newer *higher level programming languages* emerged (higher level meaning farther from hardware and closer to human language) like Basic, Pascal, and C. The syntax of these programming languages were written as **abstractions** of hardware code. Although programs written in these languages became more human-readable compared to its predecessors, these newer languages retained their imperative tones and mechanisms. This progression meant that higher level programming languages built atop of imperative languages naturally adopted the **imperative paradigm** as well. Java, Python, and C++ for example which were all written in C, followed this progression, thus establishing the imperative family as the dominant paradigm in programming language design.

The STATE

The existence of an explicit state is the foundation of imperative programming. The **state** of a program or a process on a given instance is the snapshot of its immediate relevant environment and context. The state of your CPU on a given instance for example will refer to the values found in the *registers and relevant memory*. On a specific process the state will refer to the values inside the *memory addresses* it resides in.

On a computer program the state can refer the conceptual set of variable values related to the program's runtime on some given instance. Let's use this program as an example:

```
int x = 3
int y = 4
x = x + y
```

At the start of runtime, the state of this program would be (*for all intents and purposes*) empty, since there are no relevant variables declared at this point. After executing the first line of code, the state of the program would look something like this:

variable	value
x	3

One integer variable named `x` with the value `3`. After the next line of code a new variable is introduced and immediately assigned with the value `4` so the state of the program at this instance will look like this:

variable	value
x	3
y	4

And at the last line, the value of `x` is updated by adding the value of `y`, so the final state of this program will look like this:

variable	value
x	7
y	4

Assignment Statement

Another important construct of the imperative programming paradigm is the **assignment statement**. Assignment statements and the concept of state are very related to each other.

Assignment statements allow your program to *mutate* the values of your variables. Mutation in the context of programming is a fancy term that basically means change. And as we learned earlier, *changes to the context* of a program, which includes variables, creates states. Therefore, every **assignment statement**, corresponds to **new states** of a for the program.

Assignment statements are usually executed through the use of the **"="** operator (some languages like Pascal use "`:=`" instead). Although it borrows the equality operator from math, assignment operators behave very differently from an equality statement. Instead of communicating some kind proposition, the assignment statement has an **imperative mood**. An equality a=b in math **declares** that some `a` is `b`, while an assignment operator `a=b` **commands** that `a`'s value is now the same as `b`. Mutation is introduced once you perform an assignment to `a` again, signifying a **change** in the value of `a`.

The closest corresponding mathematical construct to an assignment statement is the *let statement*. A statement in math such as "let x be equal to 3", has an **imperative mood**. But unlike an assignment statement which can change the value of a variable any number of times, a let statement can only set the value of a variable once.

For every assignment statement you feed to the computer, something meaningful happens. That particular "something" is manifests as **changes** to your program's context. The context of a program changes

for every individual mutation of a variable. And you can compare the difference between the before and after of a specific assignment by comparing the **before-assignment** state and the **after-assignment state**. The *progression* from one state to another characterizes the effect of an assignment.

This is the important takeaway that you need to remember. Imperative programming is characterized by **imperative statements**. Statements that tell the computer what to do. The most important type of these statements is the **assignment statement**. An assignment statements effect to your computer is characterized by the progression from one **state** to another. *Assignment statements make states*, and if you combine many of these assignment statements arranged in a particular manner, you can create a meaningful program that does something for you.

Structured Program Theorem

Creating meaningful programs in imperative programming is done by applying the **Bohm-Jacopini Theorem**, also known as the **Structured Program Theorem** (Böhm and Jacopini 1966). This theorem was one of the theoretical frameworks proposed to characterize imperative programming.

The theorem describes a formalism of a class called control flow graphs which are capable of representing any computable function. These control flow graphs are actually something you are intimately familiar of. It is known to you as the trusty old **flow chart**. Any control flow graph can be created by combining subprograms in three specific ways. A subprogram is a recursive unit of control flow graphs. A subprogram can be a single statement, or it can be a combination of more than one subprogram. Here are the three ways to combine subprograms:

1. Executing one subprogram, and then another subprogram (sequence)
2. Executing one of two subprograms according to the value of a boolean (selection)
3. Repeatedly executing a subprogram as long as a boolean expression is true (iteration)

The constructs described by this formalism became the natural architecture for programming language designers. This is the reason why CS students like you are introduced to programming using control flow graphs or flow charts. This is also the reason why programming languages like Pascal, C, Java and their derivatives are designed the way they are.

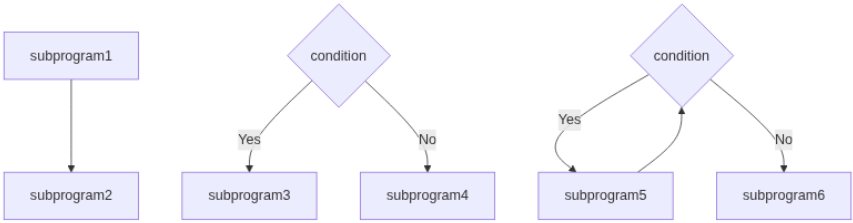


Figure 3: Flowcharts

Subparadigms under the Imperative family

Procedural programming

Programming languages like Fortran, ALGOL, BASIC, and C fall under the **procedural paradigm**. Languages under this paradigm simplify a complex system by subdividing a program into different **procedures** or functions.

Object-oriented programming

Object-oriented programming focuses on modelling a system based on the real world ontology of **objects**. It uses an expressive type system to program the interactions within a system. Most modern programming languages, like C++, Python, Java, etc., have some object-oriented programming features.

Functional Programming Formalism

During the 1930s a mathematician investigating the foundation of mathematics, named Alonzo Church, introduced a formal system of expressing computational logic. The system he created was called **Lambda Calculus**. It was until the 1960s when the system found its way through different disciplines. It became something more than a mathematical formalism and became an important concept in linguistics and **computer science**(Church 1932).

Before we dive into functional programming let's introduce ourselves to the formalism that inspired it, Lambda Calculus. These concepts may seem strange at first since it imagines a mathematical foundation beyond numbers, sets, and logic.

Expressions in the Lambda Calculus Formalism

Let Λ be the set of expressions under the Lambda calculus formalism

1. **Variables**. If x is a variable, then $x \in \Lambda$
2. **Abstractions**. If x is a variable and $\mathcal{M} \in \Lambda$, then $(\lambda x.\mathcal{M}) \in \Lambda$.
3. **Applications**. If $\mathcal{M} \in \Lambda \wedge \mathcal{N} \in \Lambda$, then $(\mathcal{M}\mathcal{N}) \in \Lambda$.

Take a look at these important precedence conventions. You might get confused if you read some lambda calculus expressions. Some people often omit parentheses or single-parametrizations to write shorter expressions:

1. Application is left associative

$$\mathcal{M}_1\mathcal{M}_2\mathcal{M}_3 = ((\mathcal{M}_1\mathcal{M}_2)\mathcal{M}_3)$$

2. Consecutive abstractions can be uncurried

$$\lambda xyz.\mathcal{M} = \lambda x.\lambda y.\lambda z.\mathcal{M}$$

3. The body of an abstraction extends to the right

$$\lambda x.\mathcal{M}\mathcal{N} = \lambda x.(\mathcal{M}\mathcal{N})$$

Reductions

Reductions are a ways to simplify and evaluate lambda expressions. You'll learn later that these reductions are basically concepts that are eventually adapted to functional programming concepts.

α equivalence:

α equivalence states that any bound variable, has no inherent meaning and can be replaced by another variable:

$$\lambda x.x \underset{\alpha}{=} \lambda y.y$$

Given a lambda calculus abstraction $\lambda x.\mathcal{M}$, this abstraction's bound variable is x. The bound variable x may appear somewhere in $\mathcal{M}$, the body of the abstraction. An alpha equivalence basically shows that the name of the variable has no inherent meaning. Therefore, you can replace it with any other variable name.

β Reductions

β reductions state how to simplify abstractions. This process is similar to applying a function in the context of programming. For example, we use the identity function ($\lambda x.x$) and apply it to some free variable y.

$$(\lambda x.x)y \underset{\beta}{=} y$$

When you beta-reduce some application $\mathcal{M}\mathcal{N}$, what you're doing is replacing all instances of the bound variable in $\mathcal{M}$ with $\mathcal{N}$. Here's another example,

$$(\lambda u.\lambda v.uvu)\lambda x.x \underset{\beta}{=} \lambda v.(\lambda x.x)v(\lambda x.x)$$

η reductions

η reductions describe equivalencies that arise because of free variables. If x is a variable and does not appear free in $\mathcal{M}$ then:

$$\lambda x.(\mathcal{M}x) \underset{\eta}{=} \mathcal{M}$$

The lambda expression here is just some redundant abstraction. These η reductions characterize higher level simplifications that are not always as obvious as the other reductions.

Reduction example

For example to reduce the following lambda expression, we must first understand what it means.

$$(\lambda x.\lambda y.(xy))(\lambda x.\lambda y.(xy))$$

In the outermost level, the expression is the application of $\lambda x.\lambda y.(xy)$ to itself. It follows the second type of lambda calculus expression discussed earlier, $\mathcal{M}\mathcal{N}$ where $\mathcal{M} \in \Lambda$ and $\mathcal{N} \in \Lambda$. In this context $\mathcal{M} = (\lambda x.\lambda y.(xy))$ and also $\mathcal{N} = (\lambda x.\lambda y.(xy))$.

When you start evaluating this expression, you might be tempted to automatically apply a β reduction by itself:

$$(\lambda x.\lambda y.(xy))(\lambda x.\lambda y.(xy)) \underset{\beta}{=} \lambda y.((\lambda x.\lambda y.(xy))y)$$
$$\underset{\beta}{=} \lambda y.\lambda y.(yy)$$

But this reduction is actually incorrect because the although x and y appear on both lambda expressions, these variables don't have the same meaning. The x and y variables inside the left lambda expression are **bound** inside this lambda expression. The x and y variables outside the left lambda expression (inside the right lambda expression) are **free** in its context, therefore, even though they look the same, it is incorrect to interchange the two variables.

Two avoid confusion with similarly named variables, it is advisable to apply α equivalencies, to give variables different names. This can be done by replacing the right abstractions' bound variables with u and v. Again, this alpha reduction doesn't change the meaning of the abstraction, it merely renames the bound variables.

$$(\lambda x.\lambda y.(xy))(\lambda x.\lambda y.(xy)) \underset{\alpha}{=} (\lambda x.\lambda y.(xy))(\lambda u.\lambda v.(uv))$$

The correct reduction now is as follows. Still a β reduction but without the ambiguity of similar variable names.

$$(\lambda x.\lambda y.(xy))(\lambda u.\lambda v.(uv)) \underset{\beta}{=} \lambda y.((\lambda u.\lambda v.(uv))y)$$
$$\underset{\beta}{=} \lambda y.\lambda v.(yv)$$

Lambda Calculus Encoding (Optional Reading)

One of the most interesting things that the Lambda Calculus System demonstrates is its use in computability theory. It's importance in computing has led to the formulation of the Church-Turing Thesis which conjectures that the Lambda Calculus System and the hypothetical Turing machine rules as complete representations of any algorithm. The lambda calculus system is said to be **Turing Complete**, which means that the system can simulate any concievable Turing

machine. To demonstrate its Turing completeness this topic will show some of the encodings in lambda calculus.

Boolean Values

A good starting point for encoding is the smallest unit of data, a boolean value. Boolean values, such as true and false can be represented by the following lambda expressions:

- **True:** $T = \lambda x.\lambda y.x$
- **False:** $F = \lambda x.\lambda y.y$

The way boolean values are encoded in lambda calculus using an abstraction that when applied to two values, produces first value for true and produces the second value for false. To demostrate the consistency of this encoding we can use this encoding of an `if_then_else` function. An `if_then_else` function in the context of any familiar programming language looks like this:

```
if(condition)
    this
else
    that
```

The function consumes three expressions (`condition`,`this`, and `that`) If the `condition` is true, then the function produces `this`, otherwise the function produces `that`. This function can be represented in lambda calculus as the following function:

$$\lambda c.\lambda x.\lambda y.cxy$$

c : `condition`, x : `this`, y : `that`

Applying this function shows how lambda calculus boolean encodings work. The example below shows what happens when the condition is applied to a true condition

$$
\begin{aligned}
(\lambda c.\lambda x.\lambda y.cxy)Tab &= (\lambda c.\lambda x.\lambda y.cxy)(\lambda u.\lambda v.u)ab \\
&= (\lambda x.\lambda y.((\lambda u.\lambda v.u)xy))ab \\
&= (\lambda x.\lambda y.((\lambda v.x)y))ab \\
&= (\lambda x.\lambda y.x)ab \\
&= (\lambda y.a)b \\
&= a
\end{aligned}
$$

[1]

The example below in the other hand is the same `if_then_else` function applied to a false condition

$$
\begin{aligned}
(\lambda c.\lambda x.\lambda y.cxy)Fab &= (\lambda c.\lambda x.\lambda y.cxy)(\lambda u.\lambda v.v)ab \\
&= (\lambda u.\lambda v.v)ab \\
&= b
\end{aligned}
$$

[2]

By representing the `if_then_else` function in lambda calculus we are also able to encode all the logic gates by reusing our `if_then_else` function.

`not` function

The `not` function can be represented using the`if_then_else` function this way:

```
if(condition)
    False
else
    True
```

In lambda calculus:

$$\lambda x.xFT$$

[1] In the first line a quick α equivalency is done on the `True` encoding.

[2] In the effort of saving lines β reductions with currying are applied in one go.

`or` Function

The `or` function can be represented using the`if_then_else` function this way:

```
if(left)
    True
else
    right
```

In lambda calculus:

$$\lambda x.\lambda y.xTy$$

`and` Function

The `or` function can be represented using the`if_then_else` function this way:

```
if(left)
    right
else
    false
```

In lambda calculus:

$$\lambda x.\lambda y.xyF$$

Natural Numbers

Natural numbers are encoded in lambda calculus similar to how numbers are described using *Peano's Axioms*. By defining 0 and defining a successor function. This encoding is called Church numerals. The first natural number 0 is defined as the lambda expression:

$$\overline{0} = \lambda s.\lambda z.z$$

Which is α equivalent to the encoding for boolean false.

All other natural numbers are derived using a special function called the successor function, such that the succesor of any natural number n is $n + 1$. The successor function in lambda calculus is represented by:

$$S = \lambda n.\lambda s.\lambda z.(s(nsz))$$

Using this function we can derive the encoding for the nfireatural number $\overline{1}$:

$$\begin{aligned}\overline{1} = S(\overline{0}) &= (\lambda n.\lambda s.\lambda z.(s(nsz)))(\lambda u.\lambda v.v) \\ &= \lambda s.\lambda z.(s((\lambda u.\lambda v.v)sz)) \\ &= \lambda s.\lambda z.(s(z))\end{aligned}$$

To derive this encoding for two we simply find $S(\overline{1})$.

$$\begin{aligned}\overline{2} = S(\overline{1}) &= (\lambda n.\lambda s.\lambda z.(s(nsz)))(\lambda u.\lambda v.(u(v))) \\ &= \lambda s.\lambda z.(s((\lambda u.\lambda v.(u(v)))sz)) \\ &= \lambda s.\lambda z.(s(s(z)))\end{aligned}$$

Continuing this process and generalizing successions will lead us the following encoding of natural numbers:

Church Numeral	Lambda Calculus Encoding
$\overline{0}$	$\lambda s.\lambda z.z$
$\overline{1}$	$\lambda s.\lambda z.(s(z))$
$\overline{2}$	$\lambda s.\lambda z.(s(s(z)))$
$\vdots$	$\vdots$
$\overline{n}$	$\lambda s.\lambda z.s^n(z)$[3]

Another useful definition of a successor function would be:

$$S(\overline{n}) = S(\lambda s.\lambda z.s^n(z)) = \lambda s.\lambda z.s(s^n(z)) = \lambda s.\lambda z.s^{n+1}(z)$$

Addition and Multiplication

The successor function will help us derive the lambda calculus representation of an addition function. In the same sense that addition is

[3] $\mathcal{M}^n(\mathcal{N})$ is a shorthand notation for n applications of $\mathcal{M}$ to $\mathcal{N}$ or $\mathcal{M}(\cdots(\mathcal{M}(\mathcal{M}(\mathcal{N})))\cdots)$.

just a repetition of increments, we can implement addition by making use of repetitive applications of the successor function.

$$\text{add} = \lambda m.\lambda n.mSn$$

To test the consistency of this function, we can try adding two arbitrary Church numerals $\overline{m}$, and $\overline{n}$.

$$\begin{aligned}
\text{add} \ \ \overline{m}\overline{n} &= (\lambda m.\lambda n.mSn)\overline{m}\overline{n} \\
&= \overline{m}S\overline{n} \\
&= (\lambda s.\lambda z.s^m(z))S(\lambda a.\lambda b.a^n(b)) \\
&= (\lambda z.S^m(z))(\lambda a.\lambda b.a^n(b)) \\
&= S^m(\lambda a.\lambda b.a^n(b)) \\
&= \lambda a.\lambda b.a^{m+n}(b) \\
&= \lambda s.\lambda z.s^{m+n}(z) \\
&= \overline{m+n}
\end{aligned}$$

Multiplication works similarly. To solve for the product of two Church numerals $\overline{m}$ and $\overline{n}$, we simply add n to 0 repetitively, m number of times. The function will look similar to a add, but with a partial application of add $\ n$ instead of the successor function S.

$$\text{mult} \ = \lambda m.\lambda n.m(\text{add} \ \ n)\overline{0}$$

Functional Programming Basics

Reimagined functions

Lambda calculus evolved from a system of logic foundation with deep roots to computation theory into something that became a basis for *programming language design*. Language designers started to consider the unconventional representation of lambda calculus expression as a valid and pragmatic way of *representing data*. Around 1950s programming languages patterned around the framework of lambda calculus started to emerge. One of the earliest and most important of these languages was **Lisp** which evolved to become a large family of programming languages(Steele and Gabriel 1996). Soon, more programming languages started to implement the same formalism described by lambda calculus. This opened a new paradigm of programming languages called **functional programming paradigm**. To explore this paradigm this section will introduce the programming

language **Haskell**. This language has become one of the most important functional programming language, setting the standards for other languages' paradigm.

How functions are treated Differently

One of the biggest difference between your classic imperative programming languages like C and Java and a functional programming language, is how it treats its *functions*. You might have probably guessed that since the paradigm has "function" in its name. To explore this contrast lets start by introducing a simple function, written in C. This function basically accepts an integer and returns the same integer but squared:

```
int square(int x){
    return x*x;
}
```

Functions like these are patterned from mathematical functions. It has a **name** to invoke it later called `square`, it has **specifications** on which type of data it accepts (`int x`) and produces (`int`), and finally it has **instructions** on what must be done when it is invoked (`return x*x`). Functional programming functions behave in more or less the same way.

```
square :: Int -> Int
square x = x * x
```

It looks different but all the parts you can find in a C function can also be found on this Haskell function.

In terms of invocation, they are also used similarly and of course they behave similarly:

```
square(5);
```

```
square 5
```

Although functions in non-functional programming behave and look similar to functions in functional programming language, they have a huge difference in the way the programming language *treats* it.

A function in C is treated differently from other types of data. In fact C programmers will rarely call a function a *value*. What this means is that canonical value types like integers, characters, and arrays (even compound value like `struct` instances and objects) can be *passed* on functions and can be *returned* as functions.

```
int* add_to_array(int arr*,int x,int size){
    for(int i=0;i<size;i++){
        arr[i]+x;
    }
    return arr;
}
```

C discriminates function from these canonical value types. Therefore, during runtime, non-functional programming languages interpret the expression `square(5)` as "the number 5 squared" while the expression `square` is just some disembodied function name (*square of which number?*). Imperative programming functions during runtime are meaningless unless they are directly invoked.

Higher Order Functions

Passing functions

Functional programming languages treat functions the same way it treats values, you *can* pass them in other functions, and you *can* return them as well.

```
s:: Int -> Int
func x = x + 1

p::Int -> Int
func x = x - 1

applytwice:: (Int-> Int) -> Int -> Int
applytwice f x = f (f x)
```

The code above shows two function definitions with some *type signature annotations* for readability. The first is the function `s::Int -> Int` which is *applied to an integer* and *produces an integer*. What it does

is it simply adds one to x. The second function is similar but what it does is subtracting one from x.

Type signature annotations are not required here, that's why they're called *annotations*. Adding these annotations will *restrict* the type the functions can be applied to. Type signature annotation syntax are understood like this

```
f :: Paramtype -> AnotherParamtype -> ... -> OutputType
```

The last type in the `->` series is the type the function produces (similar to its return type) and everything else before it are parameter types.

The third function called `apply_twice` is what we call a higher order function. A **higher order function** is a function that either accepts a function as a parameter or returns a function parameter or both. The function `applytwice`, as described by its *type signature*, is applied to a function `f` and an integer `x`and produces an integer. It applies the function `f` twice to `x`, something like $f(f(x))$.

By defining a function like this we can do something like this during runtime:

```
ghci> applytwice s 3
5
ghci> applytwice p 3
1
```

To a programmer with no experience with functional programming, this feature can be surprising especially since mathematical functions in algebra or calculus don't even explore this capability. But if you remember this is not just some arbitrary added feature added for novelty. This feature is directly patterned from lambda calculus:

$$\begin{aligned} \text{let} \quad S &= \lambda n.\lambda s.\lambda z.(s(nsz)) \\ T &= \lambda f.\lambda x.(f(fx)) \\ \overline{5} &= TS\overline{3} \end{aligned}$$

In lambda calculus an *abstraction* and an *application* does not restrict anyone from the type of expressions bound to variables. In the spirit of implementing lambda calculus, any functional programming language will allow you to do this as well.

Returning functions

On the other side of the coin, a function, in functional programming will also let you *return functions* the same way you *return* any other kind of data.

To explore this, suppose we have different functions that when applied to an integer, produces that integer plus a certain integer.

```
addTwo::Int -> Int
addTwo x = x + 2

addThree::Int -> Int
addThree x = x + 3

addFour:: Int -> Int
addFour x = x + 4
```

We can generalize these functions into a *function-maker* function, that when applied to an arbitrary integer `x`, will produce a function similar to `addx` which is a function that adds `x` to your integer.

```
addMaker::Int -> (Int->Int)
addMaker x = (\y -> x + y)
```

We are introducing new syntax here, but this new syntax is a representation of an expression we already know from lambda calculus. The definition for your `addMaker` (`\y -> x + y`) is basically an implementation of the following *lambda expression*[4]. `y` is the bound variable, and the operator `->` separates the inputs and the output, `x +`

[4]In fact, the reason why Haskell syntax uses the `\` character to represent lambda expressions is because this is your keyboard's best physical approximation of the Greek letter λ.

y[5].

$$\lambda y.\ \text{add}\ \ xy$$

What this expression means then is that `addMaker` *produces a lambda expression*, which essentially behaves exactly like a function. This allows you to create functions during runtime.

```
ghci> addSix = addMaker 6
```

Simply writing the expression `addSix` on your terminal will yield you an *error*, because printing `addSix` doesn't really have a meaning outside the world of lambda calculus. It is a lambda expression which is *basically* a function. *How do you represent a function as a string?*

But since `addSix` is a lambda that behaves exactly like a function, you can apply `addSix` to an integer, and it will give you a meaningful answer.

```
ghci> addSix 3
9
```

In fact, you can even omit the part where you bind the value returned by `addMaker` to a name, and instead use it directly. Here, `(addMaker 7)` is a lambda expression, therefore it can be applied to an integer.

```
ghci> (addMaker 7) 4
11
```

This nifty trick right here is the reason why lambda expressions are also called **anonymous functions** since these expressions on their own don't have a name. Lambda expressions generally appear in functional programming languages and even non-strictly functional programming languages. Lambdas can be useful if you want to create a function that will be used only once:

[5]Extra note: "+" does not exist in the universe of lambda calculus so instead what's used here is a reference to a lambda calculus abstraction called "add". You can check its definition in the optional reading Lambda Calculus Encodings

```
ghci> applytwice (\x -> x + 2) 3
7
ghci> (\x -> x * x) 4
16
```

Lambdas, just like any other canonical value type can be bound to identifiers[6]. Doing this will **name** the lambda thus allowing it to behave just like any other named function.

Closure

A higher order function like `addMaker` above, is not *only* producing the lambda inside its definition. What is actually being produced is a construct called a **closure** which is the function definition described by the *lambda* and the *environment* of the function call. The extra data, called environment, is the reason why the lambda `(\y -> x + y)` makes sense outside the context of `addMaker`. Without passing the *environment*, the variable `x` would be a free variable which will yield you a compilation error.

Inside your `addmaker` when you evaluate `addMaker 6`, the parameter `6` is *bound* to the variable x. Therefore, the resulting lambda produced by `addMaker 6` will behave exactly like the lambda, `(\y -> 6 + y)`.

Closures are still a direct consequence of lambda calculus' variable binding rules. Specifically how the variables x and y are bound in the innermost body of the lambda calculus abstraction $\lambda x.\lambda y.add x$.

Currying

If we look back to lambda calculus you'll notice how abstractions are defined to be:

$$\lambda x.\mathcal{M}$$

[6]Bindings are haskell's representation of a mathematical "let" statement. When you see a = operator like `x = 3`, this is not an assignment statement, but instead a let binding. In this case **3** is bound to the identifier `x`. Similar to what happens when you say let $x = 3$ in math.

Here we can see that abstractions are defined to have exactly one parameter. One can argue that this is different from how functional programming represents its own functions and lambdas since functions with *multiple parameters* are allowed in these languages. As it turns out, these functions are just *disguised* to have multiple parameters. These functions are just several single parameter functions combined to *simulate* multiple parameter functions. As an example: a function `add` that adds two numbers may look like multiple parameter functions:

```
plus x y = x + y
```

Internally, this function is equivalent to two lambda calculus abstractions, nested together to simulate *multiparameterness*.

```
plus = \x -> (\y -> x + y)
```

Here `plus` is a higher level function that accepts a single argument `x` and produces the closure `(\y -> x + y)`. This expression is a direct implementation of the following lambda calculus abstraction[7].

$$\text{plus} = \lambda x.\lambda y.\ \text{add}\ \ xy$$

Just like lambda calculus, Haskell's `->` operator is right associative, so you can write the same *plus* function as:

```
plus = \x -> \y -> x + y
```

You can even omit the first `->` and the `\` near `y`, and it will mean the same lambda expression:

```
plus = \x y -> x + y
```

Which looks almost exactly similar to a lambda calculus expression with multiple parameters:

$$\lambda xy.\ \text{add}\ \ xy$$

[7]Extra note: "+" does not exist in the universe of lambda calculus so instead what's used here is a reference to a lambda calculus abstraction called "add". You can check its definition in the optional reading Lambda Calculus Encodings

Now when you want to apply this function we write:

```
ghci> (plus 3) 4
7
```

Which means that: first, we are evaluating `(plus 3)` which will give us a *closure*. The closure is then *applied* to `4` which completes the evaluation to `7`. This is also a direct implementation of a lambda calculus application:

$$(\text{plus}\ \ \overline{3})\overline{4}$$

And just like lambda calculus, function applications in Haskell are also left associative, so you can omit the parentheses:

```
ghci> plus 3 4
7
```

All multiple parameter functions and lambdas in Haskell are nested single parameter abstractions in disguise, so for all intents and purposes, these two definitions for `plus` behave in the exact same way:

```
plus x y = x + y
```

```
plus = \x -> (\y -> x + y)
```

This means that the expression`(plus 3)` will have the same meaning regardless of the way you define `plus`. The expression `(plus 3)` has a special name, it is called a **partial application**. When you apply a function that is supposed to accept n parameters to m values (where $m < n$), i.e. you are supplying the function *less parameters* than it is expecting. Instead of getting the value, you get a partial application of that function which will evaluate to a *closure*.

The process of converting a multiparameter function or lambda to a nested single parameter lambda is called **currying**. This term is named after the mathematician Haskell Brooks Curry, which is the same Haskell, the programming language is named after.

A Stateless Paradigm

Functional purity and the absence of states

One of the most distinguishing aspects of functional programming and the declarative family of languages is its philosophy of statelessness. A programmer primarily exposed to mutable imperative programming languages will find that the concept of state is natural and maybe even inevitable. Functional paradigm challenges this concept and offers a much safer and mathematically intuitive philosophy. In the perspective of functional programming, there is no state, and everything is immutable.

Pure functions

To understand the functional programming perspective, we have to take a step away from the imperative programming definition of a function. Let's go back to the definition of a function in mathematics.

Across several, mathematical disciplines a function means the same thing. Consider two sets A and B. We can define a function as the

mapping between the elements of A and B. The elements of A and B can be anything, they can be numbers, which shows us how a function can be represented by a formula or a graph. The elements of A and B can be matrices and vectors, which defines a function as a transformation between two vector spaces. On the higher level perspective of category theory, functions are *morphisms* between objects of a given category.

$$f : (A \to B)$$

If you remember functions from discrete math, functions at its most basic form looks like the one above.

Functions in functional programming languages like Haskell are (arguably) the closest computer representation of a mathematical function. We call these functions, **pure functions**.

They differ from your standard C function because the definition inside pure functions are only instructions on how to produce a result based on the parameters. To fully understand this concept here are some examples of impure functions

```
int square(int x){
    addToExternalLogger("calculating square");
    return x*x;
}
```

The impurity in this function is the line where the function writes to some external logger, `addToExternalLogger("calculating square");`. A function can only be pure if the result of the function can be fully determined by its parameter. The only parameter here is `x`. Invoking this square function with the same parameter value does not do the same thing. The effect of changing the logger is dependent on the previous state of the external logger. The effect on the logger is what we call a **side effect** of the `square` function. It is a side effect since this line of code modifies values outside boundaries of the function.

```
int* increaseArray(int *a, int size){
    for(int i = 0; i < size; i++)
    a[i]+1;
}
```

A pass by address/reference function which changes the value of a parameter will automatically be an impure function since changing the value of `a` is a **mutation**, which is a side effect.

```
String headsortails(){
    if(randInt()%2==0)
        return "heads";
    else
        return "tails";
}
```

This function is also impure because the return value is not dependent on the parameters alone. The return value will be dependent on the randomization seed which is something outside the parameters of the function.

A pure function must satisfy these two:

1. A pure function has no side effects
2. A pure functions output must be dependent on the inputs alone[8]

A good way to test if a function is pure is if you can (theoretically) create an infinitely long *lookup table* such that, looking up the value for a specific input is perfectly identical to calling the function with the same input. And if you think about it this is the essence of a function. Functions are just of mappings between the domain and the range,

The Absence of mutation

One of the hallmarks that make imperative programming imperative is the assignment statement. It enables the program to advance to a new state. Purely functional programming languages like Haskell, lacks the mechanism to mutate anything. Using the "=" operator (which signals

[8] In fact if $f(a) = b$ and $f(a) = c$ where $b \neq c$, then f is not a function at all

an assignment statement in imperative languages) in functional languages *binds* the value on the right-hand side to the left-hand side. This mechanism is conceptually different from an assignment operation in C. It is perfectly fine to do the following in C:

```
int x = 0;
x = 1;
```

This code in C starts with a combined declaration and assignment: `int x = 0`. The second line, **reassigns** the same variable `x` to the new value `1`. These lines of code correspond to a *mutation* on the variable `x`, (from `0` to `1`). Because mutation can happen anytime during runtime, the value of the variable `x` is not definite. This is why values of variables in imperative programming **depend on the current state** of the program.

On the other hand, replicating this C code in Haskell is in fact not allowed:

```
x = 0
x = 1
```

It will give an error message upon compilation:

```
main.hs:2:1: error:
  Multiple declarations of ‘x’
  Declared at: main.hs:1:1
               main.hs:2:1
```

In Haskell, any "=" statement is a *declaration of a binding*. These bindings are *final* (in the scope of the identifier). It is even wrong to call `x` here a variable since its value does not vary. The correct way to call `x` is *identifier*, since it merely identifies the value bound to it.

Consequences of Statelessness and Immutability

Haskell's functions are better representations of mathematical functions. But what is the point of faithfully representing mathematical functions?

There are several reasons why programming without state can lead to better code. Eliminating all side effects is demonstrably *safer* against

accidental errors. Building a library of functions perfectly working without worrying about side effects makes the system easier to understand and more resilient to changes.

```
void f(int *x, int y){
    *x = *x + y;
    printf("%d\n",*x);
    return;
}
int main(void) {
    int x = 0;
    f(&x, 3);
    x = x - 2;
    f(&x, 3);
}
```

The exact behavior of the function `f()` **depends on where you use it**. Even if you use the same parameters, you're not guaranteed to get the same results.

On small chunks of code, managing the consequences of having states such as global variables, will be trivial since you can reasonably track which variables are global (*or external in general*) and which functions interact with the global variables. Even with a few lines of code like the example, the function's effects and *side effects* are not very obvious.

As the system grows, using functions and variables without double-checking for side effects becomes much harder. As a consequence the whole system becomes a nightmare of impure functions on top of impure functions which may unexpectedly affect other parts of the system. On a corporate setting where multiple people are working on the same system, refactoring becomes *unsafe* without knowledge of all the side effects of the functions in use. On systems with *shared resources* and multi-threading it becomes extra-extra difficult to keep track of things without proper documentation.

Even with all these disadvantages in the state-full mutable paradigm of imperative programming, *one can still write robust and harmonious code.*

You just have to be extra careful writing your code with discipline, *only using global variables and side effects when it is safe and necessary*. This is in fact the reason why *writing smaller pure functions* is considered good practice in any paradigm. After all, being able to code with states can be thought of as an extra feature. You can be a C programmer and just treat all your variables as immutable and all of your functions as pure.

Recursion

Since functional programming does not have the capability to change the state using assignment statement, it does not support your typical for loop. A for loop like the following example, inherently contains state changes.

```
for (int i = 0; i < 10; i++) {
    println("hello");
}
```

In the previous example, `i++` is an assignment statement that changes the value of `i`, thus changing the overall state.

In general, loops need to have some state change. If the state does not change within the loop, the boolean expression will not change as well. Therefore, the loop will either not start or never stop.

```
while (boolean_expression) {
    no_state_change
}
```

To model repetition in a functional language like haskell, we use recursion. Recursion, is repetition through recursive calls. A recursive call, is a function call that happens inside a function definition. The function being called is the same function being defined. A function with a recursive call is called a recursive function. In the definition of the recursive function `summation`, we call `summation` itself.

```
summation :: Int -> Int
summation n =
    if n <= 0 then 0
    else n + summation (n - 1)
```

Recursive functions also have a base case. Depending on the value being passed to the recursive function, it will either evaluate to the base case or the recursive case. Without a base case, the evaluation will never stop due to the infinite recursion. In the `summation` example, the base case is evaluated if the value passed is `0`.

Recurrence Relations

Recursive functions model recurrence relations. For example, the definition of factorial is a recurrence relation itself, so it is easy to convert it into a recursive function in haskell.

$$0! = 1$$
$$n! = n(n-1)!$$

```
factorial :: Int -> Int
factorial n =
    if n == 0 then 1
    else n * factorial (n - 1)
```

When solving problems through recursion, it is best to think of recursive relations that represent the solution of the problem. For example, if you want to create a string of asterisks with a length of n, you can think of a recursive relation that constructs any asterisk of length n. In this case, an asterisk of length n can be constructed by concatenating * and an asterisk of length $n-1$. From that recurrence relation we can define:

```
n_asterisks :: Int -> String
n_astersisk n = "*" ++ nasterisk (n - 1)
```

To complete the recursive function, we just need a base case that represents the simplest and most trivial case for the problem. We can choose the base case: an asterisk of length 1 is *. But we can choose an even simpler case: an asterisk of length 0 is an empty string. By choosing the latter as the base case, we cover both asterisks of length 1 and asterisks of length 0. The simpler the base case, the more complete the function is.

```
n_asterisks :: Int -> String
n_asterisks n =
    if n <= 0 then ""
    else "*" ++ n_asterisks (n - 1)
```

Another example, let's create a function that checks if a list of integers is sorted in an increasing manner. Here's a suitable recurrence relation: a list is sorted if the first element is less than or equal to the rest of the list and if the rest of the list is also sorted.

To make it easier and more readable, let's first create a helper function for one of the tasks: checking if one element is less than or equal a list of elements. This will also be a recursive function.

```
is_not_greater :: Int -> [Int] -> Bool
is_not_greater n list =
    if (length list) == 0 then True
    else (n <= (head list)) && is_not_greater n (tail list)
```

[9]

Here's an explanation of the builtin helper functions used in the definition if `is_not_greater`.

[9] In the base case for `is_not_greater` we define that an element is not greater than an empty list, since any number is indeed not greater than any element in an empty list. This base case also helps for our `is_sorted` definition.

- `length :: [a] -> Int` - returns the length of the list, or the number of elements
- `head :: [a] -> a` - returns the first element in the list
- `tail :: [a] -> [a]` - returns the entire list excluding the first element

With the helper function, `is_not_greater` defined, we can now define `is_sorted`.

```
is_sorted :: [Int] -> Bool
is_sorted list =
    if (length list) == 0 then True
    else (is_not_greater (head list) (tail list)) &&
(is_sorted (tail list))
```

One other paradigmatic design choice being used here is how we are breaking down a problem into subproblems. When there is a complex function you need to define, you can build it up as a composition of smaller functions. In this case, `is_sorted` is built by composing `is_not_greater` in its definition.

You can also define a more efficient `is_sorted` function by checking the sortedness of all adjacent pairs of elements. I will leave the definition for such function as an exercise for you.

Next, let's create a function that returns the index of an element in a list.

Accumulators

```
index_of :: Int -> [Int] -> Int
index_of elem list =
    if (length list) == 0 then error "element not found"
    else if ((head list) == elem) then <index_of_elem>
        else (index_of elem (tail list))
```

In this example, we encounter a problem. How do we find the current index of the element being compared to `elem`? Since we are recursively calling `index_of` to the tail of `list` the current element being compared is always the first element. If this was imperative programming, we

would be able to keep a variable in memory that increments every time we perform a recursive call. This is something that we cannot do in functional programming. To simulate a similar mechanism, we instead add an argument called `index` that always starts as zero, and gets "incremented" for every recursive call.

```
index_of :: Int -> [Int] -> Int -> Int
index_of elem list index =
    if (length list) == 0 then error "element not found"
    else if ((head list) == elem) then index
        else (index_of elem (tail list) (index + 1))
```

For the function to work properly, the `index_of` function must always be called with the argument `index = 0`. Otherwise, it will return an incorrect value.

To ensure proper usage, we can rename our current `index_of` to a different name (e.g. `index_of_inner`), and create an outer function that calls simply `index_of_inner` while passing 0 to the `index` argument.

```
index_of_inner :: Int -> [Int] -> Int -> Int
index_of_inner elem list index =
    if (length list) == 0 then error "element not found"
    else if ((head list) == elem) then index
        else (index_of_inner elem (tail list) (index + 1))

index_of :: Int -> [Int] -> Int
index_of elem list = index_of_inner elem list 0
```

If you really want to hide `index_of_inner`, you can place it inside the scope of `index_of` using a `where` clause.

```
index_of :: Int -> [Int] -> Int
index_of elem list = index_of_inner elem list 0
    where
        index_of_inner elem list index =
            if (length list) == 0 then error "element not
found"
            else if ((head list) == elem) then index
```

```
                else (index_of_inner elem (tail list) (index
+ 1))
```

What we ended up with `index_of_inner` is known as **accumulator passing style** (Sturm 2011). This, not only allows us to simulate a mutating variable in functional programming, it also allows us to perform tail call recursion for optimization.

Tail Call Recursion

When the last evaluation performed in your recursive function is the recursive call, it is known as **tail call recursion**.

Consider the `summation` function previously defined. This is not tail call recursion, because, after `summation (n - 1)` is called, it will still need to evaluate `n + sumamtion (n - 1)`

```
summation :: Int -> Int
summation n =
    if n <= 0 then 0
    else n + summation (n - 1)
```

Let's try expanding tracing the evaluation of such recursive function in the call stack, starting with `summation 3`

```
summation 3
(3 + summation 2)
3 + (2 + summation 1)
3 + (2 + (1 + summation 0))
3 + (2 + (1 + 0))
3 + (2 + 1)
3 + 3
6
```

Every time a function is called, we push a function call to the call stack, freezing the current evaluation. For example, when we evaluate `(3 + summation 2)`, we push the call `summation 2` to the call stack. The evaluation `(3 + summation 2)` is frozen until `(summation 2)` is evaluated. To evaluate `(summation 2)`, `(2 + summation 1)` is pushed to the stack. This freezes `(2 + summation 1)` as well.

This continues until the base case is reached. The call `summation 0` is pushed to the stack, and pops when it evaluates to `0`. The completion of each evaluation, pops each call in the stack, unfreezing the pending evaluations until we evaluate to the final answer, `6`.

Let's compare this evaluation, with a tail call recursive version of `summation`. To achieve tail call recursion, we will need to apply the accumulator passing style.

```
tail_summation :: Int -> Int
tail_summation n = s n 0
    where
        s n sum =
            if n <= 0 then sum
            else s (n - 1) (n + sum)
```

Let's trace the evaluation of `tail_summation` in the call stack.

```
tail_summation 3
s 3 0
s (3 - 1) (3 + 0)
s 2 3
s (2 - 1) (2 + 3)
s 1 5
s (1 - 0) (1 + 5)
s 0 6
6
```

With tail call recursion you don't really need to keep the previous function calls in memory to evaluate the recursive call. This is because the evaluation of every recursive call is the solution itself. For example, the last call `s 0 6` will evaluate to 6, the second to the last call, `s 1 5` will also evaluate to `6`. Because of the tail call recursion, the recurrence relation we end up with the property: `s n sum = s (n - 1) (n + sum)`.

When you compare this with non-tail call recursion: `summation n = n + summation n - 1)`. The evaluation `n + ...` has to be kept in memory

to evaluate to the final answer. Because of this the more recursive calls you have, the more memory you need to use.

When programming languages (not just functional languages) detect tail call recursion, instead of keeping the previous calls in memory, they are automatically discarded. Instead of using $O(n)$ memory for n recursive calls, it only needs to use $O(1)$. This process is known as **tail call optimization** (Abelson et al. 2002).

One caveat for haskell: since haskell is lazily evaluated, tail call recursion may not lead to memory optimization. You might need to force some more optimzation to force haskell to eagerly evaluate expressions and avoid memory leaks.

Extra Reading

Haskell Introduction - for some helpful haskell syntax Recursion and Fixed Point Combinators - to learn how recursion emerges from the lambda calculus formalism

Advantages and Disadvantages of Functional Programming

Advantages

Functional programming's main advantage lies in its stateless paradigm. Without state, you can ensure that your programs are easy to write without worrying about side effects. You can build up complex programs by writing small pure functions and compose them with each to model any algorithm.

Higher-order functions allow functions be very flexible. Combined with partial application, even simple functions can be surprisingly powerful.

Disadvantages

Functional programming has its own set of *disadvantages* as well, most of them related to this seemingly artificial crutch of statelessness and

immutability. The most obvious one is that creating new values instead of changing an existing variable has extra overhead in both processing and memory, making functional programming *slower and less efficient*. Programming without state can be difficult to do for certain mechanisms (*Like the external logger for example*). Simulating mechanisms like this may introduce conflict on how you compose your functions. It's not impossible though, you just have to learn some category theory concepts such as **monads**. Also, for most people who are used imperative programming, *recursion*, does not feel natural compared to *iteration*. But in my opinion, after being exposed to functional programming for some time, recursion can make more sense than iteration.

Nevertheless, these disadvantages are not insurmountable. In fact, there are a plenty of systems written in functional programming languages running without issues. Functional programming has had the reputation of being more conceptual and fancy than the classic imperative programming language. People mocked Haskell for its pristine white tower "elitist" approach to programming. But recently these functional programming languages like Haskell, F# and Scala has enjoyed improvements that have elevated them to be as pragmatic as your classic C, C++ or Java. In fact, functional programming has gained a considerable rise in popularity to an extent that mainstream programming languages with imperative roots like C#, Python, and JavaScript have started to introduce *features patterned from pure functional programming languages*. Features such as **higher-order functions** and **lambdas**. With the risk of sounding editorial I even argue that learning functional programming concepts has become a *necessity* for any programmer, regardless of his/her paradigm preferences.

Logic Programming Formalism

Logic programming is based on the formal system of **predicate calculus**. It is a formalism based on *logical operations* and *quantification*.

Satisfiability and Horn Clauses

Logic programming applies the formalism of predicate calculus as means to prove statements and look for logical resolutions. To introduce these concepts, we will first talk about the satisfiability problem and horn clauses.

Satisfiability

A boolean expression is said to be **satisfiable** if there exists an assignment of truth values (either TRUE or FALSE) that evaluates the entire formula to TRUE. A **boolean expression**, is an expression that uses only boolean values, variables or boolean operations.

Here's a trivial example of a satisfiable boolean expression:

$$p \vee q \vee r$$

This expression will evaluate to true if p, q, and r are all set to TRUE.

Here's a trivial example of an unsatisfiable expression:

$$p \wedge \neg p \wedge q$$

It is not possible for this expression to evaluate to TRUE, since $p \wedge \neg p$ is a contradiction (it always evaluates to false).

These examples are trivial since, the expressions are short enough, or there is an obvious contradiction that resolves the formula. But in general, satisfiability is one of the hardest problems in Computer Science. There are no fast algorithms that can find solutions to a general satisfiability problem.

For example try to check is the following formula is satisfiable:

$$(u \vee \neg v \vee w) \wedge$$
$$(\neg u \vee v \vee p) \wedge$$
$$(\neg u \vee p \vee r)$$

To make it a little easier to solve satisfiability problems, we usually write boolean formulas in **conjunctive normal form** (clausal normal form or CNF). A boolean formula is in CNF if it is a *conjunction of disjunctions of boolean literals*. Each disjunction is also known as a clause (e.g. $(u \vee \neg w \vee w)$). We call unnegated values like u as a positive literal while negated values like $\neg w$ as a negative literal.

Any boolean formula can be expanded into CNF using **logical equivalences**. The equivalent CNF of a boolean formula is easier to solve since we just need to find satisfiable assignments for each clause consistently (which is still not easy).

Horn Clauses

Horn clauses are special clauses where there is at most one positive literal in the disjunction. A boolean formula in CNF where all clauses are Horn clauses is called a **Horn formula**. The satisfiability of a Horn formula is much easier to solve.

$$
\begin{aligned}
&(\neg p) \wedge \\
&(\neg q \vee r) \wedge \\
&(\neg r \vee \neg s \vee t) \wedge \\
&(\neg s \vee \neg t)
\end{aligned}
$$

If each Horn clause in the formula has at least one negative literal, then the formula is *always satisfiable* by assigning FALSE to each variable. This is because the negative literals will all evaluate to TRUE, therefore making each clause TRUE. This Horn formula is a trivial case for satisfiability.

On the other hand if some of the clauses have no negative literals, then we can simply reduce the clauses to either a trivial satisfiable Horn clause or a contradiction.

$$
\begin{aligned}
&(\neg p) \wedge \\
&(\neg q \vee r) \wedge \\
&(s) \wedge \\
&(\neg r \vee \neg s \vee t) \wedge \\
&(\neg s \vee t)
\end{aligned}
$$

In this example, one of the Horn clauses have no negative literal. Note, that a Horn clause with no literal is a clause with exactly one positive literal.

This can be solved by assigning clause (s) as TRUE, since it can be true for the entire formula to be satisfiable.

$$
\begin{aligned}
&(\neg p) \wedge \\
&(\neg q \vee r) \wedge \\
&(\top) \wedge \\
&(\neg r \vee \bot \vee t) \wedge \\
&(\bot \vee t)
\end{aligned}
$$

> In the previous formula, TRUE is denoted by the $\top$ symbol, and FALSE is denoted by the $\bot$ symbol.

We just need to reduce the formula according to logical equivalencies:

$$
\begin{aligned}
&(\neg p) \wedge \\
&(\neg q \vee r) \wedge \\
&(\neg r \vee t) \wedge \\
&(t)
\end{aligned}
$$

After reducing, one of the Horn clauses, (t), has no negative literal. Which means it must be assigned TRUE.

$$
\begin{aligned}
&(\neg p) \wedge \\
&(\neg q \vee r) \wedge \\
&(\neg r \vee \top) \wedge \\
&(\top)
\end{aligned}
$$

Reduced into:

$$
\begin{aligned}
&(\neg p) \wedge \\
&(\neg q \vee r) \wedge
\end{aligned}
$$

This formula is a trivial Horn formula (all clauses have at least one negative literal). The remaining clauses will be assigned FALSE, while s and t will be assigned TRUE. This means that this Horn formula is satisfiable.

In this other example, the Horn formula can be reduced into a contradiction, which means that it is unsatisfiable.

$$
\begin{aligned}
&(\neg p) \wedge \\
&(\neg q \vee r) \wedge \\
&(s) \wedge \\
&(t) \wedge \\
&(\neg s \vee \neg t)
\end{aligned}
$$

First, we assign TRUE to s.

$$
\begin{aligned}
&(\neg p) \wedge \\
&(\neg q \vee r) \wedge \\
&(\top) \wedge \\
&(t) \wedge \\
&(\bot \vee \neg t)
\end{aligned}
$$

Apply logical equivalencies to reduce.

$$
\begin{aligned}
&(\neg p) \wedge \\
&(\neg q \vee r) \wedge \\
&(t) \wedge \\
&(\neg t)
\end{aligned}
$$

Assign TRUE to t.

$$
\begin{aligned}
&(\neg p) \wedge \\
&(\neg q \vee r) \wedge \\
&(\top) \wedge \\
&(\bot)
\end{aligned}
$$

In this case, we end up with a FALSE clause, which means that the conjunction cannot be true. Therefore, this Horn formula is not satisfiable.

Horn Clauses as Implications

Horn clauses also have the advantage of being neatly rewritten as *implication statements*. If you apply the logical equivalency that converts disjunctions into implications. You can convert your Horn clauses into implication statements.

$$\neg p \vee q \equiv p \vee q$$

A Horn clause that has exactly one positive literal can be converted into an implication with the sole positive literal as the conclusion, and the rest as the hypothesis.

$$\neg p_1 \vee \neg p_2 \vee \cdots \vee \neg p_n \vee q \equiv$$
$$(p_1 \wedge p_2 \wedge \cdots \wedge p_n) \rightarrow q$$

A Horn clause with exactly one positive literal and one or more negative literal is known as a **definite Horn clause**.

A Horn clause that is only made up of one positive literal and nothing else is called a **fact**. To convert it into an implication, we first write it as a disjunction using the identity property of disjunctions.

$$\begin{aligned} q &\equiv \bot \vee q \\ &\equiv \top \rightarrow q \end{aligned}$$

Note that, a Horn formula made up of only facts and definite Horn clauses is always satisfiable by assigning TRUE to every variable. By assigning, TRUE to every variable, you ensure that each positive literal is TRUE, meaning each Horn clause is also TRUE.

A Horn clause that is made up 1 or more negative literals, is called a **goal clause**.

$$\neg p_1 \vee \neg p_2 \vee \cdots \vee \neg p_n \equiv$$
$$\neg p_1 \vee \neg p_2 \vee \cdots \vee \neg p_n \vee \bot \equiv$$
$$(p_1 \wedge p_2 \wedge \cdots \wedge p_n) \rightarrow \bot$$

Resolution

Resolution is one of the inference rules of logic. According to resolution, assuming two clauses with complementary literals (a pair of literals that are negations of each other), you can conclude the resolvent. The resolvent is a disjunction of all the non-complementary literals from both clauses.

$$p_1 \vee p_2 \vee \cdots \vee r$$
$$q_1 \vee q_2 \vee \cdots \vee \neg r$$
$$p_1 \vee p_2 \vee q_1 \vee q_2 \cdots$$

In this example, the complements r and $\neg r$ are cancelled out from the resolvent.

Horn clauses are closed over resolution. This means that the resolution of two Horn clauses is guaranteed to be a horn clause. This property ensures that conclusions inferred from Horn clauses will always be Horn clauses. Because of this, it will always be easy to check the satisfiability of Horn formula regardless of any additional conclusions.

As we will discuss later, Horn clauses make up Logic programs. When you are writing Logic programs, you are simply writing a set of Horn clauses.

Logic Programming Paradigm

Introduction

Just as functional programming paradigm is patterned from the formalisms of lambda calculus, logic programming is patterned from predicate calculus. Computer Scientists usually describe families of programming languages under the logic paradigm as a sub-paradigm of declarative programming (*declarative programming being any paradigm that is not imperative*).

Learning Outcomes

1. Create Prolog facts, rules, and queries
2. Explain the process of unification
3. Explain how proof search is used to respond to queries
4. Create recursive Prolog rules

For this section we will use the programming language Prolog as the representative of logic paradigm. Other logic programming families are answer set programming, ABYSS and Datalog.

Facts

There are three basic constructs in Prolog, **facts**, **rules** and **queries**. A **knowledge base** is a collection of facts and rules in the same way a c library or a python package is a collection of function definitions. Prolog programs are basically knowledge bases. Here's an example of a knowledge base:

```
firetype(charmander).
firetype(charizard).
watertype(squirtle).
flyingtype(charizard).
```

Each line of code you can in this particular knowledge base is a fact. Just like facts, in logic, facts in Prolog are propositions that are known to be true. This means that your program knows four things. One of those are:

```
firetype(charmander).
```

In Prolog `firetype(X)` represents a predicate you've learned in discrete math, e.g. firetype(x). And just like predicates, this means `X` is firetype. Therefore, the fact `firetype(charmander)` represents the proposition, "*charmander is firetype*"

So, to summarize, the fact `firetype(charmander)` is basically a representation of the proposition, $firetype(charmander)$ where `firetype/1`[10] is a predicate and `charmander` is a value assigned to the predicate. Every fact in your knowledge base represents propositions that can be assumed to be true. Facts that are not in your knowledge base represents propositions that you cannot assume to be true.

[10] `firetype/1` indicates a predicate named `firetype` with `1` argument.

You can also write non-predicate based propositions in your knowledge base. For example:

```
this_is_a_fact.
axiom_1.
```

Facts, just like other basic Prolog constructs, are implementations of Horn clauses. Specifically, facts represent Horn clauses with one positive literal and nothing else. A fact like, `firetype(charmander)` can be written in the form of an implication as such:

$$\begin{aligned} firetype(charmander) &\equiv firetype(charmander) \vee \bot \\ &\equiv \top \rightarrow firetype(charmander) \end{aligned}$$

Horn clauses like these are called facts because, these clauses are always assumed to be true via modus ponens.

Rules

Aside from facts, you can also define rules in your knowledge base. To illustrate this, let's add one rule to our knowledge base.

```
firetype(charmander).
firetype(charizard).
watertype(squirtle).
flyingtype(charizard).

resistanttofire(squirtle) :- watertype(squirtle).
```

Rules are written with the syntax: **`u :- v`**. This is equivalent to the implication statement $v \rightarrow u$. Prolog rules are written using the "u if v", the reverse of a conventional "if v then u" implication "statement. A lot of people get confused here so just remember, `:-` is read as if. In prolog, we call the conclusion `u` as the rule **head** and the hypothesis `v` as the rule **body**.

A **rule** is an implementation of a definite Horn clause. It is a Horn clause with exactly one positive literal and one or more negative literal.

This will be more obvious if we convert the rule into its disjunction form.

$$watertype(squirtle) \rightarrow resistanttofire(squirtle) \equiv$$
$$\neg watertype(squirtle) \vee resistanttofire(squirtle)$$

Queries

We interact with a knowledge base by writing **queries** to Prolog. You can probably guess what this Prolog construct means just by looking at its name. Queries represent *questions* you ask Prolog. The answer to a question depends on what Prolog knows. And what Prolog knows is represented by the knowledge base. For example if you load the knowledge base we created earlier. We can ask Prolog the following question below. Writing the following will basically ask Prolog, "hey, is it true that charmander is firetype?"

```
?- firetype(charmander).
```

Based on the knowledge base loaded earlier Prolog knows that this proposition is indeed true. Therefore, it responds with:

```
true.
```

If Prolog is asked with the query

```
?- firetype(squirtle).
```

Prolog checks its knowledge base again. Realizing that none of the facts match this proposition, Prolog responds with:

```
false.
```

You can also ask Prolog a conjunction as a query. This is written multiple queries, separated by a comma.

```
?- firetype(charizard), watertype(squirtle).
```

A conjunction of queries can only be true if all queries in said conjunction are true.

```
true.
```

Prolog queries are also implementations of Horn clauses. When you provide a query to prolog, prolog tries to prove that the query is true using **proof by contradiction**. To do this, the query is negated, converting them to a goal. For example, the query `firetype(charizard), watertype(squirtle)`, is negated into the disjunction:

$$\neg firetype(charizard) \lor \neg watertype(squirtle)$$

This negated query or goal is then combined into the knowledge base. If the knowledge base is unsatisfiable with the introduction of the goal, then that means that the goal introduced a contradiction. Therefore, the goal's negation (the original query), must be **consistent** with the knowledge base's assumptions. This ultimately means that it is **true** with respect to the knowledge base.

Here's an example, given the knowledge base:

```
p
q
r :- q
```

And the query:

```
?- r
```

To demonstrate the answer to the query easily, let's convert the clauses in the knowledge base into disjunctions and add the query as a goal[11].

$$(p) \land$$
$$(q) \land$$
$$(\neg q \lor r) \land$$
$$(\neg r)$$

We then check if resulting Horn formula is satisfiable:

[11]When converting prolog rules into definite Horn clauses, the rule body is negated. This is because a rule body can be a conjunction, while a rule head can't. To ensure that the resulting clause is a valid Horn clause, we negate the body.

$$
\begin{gathered}
(\top) \wedge \\
(\top) \wedge \\
(\bot \vee r) \wedge \\
(\neg r)
\end{gathered}
$$

$$
\begin{gathered}
(r) \wedge \\
(\neg r)
\end{gathered}
$$

$$
\begin{gathered}
(\top) \wedge \\
(\bot)
\end{gathered}
$$

This shows that combining our goal with the Horn formula leads to an unsatisfiable formula. Thus proving via contradiction that r is indeed true. With this Prolog appropriately responds `true`.

```
true.
```

Variables

Another important thing about Prolog constructs is that you can write them with **variables**[12]. When you write with Prolog facts or rules, you are implicitly creating a *universally instantiated predicate*. For example, the fact `pokemon(X)`[13], corresponds to the proposition, $\forall x(\text{Pokemon}(x))$. By adding this to the knowledge base, you are assuming that for any value `x`, `pokemon(X)` is true. Therefore, asking the query, `pokemon(charizard)` will yield a `true` response. Of course, any value supplied to the predicate `pokemon/1` will yield true because of the universal quantification.

```
pokemon(X).
```

```
?- pokemon(charizard).
```

```
true.
```

[12]Variables, in the context of logic programming, do not refer to the same variables in imperative programming. These variables represent, mathematical variables that can be free or bound.

[13]Prolog variables must start with either a capital letter or an underscore.

You can also use variables on queries. For example, if we use one of the previous knowledge bases and ask the query: `firetype(X)`.

```
firetype(charmander).
firetype(charizard).
watertype(squirtle).
flyingtype(charizard).

resistanttofire(squirtle) :- watertype(squirtle).
```

```
?- firetype(X)
```

This query basically asks, which values when substituted to `X` in the predicate `firetype(X)` will yield true statements? This can be interpreted in natural language as "which Pokémon are fire type?" Therefore, this query will yield the response:

```
X = charmander
X = charizard
```

Variables used in rules allows the creation of richer knowledge bases. Instead of the rule `resistanttofire(squirtle) :- watertype(squirtle).` we can write a more general rule using variables:

```
firetype(charmander).
firetype(charizard).
watertype(squirtle).
flyingtype(charizard).

isresistantto(X,Y) :- watertype(X),firetype(Y).
isresistantto(X,Y) :- watertype(X),watertype(Y).
```

This introduces a more complicated rule `isresistanto(X,Y) :- watertype(X),firetype(Y)`. This rule's premise is a conjunction of predicates `watertype(X)` and `firetype(Y)`.

If we imagine that the predicate, $isresistantto(x, y)$ means "x is resistant to y", the whole rule can be interpreted as

> for all pairs of X and Y, X is resistant to Y, if X is water type and Y is fire type,

This statement, can be written as the following quantification statement:

$$\forall x \forall y((watertype(x) \land firetype(y)) \rightarrow isresistantto(x, y))$$

By writing this rule, Prolog can infer the following facts:

```
?- isresistantto(squirtle,charmander).
true.

?- isresistantto(squirtle,charizard).
true.

?- isresistantto(squirtle,squirtle).
true.
```

If you ask Prolog a harder question like the following:

```
?- isresistantto(squirtle,X).
```

Prolog interprets this as "which values of `X` make the proposition: squirtle is resistant to X, true? Therefore, Prolog will look for the pokémon, squirtle is resistant to, therefore you with the output:

```
X = charmander
X = charizard
X = squirtle
```

You can even ask Prolog for all possible pairs of resistance relationships in the knowledge base. You can do this by using two different variables for each argument in the predicate.

```
?- isresistantto(X,Y).
X = squirtle,
Y = charmander ;
X = squirtle,
Y = charizard ;
X = Y, Y = squirtle.
```

Unification and SLD Resolution

Unification

There are three types of Prolog terms (Blackburn et al. 2006). By composing these terms you can express rich knowledge bases.

1. Constants. These can either be atoms (known to us as strings such as `squirtle`) or numbers (such as `24`).
2. Variables. Those that start with an underscore or any uppercase letter such as `X`, `Z3`,`_4310`, and `List`.
3. Complex terms. These have the form: `functor(term_1,...,term_n)`. We've seen examples of these in predicates and queries such as `firetype(charmander)` and `isresistantto(X,Y)`

The way Prolog is able to respond to interesting queries involving constants, variables and complex terms is through the **unification**. Unification algorithmically identifies **logically consistent**

substitutions involving constants, variables, and complex terms. Unification in the context of Prolog works along the following rule:

> Two terms unify if they are the same term or if they contain variables that can be uniformly instantiated with terms in such a way that the resulting terms are equal.

This definition gives us the unification of trivial cases such as the unification of constants `squirtle` and `squirtle`. Prolog also unifies the complex terms `watertype(squirtle)` and `watertype(squirtle)` and the variables `X` and `X`. On the other hand, the complex terms `watertype(squirtle)` and `watertype(blastoise)` will not unite.

Prolog also unifies the variable `X` with the constant `squirtle`. Although they are not the same, the variable `X` can be assigned to `squirtle`. By the same intuition, `watertype(X)` and `watertype(squirtle)` will also unify. Unification involving variables, create what is known as an instantiation. Unifying `watertype(X)` and `watertype(squirtle)` will create the instantiation `X=squirtle`.

On the other hand the complex terms `isresistantto(X,charmander)` and `isresistantto(squirtle,X)` do not unify since you cannot find a consistent instantiation of `X`. The instantiation `X=squirtle` evaluates to the terms `isresistantto(squirtle,charmander)` and `isresistantto(squirtle,squirtle)`. The instantiation `X=charmander` makes the terms `isresistantto(charmander,charmander)` and `isresistantto(squirtle,charmander)`.

$$\text{Let} \quad x = \quad \text{squirtle}$$
$$\text{isresistantto}(\text{squirtle}, \text{charmander})$$
$$\text{isresistantto}(\text{squirtle}, \text{squirtle})$$

$$\text{Let} \quad x = \quad \text{charmander}$$
$$\text{isresistantto}(\text{charmander}, \text{charmander})$$
$$\text{isresistantto}(\text{squirtle}, \text{charmander})$$

Both scenarios cannot work because the constants `squirtle` and `charmander` do not unify with each other.

The process of unification can be summarized by the following[^1]:

Two terms a and b unify if and only if

1. a and b are constants, and they are the same number or atom
2. a is a variable and b is any type of term (in this case a is instantiated to b) or b is a variable and a is any type of term (in this case b is instantiated to a). This rule automatically unifies any pair of variables
3. a and b are complex terms and:
    1. They have the same functors and the same number of arguments
    2. all their corresponding arguments unify
    3. the variable instantiations are uniform or compatible (you cannot instantiate x to some constant a when unifying a pair and instantiate x to another constant b when unifying another pair of arguments)

You can demonstrate unification in the Prolog terminal using the predicate `=/2` (this means the = functor with two arguments).

```
?- =(squirtle,squirtle)
true.

?- =(squirtle,charmander)
no

?- =(squirtle,X)
X=squirtle
true.

?- =(X,Y)
X=_5071
Y=_5071
true.
```

[14]

```
?- =(watertype(X),watertype(squirtle))
X=squirtle
true.

?- =(f(g(X),X),f(Y,a))
X=a
Y=g(X)
true.
```

Unification with variables is an implementation of **universal instantiation**. Remember that a `p(X)` in Prolog implicitly means $\forall x p(x)$. When you assume `p(X)` in the knowledge base, you assume that $\forall x p(x)$ which implies via *universal instantiation* that $p(a)$ is also true (for some constant a).

When you have two variables unified to each other, they automatically unify because the actual variable name used does not matter at all. For example, `p(X)` will unify with `p(Y)`, because there is no semantic difference between that universal quantifications $\forall x p(x)$ and $\forall y p(y)$.

Using the same intuition you'll find why the terms `p(X,a)` and `p(b,X)` will not unify. These terms represent $\forall X p(X, a)$ and $\forall X p(b, X)$, which can only consistently instantiate if $X = a$, $X = b$, and $a = b$.

Programming with unification

Unification is crucial with how one can write interesting logic programs. By creating knowledge bases that take advantage of unification, you can generalize structures based on the facts and rules of its characteristics. For example, the following is a knowledge base describing the characteristics of vertical and horizontal lines[15]:

[14]The instantiations `X=_5071` and `Y=_5071` show that both `X` and `Y` are instantiated to the same dummy variable created by Prolog. This is also known as `X` and `Y` being aliased to each other, meaning that they share each other's instantiations.

[15]Note that the functors `line`, and `point` are not predicates. These are simply used as a way to structure the arguments. Only the outermost functors `vertical/1` and `horizontal/1` are considered predicates.

```
vertical(line(point(X,Y),point(X,Z))).
horizontal(line(point(X,Y),point(Z,Y))).
```

Therefore, asking the query:

```
?- vertical(line(point(1,2),point(1,3)))
```

Will yield the response:

```
true.
```

It is indeed a vertical line. And the knowledge base makes sense, since any line that has the same x coordinate is vertical and any line that has the same y coordinate is horizontal.

We can even ask more general queries to Prolog such as:

```
?- horizontal(line(point(2,3),point(U,4)))
```

This query corresponds to asking Prolog for horizontal lines starting at $(2, 3)$ and ends at a point with 4 as the y-coordinate. Prolog attempts the following variable unifications:

```
=(X,2)
=(Y,3)
=(U,Z)
=(Y,4)
```

Individually, these queries unify. But the unification creates inconsistent instantiations, namely, `Y=3` and `Y=4`. These instantiations are inconsistent because `3` does not unify with `4`. Since Prolog can't unify this query with any value for `U` (horizontal lines must have the same y-coordinate), Prolog responds:

```
false.
```

Selective Linear Definite Resolution

Prolog uses the algorithm called **Selective Linear Definite Resolution** (SLD Resolution) to answer queries efficiently. The process is equivalent to checking if the combination of the knowledge base clauses and the

goal leads to an unsatisfiable formula. But this method provides a step-by-step process that can be easily implemented by computers.

Let's start with a simple example:

```
q(a).
s(b).
p(X).

?- p(a), p(b)
```

Given the query, `p(a), p(b)`, Prolog produces a goal by negating the query. The goal is now $\neg p(a) \vee \neg p(b)$. Since the goal is a disjunction, it must prove that a contradiction arises from both $\neg p(a)$ and $\neg p(b)$. This creates two separate sub goals $\neg p(a)$ and $\neg p(b)$. One of the aspects of SLD resolution is how it only *selects* one **subgoal** out of the conjunction of goals. Prolog selects the leftmost subgoal first, in this case, $\neg p(a)$.

Prolog searches the entire knowledge base from top to bottom, and tries to unify the current subgoal with any fact or rule head in the knowledge base. A successful unification, resolves the subgoal away. In this case, $\neg p(a)$, unifies with $p(X)$, creating the instantiation $X = a$.

If the resolution is unclear, you can write $\neg p(a)$ and $p(X)$ as separate clauses[16].

$$p(X) \vee \bot$$
$$\neg p(a) \vee \bot$$
$$\bot \vee \bot$$

This resolves $\neg p(a)$, cancelling it out. Thus, leaving $\neg p(b)$ as the only remaining subgoal to be resolved. Prolog repeats the process for $\neg p(b)$, unifying with $p(X)$ with the instantiation $X = b$. Resolving $\neg p(b)$

[16] $p(X)$ and $\neg p(a)$ are complements of each other through unification. Remember that `p(X)` in Prolog implicitly means $\forall x p(x)$. Through universal instantiation, we know that $\forall x p(x)$ implies that $p(a)$ is true. And we know that $p(a)$ is a complement of $\neg p(a)$.

leaves all subgoals resolved. With all subgoals resolved and refuted, the query is proven `true` by contradiction.

Let's try an example that involves rules on the knowledge base (example from Blackburn et al. (2006)).

```
f(a).
f(b).

g(a).
g(b).

h(b).

k(X) :- f(X), g(X), h(X).
```

With the query:

```
?- k(Y).
```

The goal produced from the query is $\neg k(Y)$.

Current Goal
$\neg k(Y)$

Prolog goes through the entire knowledge base from top to bottom finding unification with the head of `k(X) :- f(X), g(X), h(X)`. The resolution of a rule and a goal leaves a resolvent which serves as new subgoals[17].

$$\neg k(Y) \vee \bot$$
$$k(X) \vee \neg f(X) \vee \neg g(X) \vee \neg h(X)$$
$$\neg f(X) \vee \neg g(X) \vee \neg h(X)$$

With $\neg k(Y)$ resolved away, and the resolvent added to the goals, we are left with the following goal.

[17] This resolution is just modus tollens.

Current Goal
$\neg f(X) \vee \neg g(X) \vee \neg h(X)$

With this new goal, Prolog starts resolving the leftmost subgoal, $\neg f(X)$. This subgoal unifies with two facts, `f(a)` and `f(b)`. Whenever this happens, Prolog creates two branches (one for each possible unification). Prolog tries to complete the refutation one branch at a time in a depth first manner. The other branch will be revisited once one branch is completed.

For the unification to be consistent, the other subgoals must also unify with respect to the instantiation $X = a$. With this, $f(X)$ is resolved away, and the goal narrows down to the following:

Branch $X = a$ goal (current)	Branch $X = b$ goal
$\neg g(a) \vee \neg h(a)$	$\neg g(b) \vee \neg h(b)$

In this case, it tries to complete the branch formed from $X = a$ first. The subgoal $\neg g(a)$ unifies with `g(a)` in the knowledge base.

Branch $X = a$ goal (current)	Branch $X = b$ goal
$\neg h(a)$	$\neg g(b) \vee \neg h(b)$

The subgoal $\neg h(a)$ does not unify with any clause in the knowledge base. Therefore, it cannot be resolved away. This means that the branch for $X = a$ completes without refutation.

Branch $X = a$ goal (not refuted)	Branch $X = b$ goal
$\neg h(a)$	$\neg g(b) \vee \neg h(b)$

From here, Prolog proceeds to resolve the goal of the $X = b$ branch. In this branch both subgoals do unify, with $g(b)$ and $h(b)$. This resolves the goal which completes the branch.

Branch $X = a$ goal (not refuted)	Branch $X = b$ goal (refuted)
$\neg h(a)$	$\bot$

Since one branch is refuted, Prolog responds to the query with `true`. Prolog also shows which instantiations/branches lead to a refutation. In this case, $X = b$ is refuted and since $X = Y$:

```
Y = b,
true.
```

Advanced Constructs in Logic Programming

Recursive Definitions

Similar to functional programming, logic programming represents repetition using recursion. While functional programming makes heavy use of recursive functions to implement complex behavior, logic programming languages like Prolog uses recursive rules to model complex structures. For example: consider the following knowledge base:

```
is_ancestor(Parent, Child) :- is_parent(Parent, Child).
is_ancestor(Ancestor, Descendant) :-
    is_parent(Parent, Descendant),
    is_ancestor(Ancestor, Parent).

is_parent(juan, francisco).
is_parent(cirila, francisco).
is_parent(teodora, jose).
```

```
is_parent(francisco, jose).
is_parent(brigida, teodora).
is_parent(lorenzo, teodora).
```

You'll notice that the rule `is_ancestor` is special since one of its goals is itself. The query below will yield a `true` response due to the recursive nature of the `is_ancestor` rule.

```
?- is_ancestor(juan, jose).
```

The query will match both rule heads, but the first instance (the base case) will lead to an unresolved goal since `is_parent(juan, jose)` cannot be resolved. On the other hand the second instance of the rule (the recursive case), will lead to a propagation of goals that will resolve.

Numbers in Logic Programming

Since logic calculus is a formalism for the foundation of mathematics, how do numbers emerge from predicates and propositions?

This is also another concept shared between, logic calculus and lambda calculus. You can represent numerals (specifically integers) using Peano's axioms:

0 is an integer.

The successor of an integer, denoted by s(n) is also an integer.

You can represent these axioms as a knowledge base:

```
int(0).
int(s(X)) :- int(X).
```

This knowledge base will then define all the possible natural numbers out there, demonstrated by the query:

```
int(X)
```

```
X = 0
X = s(0)
```

```
X = s(s(0))
X = s(s(s(0)))
...
```

Using this representation, you can then define arithmetic operations such as addition and multiplication (also based on Peano's axioms) (Hosch 2010).

$$a + 0 = a$$
$$a + b = c \rightarrow a + (b + 1) = (c + 1)$$

$$a * 0 = 0$$
$$a * b = d \wedge a + d = c \rightarrow a * (b + 1) = c$$

```
int(0).
int(s(X)) :- int(X).

add(A,0,A).
add(A,s(B),s(C)) :- add(A,B,C).

mult(_,0,0).
mult(A,s(B),C) :- mult(A,B,D), add(A,D,C).
```

While this representation of numbers is a good way of demonstrating how numbers can be defined using logic, it's not really that usable. We want to numbers to be easily readable, that's why we use base-10 Arabic numerals to represent numbers. Prolog does have a builtin representation for numbers. Numbers are accepted terms in the form of constants. You can also apply some predicates to numbers like =, <, >[18].

```
positive(X) :- X > 0.
```

```
?- positive(2).
true.
```

[18]Note that equality and inequality operators are also predicates. In the examples here they are written as infix operators, but you can still use them using the prefix functor syntax (i.e. `=(2,3)`)

Unfortunately, these representations are also very limited. Predicates like =, <, and > break the logical nature of Prolog. If you try to use these predicates with uninstantiated variables, you will end up with an error.

```
?- X = 2.
Arguments are not sufficiently instantiated
In:
   [1] 2>_2258
```

To solve this limitation, the library `clp(fd)` and `clp(z)` was created. The library `clp(fd)` stands for **Constraint Logic Programming Over Finite Domains**. This library offers predicates that can be used to apply logical reasoning on integers. This library was further refined to a more complete and more advanced library called `clp(z)` or **Constrained Programming Over Integers**.

These libraries include special predicates known as constraints. **Constraints** are predicates that restrict a variable to a specific set of values. Constraints are usually applied to *numerical variables* to define the domain of said variable. The libraries, `clp(fd)` and `clp(z)` include the equality and inequality constraints (Triska 2026).

- `X #= Y`: $x = y$
- `X #< Y`: $x < y$
- `X #> Y`: $x > y$
- `X #\= Y`: $x \neq y$
- `X #>= Y`: $x \geq y$
- `X #=< Y`: $x \leq y$

To import libraries in your knowledge base, use the headless rule syntax[19].

```
:- use_module(library(clpfd)).
```

[19]Older implementations, like `swi-prolog` comes with `clpfd` but not `clpz`. Newer implementations like `scryer-prolog` come with both.

You can import it on the REPL by directly writing the `use_module` predicate as a query. With the library loaded in the repl, you can use the constraints on your queries.

```
?- use_module(library(clpfd)).
true.
```

When you use the `#<` constraint with a variable, it restricts said variable's domain to satisfy the constraint.

```
?- X #< 3.
X in inf..2.
```

In the example above, with the constraint `X #< 3`, the integer variable `X` can only have values in the range $[-\infty, 2]$.

When used with numerical expressions, you can use Prolog as solver.

```
?- 4 #= 2*X + 2.
X = 1.
```

Using constraint programming, will be limited when used as a solver since the domain is finite and you the answers are based on relations. If the solution to your equation is not an integer, then the variable `X` doesn't have a valid integer that satisfies the constraint[20]. In such case, Prolog will respond with `false`

```
?- 0 #= 2*X + 3.
false.
```

You can combine multiple constraints into a conjunction to further control the domain of an integer. In the example below, the values of `X` can only be in the range $[3, 11] \cup [13, \infty]$[21].

```
?- X #>= 3, X #\= 12.
X in 3..11\/13..sup.
```

[20]There are libraries that support constraint logical programming over real numbers like `clpr` which also comes bundled with `swi-prolog`.

[21]`inf` stands for infimum, the smallest value in the domain and `sup` stands for supremum, the largest value in the domain.

You can also directly use the `in` predicate to create constraints over a domain expression. In the example below, the domain ranges `0..10` and `9..12` are combined using the union operator, `\/`.

```
?- X in 0..10 \/ 9..12.
X in 0..12.
```

To enumerate values in a domain, you can add the predicate `indomain/1` with a conjunction.

```
?- X in 0..10, 1 #\= X mod 2, indomain(X).
X = 0 ;
X = 2 ;
X = 4 ;
X = 6 ;
X = 8 ;
X = 10.
```

Data Structures

As discussed before, Prolog support compound terms in its syntax. Depending on where a term appears in Prolog code, it can be treated as a predicate with arguments, or a term representing data.

```
a(X).
b(c(d(Y)), e).
f(X) :- g(h), i(j).
k.
```

If a Prolog term appears as a fact, the head of a rule, or in the body of the rule, it is treated as a predicate with arguments. In the example above, `a/1`, `b/2`, `f/1`, `g/1`, `i/1`, `k/0` are considered as predicates. On the other hand, if a term appears as a predicate argument or a term argument, then it is treated as a term representing data. In the example above, `X`, `c(d(Y))`, `d(Y)`, `Y`, `e`, `h`,`j`, are considered terms representing data.

Compound data

Compound terms can be used to represent compound data, for example, you can represent a compound data structure that represents identity, using the generic term `id(Name, Age)`.

```
:- use_module(library(clpfd)).

person(id(artman, 16)).
person(id(bartman, 19)).
person(id(cartman, 21)).
adult(Name) :-
    person(id(Name, Age)),
    Age #>= 18.
```

Lists

Prolog uses a special representation for lists. An **empty list** is represented by the special atom `[]`. A special functor can be used to denote recursively from here. For example, we can define use `list/2` to denote any list. Any term defined as `list(Head, Tail)` is a *list*, if `Tail` is a *list*.

With this you can represent a list with *one element* as `list(elem1, [])`. You can define a list with *two elements* as `list(elem1, list(elem2, []))`.

As the number of elements grow, this representation can be cumbersome, so Prolog uses a special syntax to lists with any number of elements using comma separated terms. For example, a *list with four elements* can be written as `[elem1, elem2, elem3, elem4]`. Prolog still allows us to use the *head-tail* pattern for defining lists as `[Head|Tail]`. Such a list is valid if `Tail` is also a *valid list*. For example, the four element list can be represented as `[elem1 | [elem2, elem3, elem4]]`.

```
?- =([elem1, elem2, elem3, elem4], [elem1 | [elem2, elem3,
elem4]]).
true.
```

You can also combine, comma separated lists with the head-tail pattern as such:

```
?- =([elem1, elem2, elem3, elem4], [elem1, elem2 | [elem3,
elem4]]).
true.
```

You can use the `[Head|Tail]` pattern to create interesting predicates involving lists. For example, you can find the length of a list by creating the following rule:

```
:- use_module(library(clpfd)).

list_length([], 0).
list_length([Head|Tail], Length) :-
    Length #= Tail_length + 1,
    list_length(Tail, Tail_length).
```

```
?- list_length([a,b,c,d],X).
X = 4.
```

This predicate is very similar to the way a list's length is calculated in functional programming. But it can be used in more interesting ways. For example, we can create the predicate `list_nth_element/3`. Given, `List`, `N`, and `Elem`, `list_nth_element(List, N, Elem)` is true if `Elem` is found on index `N` in the list. To create this predicate, we first define a helper predicate called, `list_take_n/3`. Given `List`, `N`, `Leftover`, `list_take_n(List, N, Leftover)` is true if, `Leftover` is the list you end up with after taking `N` elements from `List`.

```
:- use_module(library(clpfd)).

list_take_n(List,0,List).
list_take_n([Head|Tail], N, Leftover) :-
    N #> 0,
    M #= N - 1,
    list_take_n(Tail, M, Leftover).

list_nth_element(List, N, Elem) :-
    list_take_n(List, N, [Elem|Tail]).
```

With this `list_nth_element/3` created, we can find the nth element of a list.

```
?- list_nth_element([a,b,c,d,e],2,X).
X = c.
```

But at the same time, we can also use `list_nth_element/3` to check which indices an element is at.

```
?- list_nth_element([a,q,c,q,e],Index,q).
Index = 1 ;
Index = 3 ;
false.
```

[22]

We can write such queries because we are working with predicates that act as *pure relations*. Pure relations, unlike functions *do not have a set input and output*. A correctly written predicate should work no matter which argument is replaced by a variable.

[22]The extra `false` branch is a result of SLD resolution unifying the query to both the fact and the rule head in the knowledge base.

Advantages and Disadvantages of Logic Programming

Advantages

Logic programming shares a lot of similarities with functional programming. It also shares its advantages and disadvantages as well. Both paradigms offer a safer and more consistent framework since they are both patterned form mathematical formalisms. Functional programming has lambda calculus and logic programming is based on *first-order predicate calculus.*

Being non-imperative also gives them an edge of automatically being *immune to the perils of state* and at the same time being *prone to the perils of its absence.*

Beyond the general advantages of declarative programming, Logic programming can be a powerful solver in problems like **automated theorem proving**, **constraint related search problems**, **domain specific expert systems** and more. The straight forward way of listing facts and rules makes it suitable for representing complex information that can be usually found in these use cases. The beauty of unification and SLD resolution shines on these domains as they often require, complex representation involving multiple interconnected rules, recursive structures, and constraints.

Logic programs also use predicates as *pure relations*. Because of this, the predicate definitions can support wide uses.

Disadvantages

Logic programming's way of expressing knowledge gives it a lot of niche uses. Most of the distributions Prolog are admittedly meant for a *limited* use cases only. You are not going to use it for web-development, embedded systems, or scripting.

Logic programming's disadvantages are also similar to functional programming, but can be much *worse*. The obvious *inefficiency* due to the absence of state is much more evident in logic programming because of the thorough approach of backtracking in SLD resolution. The *strangeness* of the paradigm itself as compared to the imperative way of thinking is also felt more in imperative programming. At least functional paradigm and imperative paradigm share the concept of functions Prolog only has predicates. To get used to writing logic programs, you have to get used thinking in terms of *relations* and *declarative definitions*.

Just like functional programming though, the spirit of logic programming can be found in other paradigms through the existence of unification libraries. Although logic paradigm is admittedly less relevant than other paradigms, its strange features are definitely useful and worth studying.

Object-Oriented Programming Paradigm

Introduction

As procedural programming became more and more mainstream, computer scientists started to notice the issues behind states and side effects. Some languages offered a complete paradigm shift, completely abandoning the notion of state. This exodus formed the alternative paradigm family, declarative paradigm. Other languages however, went on a different direction. These languages offered a solution to fixing state by introducing richer features and an intuitive design. From this approach the paradigm object-oriented programming was born.

Learning Outcomes

After this discussion you should be able to

1. Explain how programmers started to focus on writing maintainable code

2. Differentiate between the object and the class
3. Explain OOP's philosophy, the surface vs. the volume
4. Explain what abstraction means
5. Describe the fundamental concepts: encapsulation, inheritance, and polymorphism

---

As time went by and as technology evolved, computer scientists noticed new issues on the code they were writing. The goal of building the most optimized machine code became less and less of a priority. Programmers had, under their disposal, faster and better hardware. As the demand for complex systems grew, the priority shifted from *code that run*s, to *code that was intuitive*. As programmers started to build bigger and bigger systems, they started to notice the issues in terms of maintainability.

This shift in focus can be seen from the extremes of procedural paradigm such as the programming language Assembly. Assembly code was not really built to be intuitive. It was built to reliably work. Assembly contained mechanisms to move data around and to control program flow. These features were enough for that time since the objective of programming was to write efficient code that bulky slow CPUs understand. Assembly programmers did not have time to worry about code elegance or intuitiveness.

The landscape started to change when hardware started becoming better and software started becoming more complex. Speed and memory wasn't that much of an issue anymore, so computer scientists' focus shifted towards the issue of maintainability. Software became bigger and more complex and understanding other programmers' code became more and more difficult. In fact, understanding your own code was also becoming difficult. Because of this writing code that a computer understands isn't enough anymore, a good programmer must write code that humans understand as well. Code shifted from computer centered to human centered.

But instead of redesigning the concept of imperative paradigm to solve maintainability, object-oriented programming sought to build on top of the features of imperative programming. State despite its known issues, still exists but OOP gave imperative programmers extra tools to protect code from being carelessly mutated. Because of this OOP stayed under the imperative family since the programmer is still focused on telling the computer what to do using assignment statements and mutation.

Fundamental Concepts of OOP

Object-oriented programming is usually defined using its four core design principles:

- Encapsulation
- Abstraction
- Inheritance
- Polymorphism

It is said that any programming language that contains these mechanisms is an object-oriented programming language. But calling the entirety of a programming language, OOP can be problematic since you are free to write code written in an "OOP language" without actually using these design principles. At the end of the day OOP is not merely a language classification, but a paradigm. Some programming languages are indeed intentionally written to be used for OOP design, but these classifications are less important than judging if the code itself written by the programmer adheres to OOP's design principles.

The Object

Let's start with the star and the building blocks of object-oriented programming. What exactly is an object?

An object is a living organism in your code. To grasp the capabilities and limitations of an object we will anthropomorphize objects. Treating an object as an organism will guide you on how you use objects effectively.

Just like any creature an object has both form (attributes/fields) and behavior (methods) (this is one of the reasons why C's `struct` is not an object since `struct` instances only have form). Objects are written to be representations of real world nouns such as a person or an employee or a file. The best way to design objects is to simulate the real world form and behavior of what these objects represent. Think of objects as the **representatives** of things in your code. Since you cannot shove an actual employee in your system, you instead create a representative of that employee using an employee object. If you start thinking about objects like representatives instead of mere data holders you'll be able to design a safe and elegant object.

The Class

Often discussed alongside an object is the class. There's usually a lot of confusion when identifying the difference between a class and an object, and it is probably because in the universe of your code, the class and the object will have the same name.

A class is the specifications for the creation of objects. If you want to give a class a more proactive role, you can think of the class as the factory that builds objects. There are a lot of analogies out there to illustrate the relationship of classes and objects. For example, a star shaped cookie cutter (class) that makes star shaped cookies (objects). If you want to make star shaped cookies you use the star shaped cookie cutters. You can also call objects as instances of a class. In fact that's what I call objects most of the time. For example, the Cash class, `$2.75` is an instance of a Cash object. Or a cat named Garfield is an instance of a class called Mammal.

If an object is a representation of a tangible real world object, then the class is the conceptual type/category of that real world object.

The following class diagrams are the representations of a book and an Employee. The attributes of the book are `title`, `author`, `publishDate`, and `pages`. These attributes also simulate the form of a real world book. This object also has methods called `ISBN()` and `numPages()`. The

attributes of an employee are `name`, `string`, and `salary`, and it has a method called `reassignJob()`.

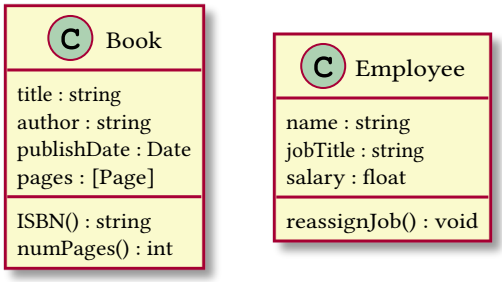


Figure 4: Class diagram of a book class

The following are representations of objects:

```
book1{
title: "Corpus Hermeticum"
author: "Hermes Trismegistus"
publishDate: Dec 12, 2008
pages: ......
}

book2{
title: "Behold a Pale Horse"
author: "Milton William Cooper"
publishDate: Dec 1, 1991
pages: ......
}

employee1{
name: "Rubelito Abella"
jobTitle: "Instructor 1"
salary: [REDACTED]
}
```

The Surface and the Volume

Data Hiding and the Interface

Before we play around with some code, lets dive deep into the philosophy of object-oriented paradigm. The base premise of OOP is the concept called **data hiding**. And this formal OOP term describes what I mean when I say that, what OOP did for imperative programming was to allow the programmer to create artificial boundaries between irrelevant data and behavior. In the eyes of an OOP design, procedural code is a mix of data and functions arbitrarily tossed in a spaghetti of mutations and side effects.

OOP's mechanism to create boundaries in the form of objects brought structure to imperative programming. All the attributes of a `Book` object doesn't need to be accessed by the methods of a `LibraryCard` object. That's because in the real world, a library card doesn't need to know what's written on page 69 of some book. Some of the data in `Book` should be **hidden** from a client object `LibraryCard`.

Object-oriented programming's obsession to simulate real world objects is not caused by its dream to become an exact representation of the real world. Object-oriented programming obsesses over structure and simulation because it is a necessity for human comprehension and therefore, maintainability. Systems need good structure not because well-structured code is pleasant and elegant to look at, but because our feeble minds can't process poor structure efficiently. In fact the reason why we call a well-structured piece code, "elegant" is because our tiny limited minds can process it well.

Trying to read this code would be comparable to looking at some Lovecraftian cosmic horror. If such a code exists it'll probably contain the secrets of the universe.

The process of modeling elegant object representations is basically determining what the **public interface** of that object may be. The interface of an object is the set of attributes and methods (methods are what we call functions that inside a class) that other objects can use to

interact with it. The attributes and methods that are not in the interface is essentially hidden information, inaccessible from the client objects. For example, given a `Book` object that interacts with a `LibraryCard` object, the `LibraryCard` object's interface to interact with the book may only contain, `title` and `author`, because this is all the information it needs to properly do its job. You can even hide those attributes and let the `LibraryCard` only interact with the `Book` using a `borrow()` method which lists the `Book` as borrowed in the library card. It doesn't need access to the attributes of the book to do this. It interacts with the Book itself as a whole.

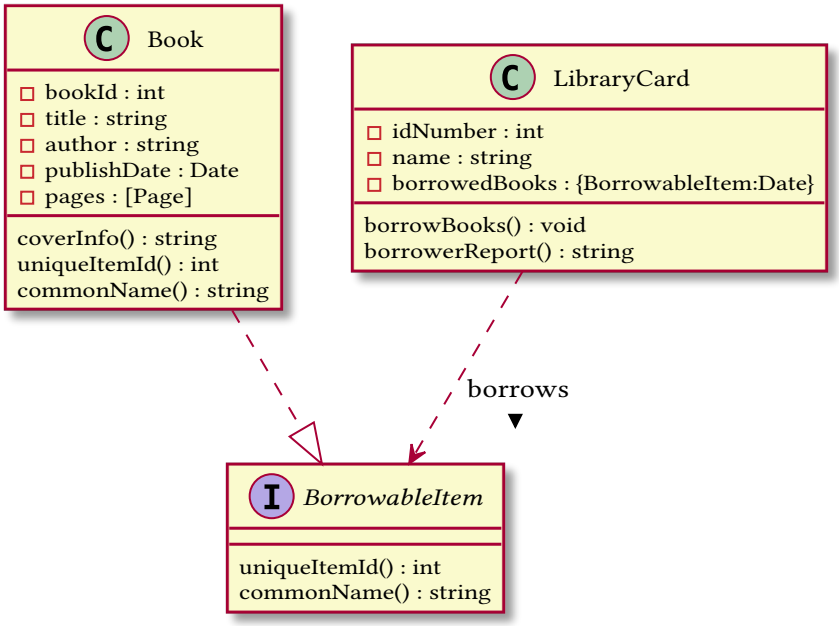


Figure 5: Class diagram of a book class

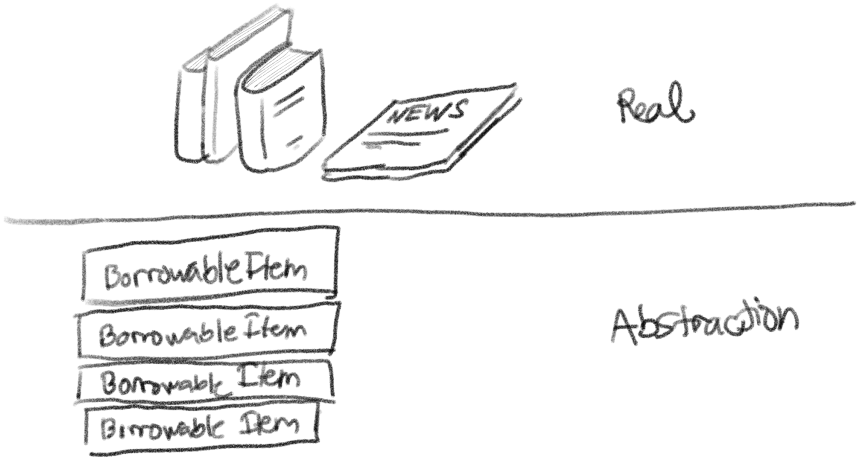


Figure 6: abstraction borrowable item

Abstraction of Objects

Creating interfaces like these provide OOP with the mechanism to create **abstractions**[23] in the object level.

An abstraction in computer science is basically a model of computation that is free from its implementation. In the same way that functional programming creates abstractions of mathematical functions by writing lambdas without side effects, OOP creates abstractions of objects using interfaces that don't specify the exact implementation of an object.

In the example above, the interface `BorrowableItem` is an abstract representation of *something from the library that can be borrowed.* An interface like `BorrowableItem` contains method names and type signatures, but it doesn't actually contain code. That is because a `BorrowableItem` is an abstract representation. We are not supposed to care about the implementation of the methods `uniqueItemId()` and `commonName()` all we should care about is that `uniqueItemId()` should return some `int` and `commonName()` should return a `string`.

[23]The term abstraction is actually quite overloaded in OOP and CS in general. The "abstraction" I'm talking about in this context is the concept of abstract interface.

The reason why this structure still works, is because we have a concrete class called `Book` which is a **realization** or an **implementation** of `BorrowableItem`. A book is *something from the library that can be borrowed*. Because a `Book` is a `BorrowableItem`, it must also behave based on the specifications of a `BorrowableItem`. Meaning it must contain the methods `uniqueItemId()` and `commonName()` (which should also have the same type signature as the methods of `BorrowableItems`). Since `Book` is a concrete class it's methods `uniqueItemId()` and `commonName()` should be implemented (meaning there should be code inside these methods).

Why even go through all this trouble? If *something from the library that can be borrowed* is an abstract idea, what is the point of modelling its representation? This looks like extra code just to represent something that doesn't really have an exact form in the real world.

For the current structure we created, this feels like extra code because our system is small enough right now. But imagine if our system grows, and we need to incorporate other things from the library that are not books but can be borrowed. For example, a library also contains periodicals that you can borrow as well, and these periodicals do not follow the form of the book. You need a different representation for a periodical, therefore you need to create a new concrete class called `Periodical`. Since a periodical is also *something from the library that can be borrowed*, a periodical is another **realization** of `BorrowableItem`. And with the tiny effort of writing the implementation of a periodical (including the realized methods `uniqueItemId()` and `commonName()`), we added an extra interaction that allows a `LibraryCard` to borrow periodicals as well.

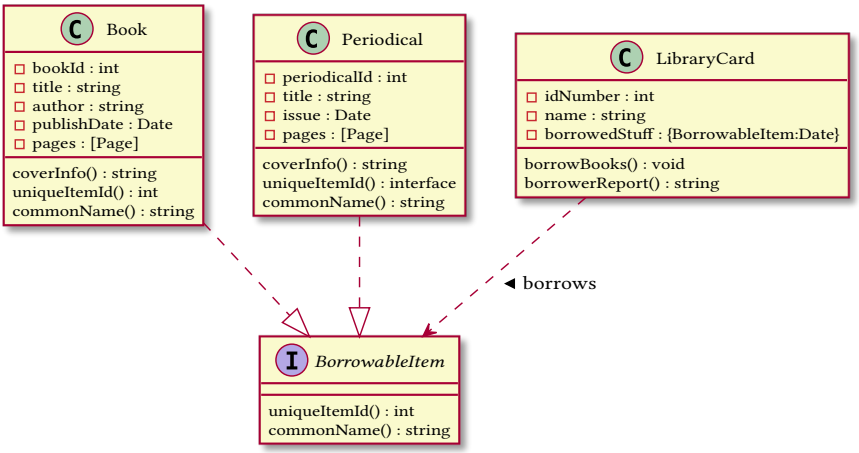


Figure 7: Class diagram of a book class

The Interface and the Implementation

Let's summarize what we learned so far by discussing how these capabilities characterize the philosophy of OOP. The paradigm aims to solve the issues of state and maintainability by allowing programmers to create boundaries between its mix of attributes and methods. The boundaries you enforce are basically the object structure you create. A library card name shouldn't mix with a book title, so we put a boundary between them by **encapsulating** them into their respective objects.

You can imagine these objects as amorphous blobs with surface separating the methods and attributes inside it from other things in your code. These amorphous blobs have volume and surface. The volume of these blobs represent the **implementation** of these objects and the surface of these blobs represent the **interface** of these objects. The interaction, between two objects is characterized by the surfaces of objects, the implementation. The volume of the object shouldn't dictate how the objects relate to each other, in fact everything inside the object should be inaccessible to other objects. Objects should only see each other's surface. This means that the interaction between objects should be defined by their interface not their implementation.

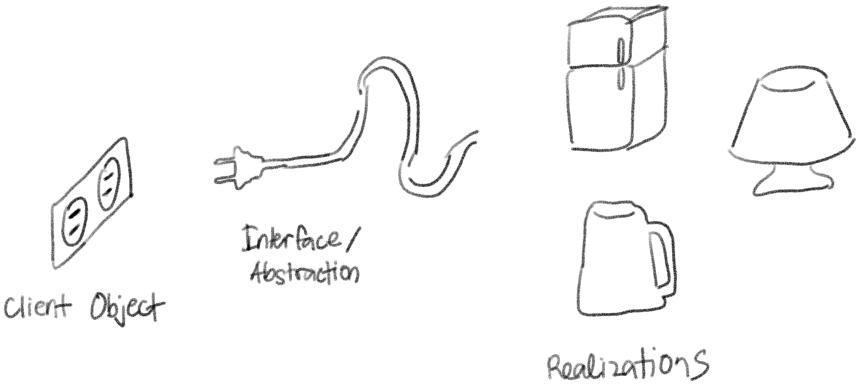


Figure 8: Abstraction borrowable item

Your job as an OOP programmer is to make sure that the complexity of the surface grows slower than the complexity of the core. This means that as your system evolves, changes that happen in the core, the implementation hidden inside each object, (as much as possible) shouldn't affect the surface, the interfaces of each object. This is what harmony and elegance in OOP means. Objects interact with each other seamlessly, and intuitively, regardless of what their inside look like.

Fundamental Concepts of OOP

Encapsulation

One of the most important design principle of object-oriented programming is the concept called encapsulation. I've said it again and again, and I don't mind saying it again right now, oop's innovation that made the paradigm a solution to the issues of state is its mechanism to construct boundaries wherever you want (you should want to put it between irrelevant data). This mechanism is also called **encapsulation**. Here are a few important points to remember:

- Encapsulation, when done correctly, makes your system approach a more accurate simulation of the real world. The more you encapsulate related data and methods, the more you'll create cohesive classes that have definite and indivisible purpose.
- You should **encapsulate what varies**, meaning, things that always change should be encapsulated deep into the structure of your code.

This will help with maintainability since the changing isolated data or behavior will have less impact to the whole system.

- Encapsulation means both attributes and behaviors. Concrete objects should be given the responsibility of implementing their own behavior. This means that a method that describes the behavior of a certain class should belong to that class.

Which implementation is better?

Tax rate as a global variable

```
TAX_RATE = 1.1

product1_price = 100
product2_price = 35
exempt_product_price = 22 #exempted from tax

final_price += product1_price * TAX_RATE
final_price += product2_price * TAX_RATE
final_price += exempt_product_price
```

Tax rate hard coded for every instance of use

```
product1_price = 100
product2_price = 35
exempt_product_price = 22 #exempted from tax

final_price += product1_price * 1.1
final_price += product2_price * 1.1
final_price += exempt_product_price
```

A function called `taxedPrice()` which calculates tax

```
TAX_RATE = 1.1

def taxedPrice(price):
    return price * TAX_RATE

product1_price = 100
product2_price = 35
exempt_product_price = 22 #exempted from tax
```

```
final_price += taxedPrice(product1_price)
final_price += taxedPrice(product2_price)
final_price += exempt_product_price
```

Hardcoding tax calculation for every instance is the least encapsulated solution. This solution is the least future-proof version since changes in tax rate will require changing every instance of tax calculation manually as well. To improve upon this we can encapsulate tax rate into a global variable. Anytime there are changes to the tax rate, we simply change the value of `TAX_RATE` and all tax calculations will be updated as well.

This can be improved even more by encapsulating the tax calculation deeper into its own helper function. This version makes it more resilient to tax policy changes. Maybe in the future, tax calculation becomes more complex than simple multiplication. If this does happen our code is ready to accept the change by simply changing the body of `taxedPrice()`.

As seen here, we see how tax calculation is something that has potential to change. It is volatile. As mentioned earlier, it is prudent to encapsulate volatile code to make it easier to update.

Inheritance

Another important design principle in OOP is the concept of **inheritance**. Inheritance is the concept in which the definition of a class is derived from another class. An existing class, called the **super class** (also called the **base class** or the **parent class**) passes all visible attributes and methods to a **subclass** (also called the **derived class** or the **child class**).

The concept of inheritance is also a representation of the real world. You use inheritance to represent generalizations and specializations. A super class is a generalization of a subclass and a subclass is a specialization of a super class.

In this example the supertype animal is a generaliztion of the subtype mammal. Although it isn't shown, `Mammal` will also have the attributes `name` and `weight` and the method `sound()` since it inherits these from the parent class. Mammal has a method of its own called `lactate()` which it doesn't share with animal.

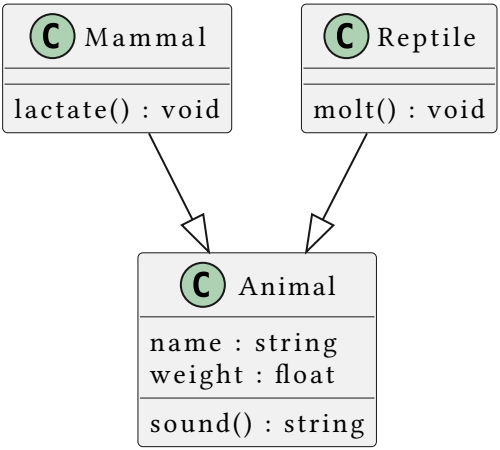


Figure 9: Inheritance

A subclass can also be a super class for another class. This is used to represent specializations of specializations.

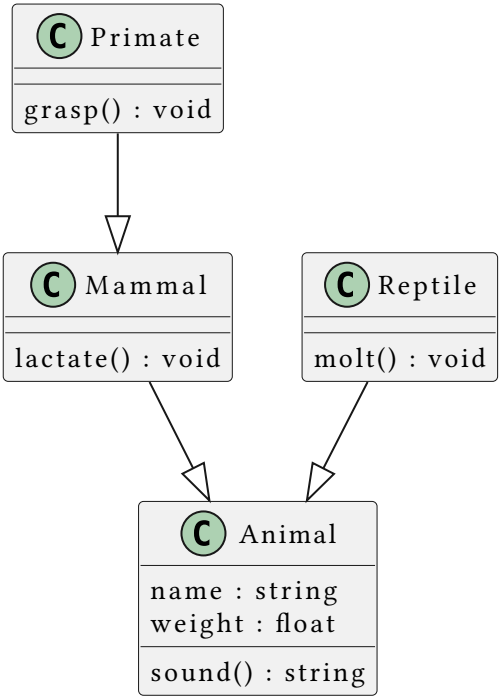


Figure 10: Inheritance

The class primates will then inherit all visible attributes and mehtods of `Mammal` which include those that are inherited from `Animal`.

Some programming languages will allow you to add restrictions to the inheritance of an attribute or a method. Languages like `C++` or `Java` does this using the modifiers `private` and `protected`

super class visibility	public derivation	protected derivation	private derivation
`public`	`public`	`protected`	`private`
`protected`	`protected`	`protected`	`private`
`private`	*not inherited*	*not inherited*	*not inherited*

Polymorphism

Polymorphism literally means *multiple forms*. One of the core philosophy of OOP allows object instances to exist in multiple forms. What this means code-wise is that the types of object instances can be decided during runtime.

Compile-time polymorphism

There's also another type of polymorphism that is not necessarily shared by all OOP languages, compile-time polymorphism. This is basically the feature where multiple functions can have the same name as long as they have different parameter type signatures. We also call this feature as method overloading or dynamic dispatch.

Run-time polymorphism

Run-time polymorphism on the other hand, is basically achieved using specialization and realization relationships between objects. This is usually what Polymorphism refers to in the scope of OOP.

For example, An object instantiated to be of type `Primate` is also an instance of `Animal` because a primate is just a specialization of an animal. This is a reflection of how the real world works because a primate is indeed an animal. On realization relationships like a `Book` and a `BorrowableItem`, the same is also true, because a book is also something that can be borrowed. Realization and specialization relationships guarantee that you can interact with a subtype as its super type, and you can interact with concrete class as its abstraction.

SOLID Objects

Introduction

SOLID is an acronym describing design principles for creating OOP systems. By committing your code to these principles you will naturally build systems that don't only work perfectly, but work elegantly.

Learning Outcomes

1. Design methods and classes with exactly one responsibility
2. Design extension to classes to incorporate new behavior
3. Design proper realization and specialization relationships
4. Design purposeful abstractions and abstract methods
5. Design systems that interact through abstractions

Briefly, these five principles are:

- **Single Responsibility Principle** - Objects should have cohesive and complete responsibilities. It shouldn't be aware of knowledge it doesn't need, and it shouldn't perform responsibilities that are irrelevant from it.

- **Open/Closed Principle** - Classes should be open to extension and closed to modification. Instead of changing the form and behavior of an existing class, you should extend the class
- **Liskov-Substitution Principle** - Substituting objects by their subtypes/realizations should always work.
- **Interface Segregation Principle** - A client shouldn't be forced to implement methods that it doesn't use
- **Dependency Inversion Principle** - Object relationships should depend on abstractions instead of implementations

Single Responsibility Principle

The GOD Class

One of the canonical examples of violations against SRP is the concept known as the **god class**. A god class is a class that basically contains all the attributes and methods of the whole system. You'll recognize these god classes as those classes that control the behavior of objects (they contain the implementation of client objects' behavior). These god classes are also aware of all the objects secrets (they expose and manipulate private attributes and methods).

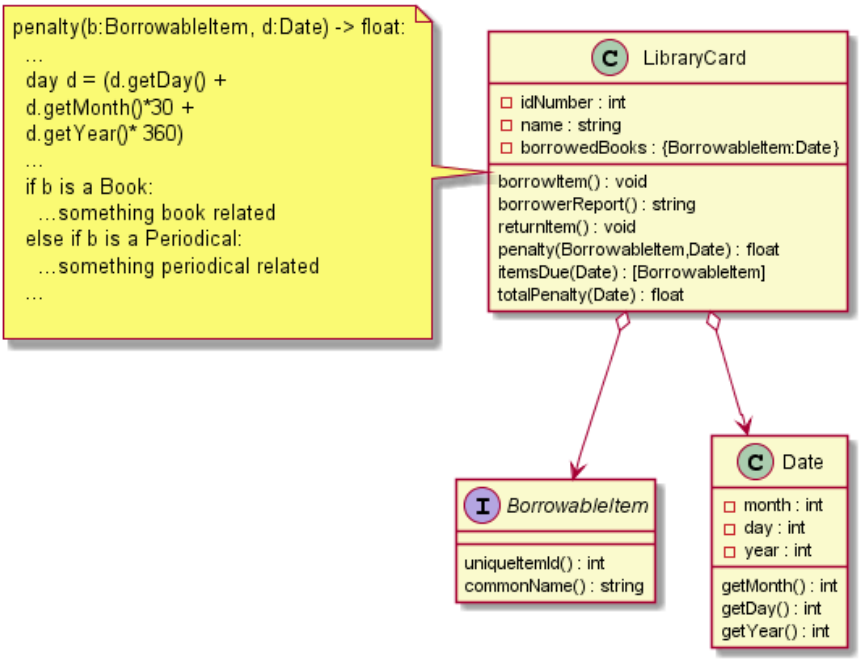


Figure 11: God class library card

On the example above `LibraryCard` is a god class since it exposes the secrets of `Date` by forcing the creations of *evil* getters (`getMonth()`, `getDay()`, `getYear()`) for otherwise private details. Although it isn't obvious, it also tampers on the responsibilities of `BorrowableItem` by deciding by itself how penalty is calculated for each realization.

The moment you ask an object what it's exact type is, you should consider refactoring your code since introspective checks like `isinstance`, `instanceof`, `typeof`, etc. are symptoms of smelly design. You can think of these checks as analogies to racial discrimination since your client object asks the race (type) of the dependency.

Assigning the correct responsibilities

When you're introducing new behavior or information to a system, you should ask first: *who should be responsible for this behavior or information?*

- Who should be responsible for calculating the differences between dates? **`Date` should be responsible, not `LibraryCard`**.
- Who should be responsible for calculating the penalties of specific `BorrowableItem` realizations? **`BorrowableItem` realizations should be responsible, not `LibraryCard`**

The best implementation of the system contains multiple examples of SRP. The `Date` class should be responsible for subtracting dates and adding dates, not the `LibraryCard` class. Forcing `LibraryCard` to contain code for `Date` operations will force you into breaking the boundaries of your classes (e.g. creating getters to expose private attributes). `LibraryCard` should not be aware of **how** dates are subtracted to be able to subtract dates. The same is true for a `BorrowableItem`'s due date. The `LibraryCard` shouldn't contain the specifications as to how `BorrowableItems` calculate their due date. In fact `LibraryCard` shouldn't even be aware of the exact type of the `BorrowableItem` (`isinstance` violates this).

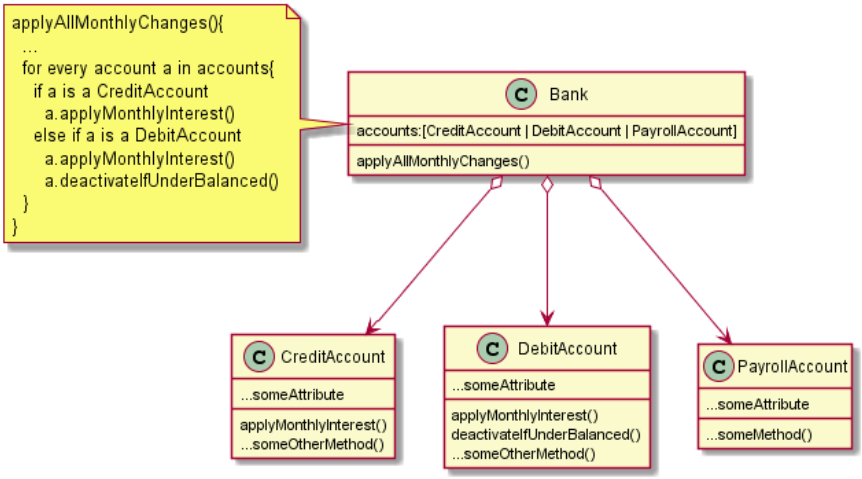
The process of designing these cohesive systems requires not only OOP design techniques but also domain knowledge. The designer of the system, should know how a library system operates so that, he/she can accurately simulate their responsibilities on code.

You can also apply SRP on individual methods inside objects. Methods should be responsible for one thing only. Keeping methods pure like these will help reduce unwanted side effects. The builder/manipulator naming scheme will help you with this. Also, a method should not be responsible for handling the problems such as errors/exceptions. The method should delegate that responsibility to the clients of that method. The methods itself should only report/raise/throw the problem. The best way to do this in OOP is by raising an exception.

Open/Closed Principle

A good class in OOP is both open and closed. It is open for extension but closed for modification.

To understand this principle let's have an example of a system that is closed for extension:



24

This is a system that indeed works perfectly. The bank will be able to apply the appropriate changes to the account types because of the `if-else` block that segregates the accounts based on its type (another instance of type discrimination so that's a hint that this is bad design). The problem with this is that whenever there are changes regarding the monthly changes or interest calculations, you would have to tamper with the contents of bank. This is **opening `Bank` up for modification**. Every time there's a change related to monthly updates you would have to do some kind of surgical procedure on `Bank` and rearrange its internal organs so that the change may be supported. This is extra rough on `Bank` because `Bank` shouldn't even be responsible for these behaviors (a

[24]Forgive the long Java-like method names, they're named as descriptive as possible so that I can skip actually explaining what they do.

violation of SRP). If there are new types of account, then you would have to open up `Bank` again and to add another `else if` block. Poor `Bank`, who knows how many more new types of accounts there are in the future.

Instead of rearranging the organs of your classes to accommodate changes to behavior they are not even responsible for, you should close the classes for modification and open them for extension instead:

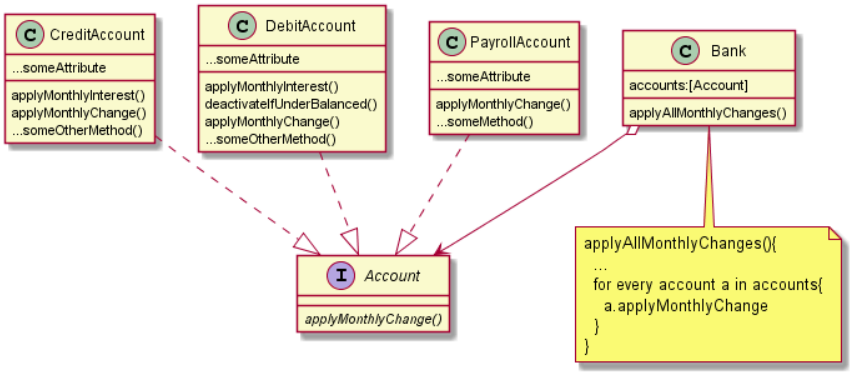


Figure 12: open for extension

Since `applyMonthlyChange()` is an abstract method of account, all its realizations are required to implement it. We extend the functionality of `CreditAccount`, `DebitAccount`, and `PayrollAccount` by adding an extra method.

- Inside `CreditAccount.applyMonthlyChange()` you just call `applyMonthlyInterest()`,
- Inside `DebitAccount.applyMonthlyChange()` you call both `applyMontlyInterest()` and `deactivateIfUnderBalanced()`
- Inside `PayrollAccount.applyMonthlyChange()` you do nothing

Is a method that does nothing inelegant? Not really because this is an accurate representation as to how a `PayrollAccount` changes every month— it doesn't change. Also, in the future, the inertness of `PayrollAccount` may change so at least you have the function prepared.

> The previous library card example and bank examples are indeed similar. This is because introspective checks like `isinstance` are again symptoms of inelegant design.

Another example of this is how you need to augment the `PayrollAccount` class to contain the `recieveFund(float)` or `deposit(float)` method so that it can become a recipient for transfer.

> Closed for modification does not mean you cannot change literally anything in the class. Of course if there are mistakes in the specific behavior of the class then you have to modify it.

Liskov-Substitution Principle

This principle basically dictates when should an object be a subtype or a realization of another object. Should `PayrollAccount` be a realization of `Account`? If you can substitute any an instance of `Account`with a `PayrollAccount` then the answer is yes. The same is true if you want to establish an inheritance relationship between `Account` and `PayrollAccount`. LSP is important because it ensures the polymorphic capabilities of your realizations and subtypes.

Interface Segregation Principle

Sometimes the subtypes/realizations of a certain object may have diverse functionality. Some subtypes can deposit, some subtypes can't, all subtypes can be recipient for transfers but not all can be senders. The diversity of functionality support may sometimes force the designer to pollute the system with methods that the subtypes don't actually use. `PayrollAccount` will not use deposit but since it realizes `Account` then we reluctantly add it. This is a violation if LSP which states that an object should not be forced to implement methods it doesn't use.

Role Interfaces

The best way to design these diverse systems is by refactoring your architecture to have diverse role interfaces instead.

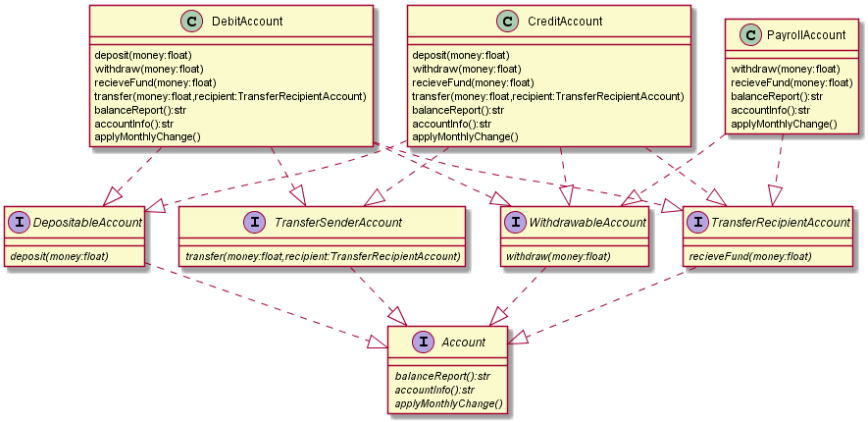


Figure 13: God class library card

Now instead of cluttering you realizations with useless methods, your system is now cluttered with role interfaces. This is a benevolent kind of clutter. Because more objects and looser dependencies make for a maintainable and therefore elegant system (in the same way a language with more words have less chance for ambiguity). Because of this clutter, you can have rich polymorphism without sacrificing ISP. Although be careful not to overdo it though. You wouldn't want a role interface for every conceivable method out there.

When you have role interfaces, you can refine lines of your code by describing which exact role interface applies. For example, instead of writing `transfer` as some `Account` to `Account` process they, it would be more specific to write it as a `TransferSenderAccount` to `TransferRecipientAccount` process.

Dependency Inversion Principle

The relationships between objects should be defined by surface instead of their interior. This means that How a bank interacts with an account should not be dependent on the implementation. The way `Bank` prints the balance report should not be by directly concatenating `"Your balance is" + account.balance`. Bank should interact with an account by printing the return value the abstract builder method,

`balanceReport()`. The class `Bank` should not dissect `Account` to retrieve the balance. It should tell the `Account` to build a balance report.

Optional Reading

Bailey D., (2009) SOLID Development Principles – In Motivational Pictures. Accessed August 31, 2020

Class Relationships

Introduction

The interactions between one an instance of a class to another is largely characterized by the relationship between them. Here we talk about relationships that define the polymorphism between classes and relationships that define dependency between objects.

Learning Outcomes

1. Differentiate realization relationships and specialization relationships
2. Describe how class abstract methods work in realization relationships
3. Differentiate aggregation relationships and composition relationships

Type Based Relationships

Type based relationships are characterized by how two classes are related to each other through ontological hierarchy. A mammal class's relationship with an animal class's relationship is type based. That is

because a mammal is considered as an animal while an animal is not necessarily a mammal.

There are two type based relationships (there can be an extra one which is a type relationship that is sort of a hybrid of the two).

Realization
A realization relationship is a one way relationship that describes how something abstract is **REAL**ized by something concrete. Given a `Realization` class and some `Abstraction` class, The `Realization` class realizes the `Abstraction` class.

A realization relationship is also called an **implementation** relationship. An implementation, implements some interface. **Interface** is also a fitting term for abstractions because it is through these classes that other objects interact with each other.

An `Abstraction` is a special type of class that does not contain any implementation. This means that `Abstraction` doesn't have code that controls the form and behavior of the class. It only contains code that specify how this class interacts with other objects. This means that abstract classes only contain method names and type signatures with empty bodies.

These `Abstraction` classes appear useless at first since it doesn't do anything at all. In fact, you cannot even create an instance of an abstraction. Even if you do, it will be pointless since it doesn't have code that controls how it behaves.

An abstraction can only be useful if some other class realizes this abstraction. These `Realization` classes provide abstractions their form and behavior.

What's the point in maintaining some realization relationship between classes? If abstractions can only be used through their realizations, then why create the abstraction at all?

The importance of this seemingly pointless relationship lies in OOP's data hiding principle. We will explore more about why these relationships are very common in a future discussion about SOLID principles. For now, I'll show one of the reasons why this is useful through an example:

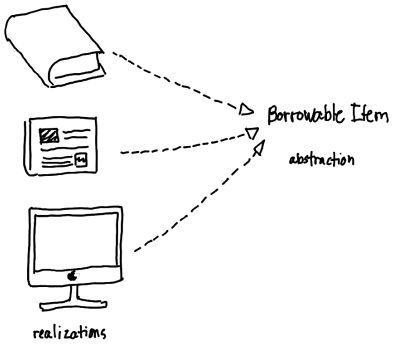


Figure 14: Realization Relationship Example

Consider a library system. In a library, you are able to borrow resources such as books, newspapers, and computers. When the library system interacts with these resources to facilitate transactions such as borrowing and returning, the library system treats these resource like general **Borrowable Items**. It doesn't really need to concern itself of the specific type. Operating like this is better for the library system for reasons such as maintainability and future proofing (this will be explained in detail in when we discuss SOLID design principles). Therefore, the best architecture to use in this situation is to make each resource type a realization of Borrowable Item. A `BorrowableItem` class would merely be an abstraction. This class would contain no behavior or form, it only contains specifications of how to interact with it. And it does make sense, I mean, how exactly does a general borrowable item

behave or look like? It's abstract. What we know is that any `BorrowableItem` can be borrowed or returned, so we write empty `borrow()` and `return()` functions (it specifies how it interacts with others, but it doesn't have any specific behavior, these methods are called abstract methods).

Any realization of `BorrowableItem`, such as `Book` or `Newspaper` will be forced to implement the `borrow()` and `return()` methods as well. In general, all realizations are forced to implement the methods of its abstraction[25]. This means, any realization needs to include these methods with each of their own method bodies for borrowing and returning. If books and newspapers are borrowed or returned in different manners, then you write different method bodies for each.

This is how realization relationships enable **polymorphism**, a `Book` is a `BorrowableItem`, allowing the library system to interact with it like any `BorrowableItem`. But at the same time `Book` is a book, so it behaves in the manner a book behaves.

By building all of these relationships, the library system is able to interact with resources without explicitly knowing which exact resource it is. The system knows that it is borrowing some instance of a borrowable item, but it does not know if it is a book or a newspaper. The exact type of this instance will then behave depending on its type. Although all this effort may seem unnecessary, you will learn in this course that through the establishment of these relationships, OOP is able to uphold one of its core design principles, **data-hiding**.

Abstractions can also realize abstractions. For example given an abstraction A, another abstraction, called B, can realize A. A class C then realizes B. When this happens, B does not need to implement A's abstract methods because B is an abstraction itself. Therefore, C will inherit all of A and B's abstract methods. The abstract methods of an

[25]While realizations are forced to implement all of its corresponding Abstraction's methods, realizations can have the option to implement extra methods that are not present in its abstractions.

abstraction cascades down to the realization realizing any of its realizations as well.

In this case C instances can be treated as B instances or A instances, but it will behave in the way C behaves.

Specialization
Specialization relationships are very similar to realization relationships. You can think of these specialization relationships as realizations but between two real/concrete classes. A specialized class specializes some general class. By establishing this relationship, you are able to extend the form and behavior of a specific class.

Specialization has plenty of names. This relationship is also called **extension** between the special/child/subclass and general/parent/super class. Another name for it would be **inheritance**.

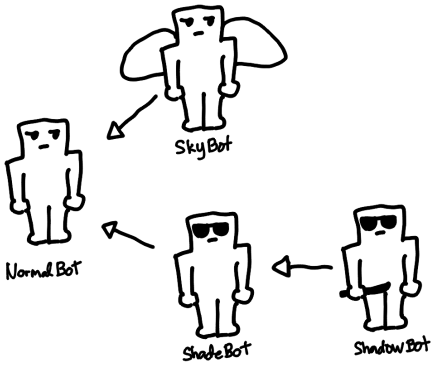


Figure 15: Specialization Example

Here's an example that would illustrate what the specialization relationship means. Consider a factory that builds robots. This factory is able to build `NormalBots` which are the general type of robots. Instead of creating entirely separate mechanisms to build each type of robot, the

factory is able to exploit the fact that other robots are just specializations of `NormalBot`. `Skybot` is just the same as `NormalBot`, but it has extra flight capabilities. `ShadeBot` is just the same as `NormalBot`, but it has UV Protection. Because of this the factory is able to use the building recipes for `NormalBot` to build `Skybot` and `ShadeBot`. All they need to do is to add some extra layers of construction such as adding wings or outfitting shades.

Specialization relationships work like this as well. When you write code for the general class, you do not need to rewrite it for specializations. What you write inside specializations are the attributes and method that make it special. If this specialization has flight capabilities then add attributes for wings and methods for flight. If this specialization behaves differently for some specific method then you only change that specific method.

Because of this relationship, you only need to write one copy of the code that is common for the general class and its specializations. This means that you do not need to rewrite said code, saving time and effort but more importantly, having one copy of code helps for maintainability. When the recipe of all robot types need to change, the factory only needs to change the recipe of `NormalBot`, all of special robots' recipes will change as well since they all use `NormalBot`'s recipe. When the code for the general class and the special classes need to be updated, changing the shared code found inside the general class will automatically affect special classes. Through this mechanism, specialization relationships enable **inheritance**, one of OOP's core design principle.

This is where the terms **inheritance** and extension make sense. Special class inherit all the attributes and methods of the general class. Special classes can also be thought of as extensions of the general class, since these special classes extend or tweak the capabilities of the general class.

You can also specialize, specializations. This is illustrated by `ShadowBot`, which is a special `ShadeBot` that has knife. Since `ShadowBot` is a special `ShadeBot` and `ShadeBot` is a special `NormalBot`, `ShadowBot` is automatically a specialization of `NormalBot` as well.

Specializations also allow polymorphism in the same way realizations do. A `ShadowBot` can be interacted with like any `NormalBot` or `ShadeBot` but since it is also a `ShadowBot` it will behave specifically like a `ShadowBot`.

Abstract Classes

An abstract class is something in between an abstraction and a generalization. It contains attributes and methods with bodies, but it also contains abstract methods as well. When classes specialize/realize abstract classes, they inherit the attributes and methods with bodies, but they are forced to implement the abstract methods as well. These relationships are sometimes used if the system requires a mix of inheritance and implementation between classes.

Multiple Type Relationships

It is possible for a class to realize/specialize multiple abstractions/generalizations. For example, given abstractions, A, B and C, and generalizations E, F, and G, a class X can realize/specialize all of them. When you do this, X will be forced to inherit all the abstract methods in A, B and C, and automatically inherit everything from E, F, and G.

Dependency Relationships

Dependency relationships, characterize how two classes interact with each other. A class which is dependent on another class, needs to know how to interact with it. These interactions range from being used as method parameters, being returned in methods, being used inside method bodies, being used as attributes, etc. A dependency relationship is one way (but it is also possible for two objects to be dependent on each other). A **client** class is dependent on some **dependency**.

Weak Dependency

On weak dependency relationships a client class merely **uses** instances of a dependency. This is when, a class uses other classes as method parameters, local variables, or return types.

Structural Dependencies

Structural dependencies, define relationships between a collective and its parts. If an object instance contains instances from a different class. Then they have a structural dependency.

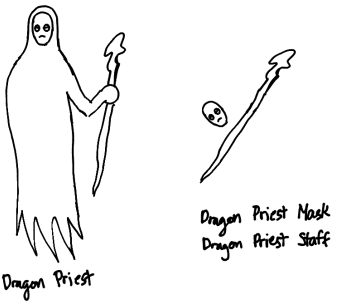


Figure 16: Dependency Example

Aggregation

When a dependency is an aggregate of some client, it means that the client merely **has** instances of this dependency. These relationships are the looser forms of structural dependency, because the dependency instance can exist outside the lifetime of the client instance.

For example, in a videogame, a hostile enemy instance of `DragonPriest` spawns equipped with instances of`DragonPriestMask` and `DragonPriestStaff`. The `DragonPriest` instance is a client of the dependencies `DragonPriestMask` and `DragonPriestStaff`.

`DragonPriest` interacts with these dependencies to calculate its attack damage, its defenses, etc. But when this specific instance of `DragonPriest` is defeated, it despawns, leaving behind its `DragonPriestMask` and `DragonPriestStaff`. These instances will continue to exist since it can still be used by other client objects in the game, such as the player character, some storage chest or whatever. This means that the relationship between `DragonPriest` and the dependencies `DragonPriestMask` and `DragonPriestStaff` is aggregation.

Composition

Composition relationships are ownership dependencies. When a client is composed of some dependency, this means that the client **owns** the instances of this dependency. These relationships are stronger forms of dependency since the existence of the dependency instance is tied to the client, meaning, the dependency ceases to exist outside the lifetime of the client instance.

In the same videogame, the hostile enemy instance of `DragonPriest` appears in game using some `DragonPriestCharacterModel` instance. When the `DragonPriest` instance is defeated, it despawns. It makes no sense for the `DragonPriestCharacterModel` instance to stay behind after the `DragonPriest` instance is defeated, therefore it ceases to exist as well. This means that the relationship between `DragonPriest` and the dependency `DragonPriestCharacterModel` is composition.

Unified Modelling Language for Class Diagrams

Introduction

The modelling language we are going to use to represent architecture would be UML or **Unified Modelling Language**. We use this to show the relevant classes in the system including their attributes, methods and relationships with other classes. UML differ a little depending on the source. The specifications we'll be following in this class would be based on PlantUML.

Learning Outcomes

1. Create class diagrams to represent OOP architecture in UML
2. Create classes with attributes and methods in UML
3. Establish class relationships in UML

Modelling Classes

The example below shows three classes. One is concrete class called `ExampleClass` (notice the "C" in the title that denotes it is a concrete class). Another is an abstraction called `ExampleAbstraction` (denoted by "I" which stands for interface). The last one is `ExampleAbstractClass` which is an abstract class (denoted by "A").

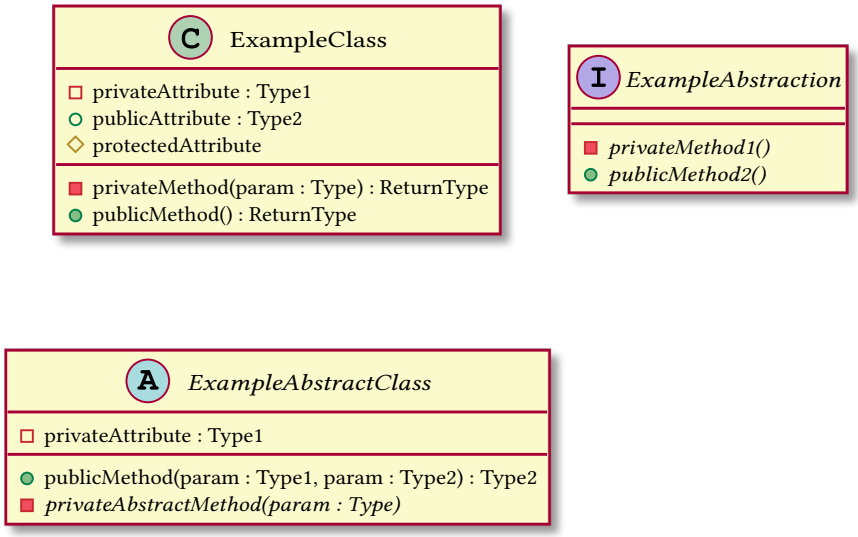


Figure 17: Class Diagrams

Attributes are placed in the top portion and methods are placed in the bottom portion. The shape to the left of each attribute or method indicates its visibility. Filled shapes indicate visibilities for methods while unfilled shapes indicate visibilities for attributes.

The names for abstract methods, abstractions, and abstract classes are italicized.

If possible, write the expected type of attributes, parameters and function returns. This is written on the right side of their names, to the right of ":".

Modelling Class Relationships

Here's a reference arrows that indicate the relationship between two classes:

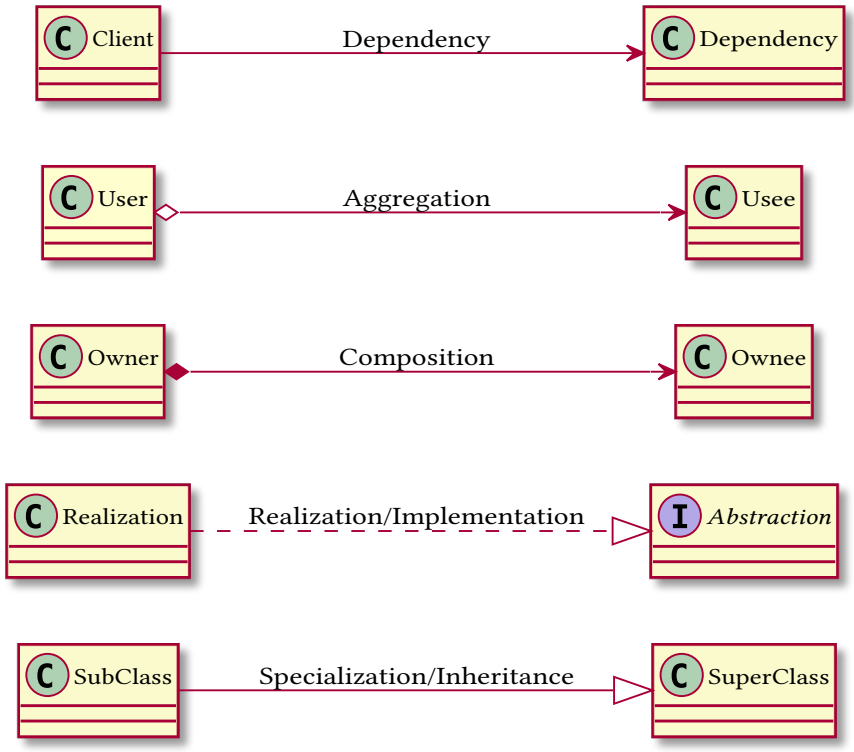


Figure 18: class relationships

Optional Readings

PlantUML Class Diagrams. https://plantuml.com/class-diagram Accessed August 31, 2020

Design Patterns Introduction

Introduction

Design patterns, are general, reusable solutions to a commonly occurring problem within a given context in software design. Unlike algorithms, design patterns are not clear instructions that can automatically be transferred to your system. Design patterns are more like templates that describe the general concept to solve the problem. It doesn't contain implementation details; it contains structural blueprints.

Learning Outcomes

1. Discuss the origins of design patterns in OOP
2. Explain the advantages of design patterns
3. Explain the disadvantages of design patterns
4. Identify the three classifications of design patterns

History of Design Patterns

Design patterns are not novel and sophisticated discoveries, they are instead, typical solutions to common problems. The pattern of these solutions become so ubiquitous that it becomes worthwhile to put a name to it. Design patterns in software engineering are just borrowed concepts from architecture/design.

The concept of design patterns is often attributed to Christopher Alexander, from his book, *A Pattern Language: Towns, Buildings, Construction* (Alexander et al. 2017). These patterns may describe how high windows should be, how many levels a building should have, how large green areas in a neighborhood are supposed to be, and so on.

Four software engineers, Erich Gamma, John Vlissides, Ralph Johnson, and Richard Helm, used this as an inspiration to publish the famous book, *Design Patterns: Elements of Reusable Object-Oriented Software.* (Gamma 2011) The four became collectively known as the "**Gang of Four"**. And their book became known as the GoF book. It contains a catalog of 23 design patterns solving various problems of OOP design.

Why Patterns?

The answer to this problem is similar to the reason as to why you don't "reinvent the wheel". Design patterns are tried and tested solutions, knowing these patterns give programmers a toolset to solve a variety of problems in software design.

Design patterns also help with communication. A team of software engineers well versed in design patterns wouldn't need to explain to each other what exactly must be done to use an "Adapter pattern".

Why not Patterns?

Design patterns are sometimes used to simulate features that the programming language doesn't have. If you use a powerful enough language you wouldn't need the pattern at all. Example of this is how the Strategy pattern can be replaced by lambdas.

Patterns are not end-all be-all solutions to any design problem out there. At the end of the day context matters the most. An inexperienced programmer will implement a problem to the dot, instead of adapting the pattern for the context. Patterns are not end-all be-all solutions to any design problem out there. At the end of the day context matters the most.

Sometimes, you don't even need a pattern at all. A simple problem solved using a complicated solution is inelegant.

Classifications of Design Patterns

- **Creational Patterns** provide object creation mechanisms that increase flexibility and reuse of existing code
- **Structural patterns** explain how to assemble objects and classes into larger structures, while keeping the structures flexible and efficient.
- **Behavioral patterns** take care of effective communication and the assignment of responsibilities between objects.

Optional Reading

Gamma, Vlissides, Johnson, and Helm (1994). Design Patterns: Elements of Reusable Object-Oriented Software

Creational Patterns

Introduction

Almost all programming languages with object-oriented support provides you rich features in creating instances of classes using constructor methods. Inside the constructors you can add business logic to initialize objects and make them ready for use. Some programming languages (like Java or C++) even have the capability to have more than one constructor method so that instances can be shipped with different states depending on the chosen constructor.

Unfortunately native constructors capabilities are not powerful enough for our standards of elegance. Some systems have complicated object production mechanisms that require extra capabilities. Sometimes classes have too many attributes for simple constructors. Sometimes object creation require polymorphic support and decoupling against the exact subtypes or realizations. Sometimes you need to ensure that certain classes have exactly one instance throughout the lifetime of your system.

Learning Outcomes

1. Design systems that apply the factory method design pattern
2. Design systems that apply the abstract factory pattern

Factory Method Pattern

Problem

The exact type of the dependency (a product) created and used by some client (a factory) is decided by a client of that factory. Somewhere, inside this factory class, a specific product is being instantiated and maybe used (this instantiation happens maybe more than once). But, as it turns out, there are different types of products, (there's also the possibility of more product types in the future). You can change the code of the factory class to accommodate multiple product types. For every product, you modify the factory and add some if-else clause to produce the correct product type.

As you see this process is quite tedious. For every new product type that is added to your system, you perform surgery to the factory class. This process will end up forcing you to create smelly if-else checks to switch to the correct product type.

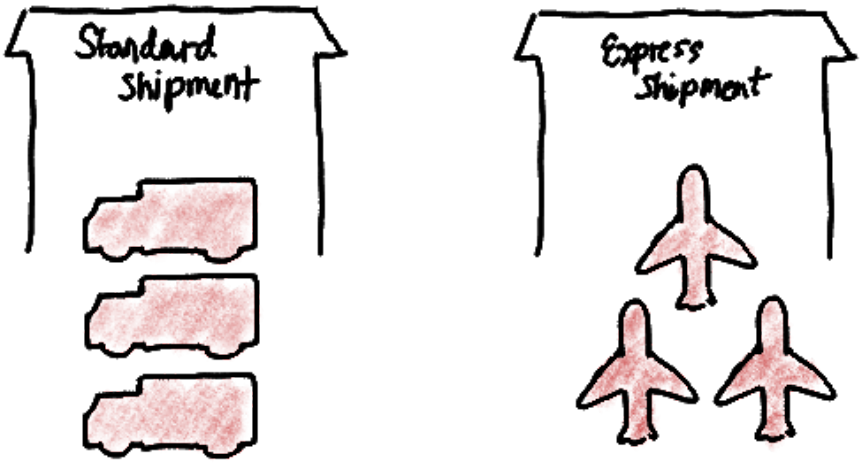


Figure 19: Factory Method

Solution

You encapsulate the creation of a class inside a **factory method** that is specified to return an abstraction of the product. If there are other real product types that have to be produced, you create a specialized factory which overrides the factory method.

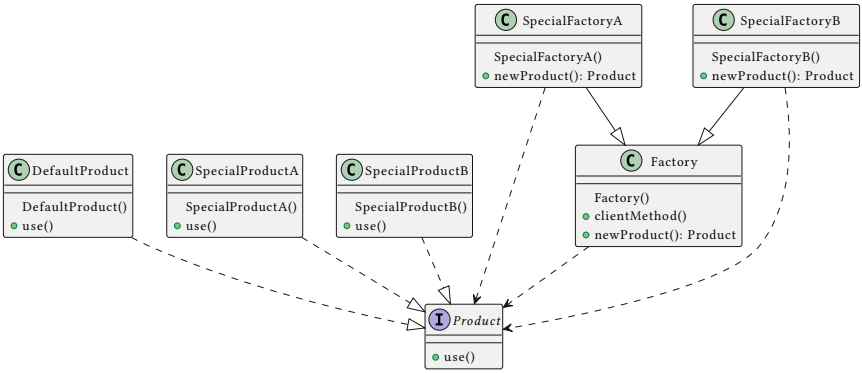


Figure 20: factory method class diagram

Somewhere inside factory you have one or more instances of creating or using the product.

If you choose to build `Factory` as a concrete superclass, the factory method inside the `Factory` should return some realization of `Product`. This is the default product returned by any `Factory`. If you need to return a different `Product` realization, you override the factory method to return that particular `Product` realization.

Example

Online Marketplace Delivery
Consider you're developing the product delivery side of an online marketplace app (think Amazon/Lazada). Your app is on its early stage so their is only one delivery option, standard nationwide delivery that takes a minimum of 7 days.

What you have is `Shipment` class that contains a `StandardDelivery` class. Inside the shipment class is the `shipmentDetails()` builder which builds a string representing the details of the shipment, this includes the delivery details (which requires access to the composed `StandardDelivery` instance). Inside the constructor of `Shipment` an instance of `StandardDelivery` is created so that every `Shipment` is set to be delivered using standard delivery.

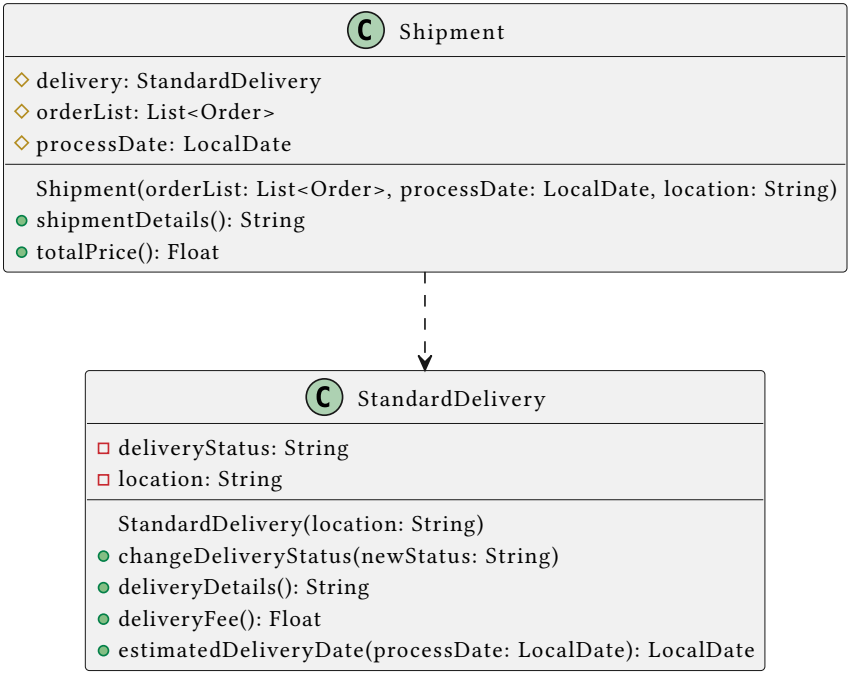


Figure 21: online marketplace

This system does work. It works but it is still inelegant. As soon as your app grows, you will incorporate new delivery options like express delivery, or pickups or whatever. Every time you need to add a new delivery method you will need to perform surgery in `Shipment` since the `StandardDelivery` instance is created inside the constructor of `Shipment`. `Shipment`'s code is too coupled with `StandardDelivery`.

To solve this you need to implement the factory method pattern. Right now shipment is a factory since it constructs its own instance of `StandardDelivery`. To refactor this into elegant code, you need to so create an abstraction called`Delivery` first to support polymorphism. Inside `Shipment` instead of creating instances of `Delivery`'s using a constructor, you invoke a factory method that encapsulates the instantiation of `Delivery`. In this case we name this method

`newDelivery()`. All it does is return an instance of `StandardDelivery` using its constructor.

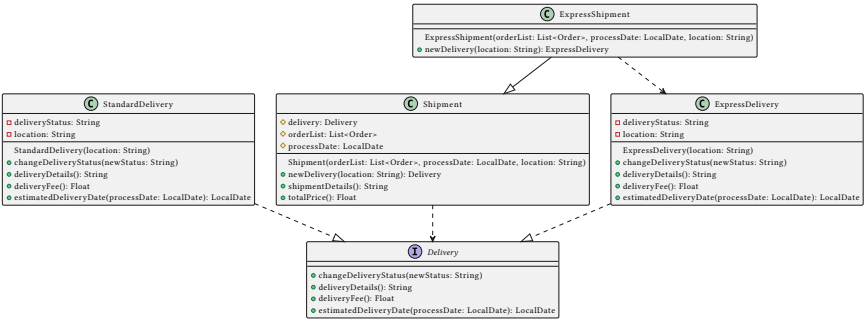


Figure 22: online marketplace

In this new architecture, whenever there are new delivery methods a shipment could have, all you have to do is to create a realization of that delivery method. In this case the new delivery method is `ExpressDelivery` which delivers for two days but is twice as expensive. And instead of changing `Shipment` (violates Open/Closed Principle), you make an extension to `Shipment`. This extension is the specialization to shipment called `ExpressShipment` (a shipment that uses express delivery). In this specialization, you only need to override the factory method delivery, so that every instance of delivery construction creates `ExpressDelivery`. The difference between `ExpressDelivery` and `Delivery` is that `ExpressDelivery` has a delivery fee of 1000 and the estimated delivery date is 1 day after the processing date.

Why this is elegant

- **Single Responsibility Principle** - the extra level of encapsulation on the construction of the product (factory method), allows the factory to be responsible of creating the exact product type it needs.
- **Open/Closed Principle** - instead of modifying the factory to incorporate the creation of different product realizations, you instead create an extension of the factory. No need for introspective checks

since the factory method supports polymorphism of the product it creates.
- *Encapsulate what varies* - This pattern upholds one of OOP paradigms most important principles. Since the construction of product varies from product type to product type, it is encapsulated into the factory method.
- This avoids tight **coupling** between the factory and the product

> Coupled classes are classes which are very dependent on each other. Changing the code of one will most likely affect the other

How to implement it:

1. Create an abstraction for all product types (`Product`).
2. Inside the base factory create the the factory method function `newProduct():Product`. Make sure it is specified to return abstraction `Product`.
3. For every new product type that is added to the system, create 2 new classes: the new product type as a realization of `Product` and, and the factory for the new product type as a specialization of `Factory`.
4. Inside each factory specialization override `newProduct()` to return the correct realization of `Product`.
5. Replace every instance of constructor calls inside `Factory` with a call to the factory method `newProduct()`.

Abstract Factory

Problem

Your system consists of a family of related products. These products also have different variants. Some client class or function dependent on your products needs a way to create these products so that the products match the the same variant. The exact variants of the family of products are decided during runtime somewhere else in the code (similar to product creation in a factory method). To accomplish this you can

simply switch between products using some kind of selection (if-else, switch-case, when, etc.)

This solution works right now, but what if there are new product variants? Every time there are new variants you will end up adding new branches to your client code.

Figure 23: Abstract Factory

Solution

You create different kinds of factories that realize under the same abstract factory. Said abstract factory will contain abstract factory methods that delegate product construction for each product type. These factory methods are named `newProductA()` and `newProductB()`.

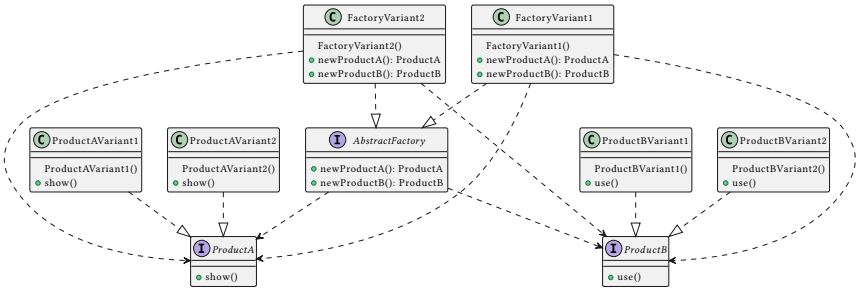


Figure 24: abstract factory

For every possible variant, you create a realization if `AbstractFactory`. The exact realization of factory (`FactoryVariant1` or `FactoryVariant2`) will decide the variant of the family of products that are created. These realizations will implement the factory methods based on their variant.

For example `FactoryVariant1`'s `newProductA()` will return a new instance of `ProductAVariant1`, while `FactoryVariant2`'s `newProductA()` will return a new instance of `ProductAVariant2`. Using this an instance of `AbstractFactory` can either produce products from variant 1 or variant 2 depending on its exact realization.

Whenever there are new variant types that are added to your system, you will not need to change any client code. All you need to do is to create new `Product` realizations based on the new variant as well new `AbstractFactory` realizations that produce these product variants.

Example

Bootleg Text-based Zelda Game

You're creating the dungeon encounter mechanics of some bootleg text-based zelda game. In this game, every time you enter a dungeon, you encounter 0 to 8 monsters (the exact number is randomly determined). There are 3 types of monsters, bokoblins, moblins, and lizalfos (different types have different moves). The exact type of monster is randomly decided as well.

Right now the game works like this:

As soon as you enter the dungeon, all the enemies are announced:

```
5 monsters appeared
A lizalflos appeared
A lizalflos appeared
A lizalflos appeared
A moblin appeared
A moblin appeared
```

After this, each enemy in the encounter attacks. They randomly pick an attack from their moveset.

```
Lizalflos thorws its lizal boomerang at you for 2 damage
Lizalflos thorws its lizal boomerang at you for 2 damage
Lizalflos camouflages itself
```

```
Moblin stabs you with a spear for 3 damage
Moblin stabs you with a spear for 3 damage
```

The encounter ends with Link dying since you haven't coded anything past this part.

You decide to make things exciting for your game by adding harder dungeons, medium dungeon and hard dungeon.

Medium dungeon
Instead of encountering, normal monsters you encounter stronger versions of the monsters, these monsters are blue colored:

- **Blue Bokoblin**
- equipped with a spiked boko club and a spiked boko shield
- bludgeon deals 2 damage
- **Blue Moblin**
- equipped with rusty halberd
- stab deals 5 damage
- kick deals 2 damage
- **Blue Lizalflos**
- equipped with a forked boomerang
- throw boomerang deals 3 damage

Hard dungeon
These monsters are silver colored extra stronger versions of the monsters

- **Silver Bokoblin**
- equipped with a dragonbone boko club and a dragonbone boko shield
- bludgeon deals 5 damage
- **Silver Moblin**
- equipped with knight's halberd
- stab deals 10 damage
- kick deals 3 damage
- **Silver Lizalflos**
- equipped with a tri-boomerang

- throw boomerang deals 7 damage

To seamlessly incorporate these harder monsters in your system, you need to create an abstract factory for each dungeon difficulty. There are now three variants for each monster. For every variant, there is a factory that spawns new instances of each monster.

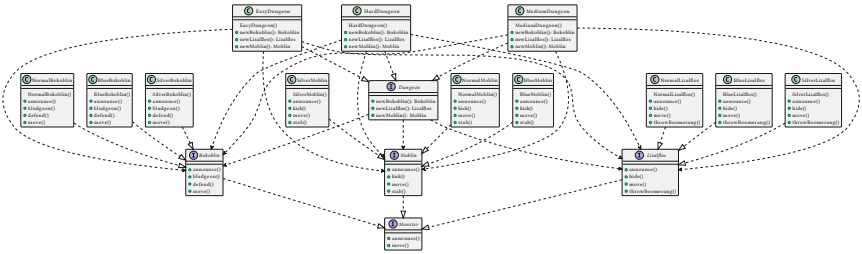


Figure 25: abstract factory example

Why this is elegant

- **Open/Closed Principle** - This solution is easier to maintain since you can add more variants of `Product` without touching any existing code. All you have to do is to add new realization for `Product` and a new realization `AbstractFactory`
- Changing the form and behavior of specific variants are isolated since its creation is abstracted.
- You can easily switch between variants by swapping out the factory.

How to implement it:

- For every product in the family of products, create an abstraction of it (`ProductA`, `ProductB`).
- For every variant of the products, create a factory, (`FactoryVariant1`, `FactoryVariant2`). These factories must realize under an abstraction `AbstractFactory`. The factory should contain abstract factory methods for each product,
- Inside every factory realization implement all factory methods.

Singleton Pattern (Optional Read)

Problem

Sometimes it wouldn't make sense for a class to have more than one instance in the lifetime of the application. These things are called singletons.

Solution

Inside the singleton. Create a static attribute that represents the singleton. Since it is static all instances of the class will share this value. Create a builder to lazily instantiate the value of the singleton and expose the value of the instance.

Disallow the usage of the normal constructor as much as possible. To access the shared static instance, use the builder.

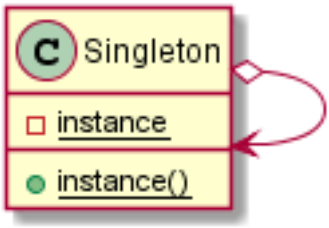


Figure 26: Singleton

Disallowing the creation of a singleton depends on the language you use, you can set the constructor to private, or you can raise an error if you try to use the constructor outside the instance builder.

You can also choose to not disallow the use of the constructor, as long as you trust the users to always use the instance builder instead.

Example

A Catalog of Globals

It doesn't make sense for you to keep multiple copies of global variables in your application, so you decide to place them in a singleton class.

Why this is NOT elegant

A singleton pattern is actually hated by most developers. Yes you can ensure that there is exactly one instance of a class, but its advantages come with a lot of drawbacks.

- You can ensure single instance classes, just by being vigilant.
- Singletons require the use of static attributes and methods. Statics are anti-pattern because they are global variables and manipulators that can have invisible changes to state.
- Singletons are usually symptoms of bad design.

Optional Reading

Shvets A. (2018) Creational Patterns Accessed August 31, 2020

Structural Patterns

Introduction

As your system evolves, the structure of your classes could get complicated. As you introduce more features, your classes become bigger and harder to maintain. To alleviate these issues, you can assemble objects into maintainable structural patterns.

Learning Objectives

1. Design systems that apply the decorator pattern
2. Design systems that apply the adapter pattern

Decorator Pattern

Problem

Some of your classes require extra features that can be added and removed during runtime. Sometimes you even need to support a set of extra features that can be arbitrarily combined with each other. You

need to do this without breaking how these classes are being used by their clients.

Figure 27: decorator

Solution

To solve this issue, all you have to do is to apply the open/closed principle. For every feature that can be arbitrarily added to some a simple class, you need to create a `Decorator` that extends the features of classes using inheritance and composition at the same time. The neat thing about this pattern is that the `Decorators` will have polymorphically the same type as the simple class due to inheritance. `Decorators` will also be able to control instances of the simple class because of composition.

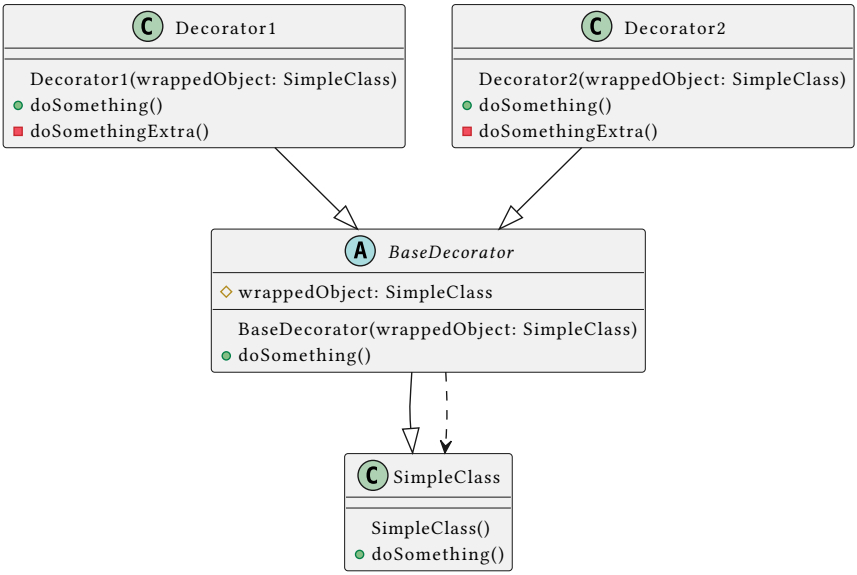


Figure 28: decorator

To create an instance of a `SimpleClass` decorated by `Decorator1`, all you need to do is to wrap the `SimpleClass` instance with an instance of `Decorator1`. When this `Decorator1` instance, calls `doSomething()` it calls the wrapped `SimpleClass` instance's `doSomething()` and do some extra behavior.

```
//Decorator1's implementation of doSomething():

override fun doSomething() {
    wrappedObject.doSomething()
    doSomethingExtra()
}
```

It would be handy to create a `BaseDecorator` abstract class that is inherited by all decorators. It's not required but this class will form a class hierarchy for all decorators. Plus, you can write all of the common behavior and data into this class. It would be better for the `BaseDecorator`'s `doSomething()` method to be abstract so that the decorator realizations are forced to override `doSomething()`.

Example

Decorating Sentences

A sentence can be defined as a list of words (words are strings). The string representation of a sentence is the concatenation of all of the words in the list, separated by a space.

Instances of sentences can be printed with formatting:

- **bordered** - Given the sentence, `["hey","there"]` it prints:

```
-----------
|hey there|
-----------
```

- **fancy** - Given the sentence, `["hey","there"]` it prints:

```
-+hey there+-
```

- **uppercase** - Given the sentence, `["hey","there"]` it prints:

```
HEY THERE
```

The formatting of a sentence is decided during runtime. These formats should also allow for combinations with other formats:

- **bordered fancy** - Given the sentence, `["hey","there"]` it prints:

```
---------------
|-+hey there+-|
---------------
```

- **fancy uppercase** - Given the sentence, `["hey","there"]` it prints:

```
-+HEY THERE+-
```

To accomplish these features, you need to implement the decorator pattern. Each formatting will be a decorator for `Sentence` objects. These formats need to inherit from some abstract `FormattedSentence` class. This abstract class is specified to compose and inherit from sentence. The behavior that needs to be decorated is the `__str__()` function since you need to change how sentence is printed for every format.

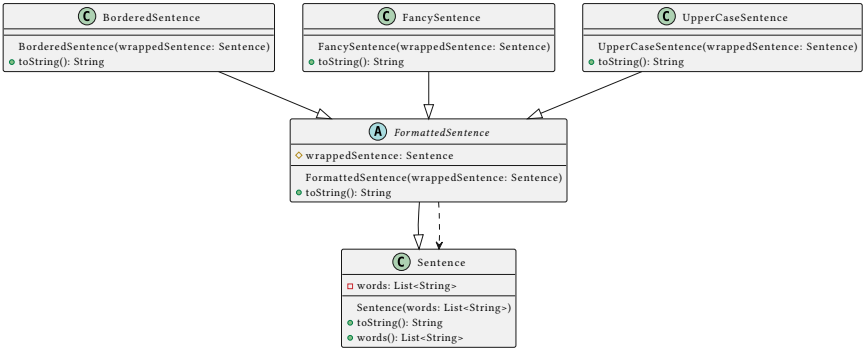


Figure 29: decorator example

Why this is elegant

- **Open/Closed Principle** - Decorators extend classes via inheritance. It is easier to add new decorators without touching any exiting code.
- A class can have many variants because of diverse combinations of behaviors can be cleanly implemented using this pattern.
- You can arbitrarily mix and match decorators without the worry of polymorphic incompatibility during runtime

How to implement it

1. Create an abstract class `BaseDecorator` that specializes some `SimpleClass`. This `BaseDecorator` will also have the attribute `wrappedObject` which is a reference to an instance of the decorated `SimpleClass`. This attribute is set as protected so that it may be inherited by `BaseDecorators` specializations. `BaseDecorator` also contains the abstract method `doSomething()`. This method's behavior, when invoked by `BaseDecorators` specializations, changes depending on the decorations attached to `SimpleClass`
2. For every decoration, that can decorate `SimpleClass`, a specialization for `BaseClass` is created. These specializations implement `doSomething()` in a manner that augments/modifies `SimpleClass`'s own `doSomething()`

Adapter Pattern

Problem

As the system evolves, you'll likely encounter interfaces of instances that are incompatible with their intended clients. These interfaces do perform the necessary behavior, but maybe the method names are just different. This happens quite a lot since the interface of the dependency may be originally built for different reasons. The interface may be an external module imported on existing client code. You can just change the incompatible interface to support the functionality you need but this is not always possible and may introduce code duplication.

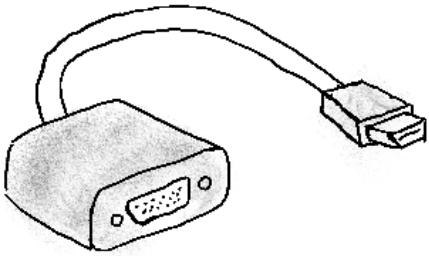


Figure 30: adapter

Solution

In the same way vga is usable on an hdmi port using an adapter, you can use an incompatible dependency on a client as a compatible instance using the adapter pattern.

Say you have an instance of `AbstractDependency` (it could be any realization of `AbstractDependency`), that needs to be used like an instance of `RequiredInterface` by some client. In this example, the `RequiredInterface`'s `method()` works the same way as

`AbstractDependency`'s `dependencyMethod()`. But unfortunately they have different names, thus creating the incompatibility.

You can rename `dependencyMethod()` into `method()` and change `RealDependency`'s abstraction from `AbstractDependency` to `RequiredInterface`. This will fix the incompatibility but it will break existing clients of `AbstractDependency` since the method names have been changed. To avoid breaking clients of `RealDependency` you instead make it so that `RealDependency` realizes both `AbstractDependency` and `RequiredInterface`. This will fix the incompatibility without affecting any client code but this introduces redundant code and violates the Interface Segregation Principle. If `RealDependency` has specializations, it will clutter its specializations with redundant code as well.

What you need to do is to create an adapter to `AbstractDependency` called `Adapter` which realizes `RequiredInterface`. To adapt the instance of `AbstractDependency`, you have to compose it inside the `Adapter`. This way `dependencyMethod()` is now adapted to `requiredMethod()`.

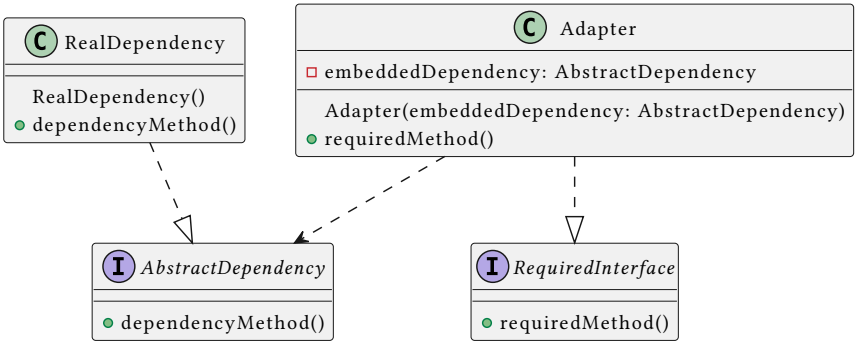


Figure 31: adapter

Whenever, an instance of `Adapter` calls `requiredMethod()` it instead delegates the behavior to the embedded `dependency`, which instead calls `dependencyMethod()`

Example

Printable Shipments

Looking back at our previous lab exercises, some of the example classes contain string representation but do not implement the `toString()` function. An example of this is `Shipment` back from the factory method example. It does contain a string representation builder called `shipmentDetails()`, but printing a shipment is quite tedious since you have to print, `s.shipmentDetails()`. You can replace the name of `shipmentDetails()` to `toString()` but this will potentially affect other clients of shipment. You can add the `toString()` function which does exactly the same but this may introduce unwanted code duplication.

The best solution for this problem is to create an adapter for shipment called `PrintableShipment`. This adapter will realize some `Printable` abstraction, which only contains the abstract method `toString()`.

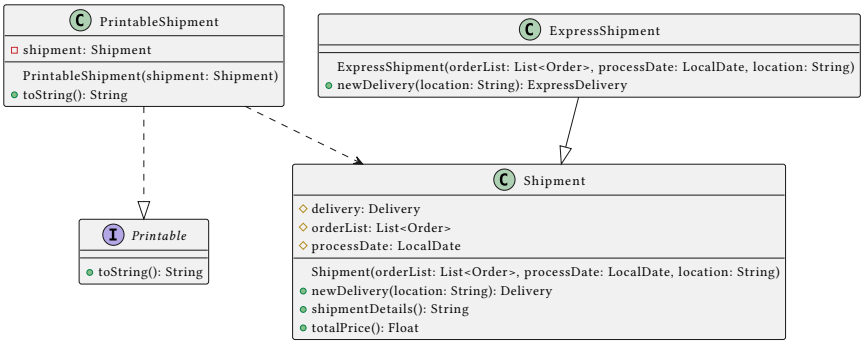


Figure 32: adapter example

Why this is elegant

- **Open/Closed Principle** - Instead of changing existing incompatible interfaces, you can extend them by creating adapters.
- **Interface Segregation Principle** - Instead of cluttering up your code with duplicate functions and unused interface methods, you instead create adapters only when it is needed.

How to implement it

1. If you want an `AbstractService` to be used like a `RequiredInterface`, Create a `ServiceAdapter` that realizes `RequiredInterface` and contains an attribute `service` that is a reference to an instance of `AbstractService`.
2. Inside `ServiceAdapter`, the implementations of `RequiredInterface`'s abstract methods are merely calls to the methods of the reference `service`.

Composite (Optional Read)

Problem

When entities in your system needs to be represented like trees, then you represent them like trees.

Solution

The composite pattern describes a tree structure described polymorphically. A tree node can either be a general tree or a leaf. in the composite pattern, a `Component` (tree node) can either be `Composites` (general tree), or a `Leaf`. Leaves and Composites are realizations of `Component`.

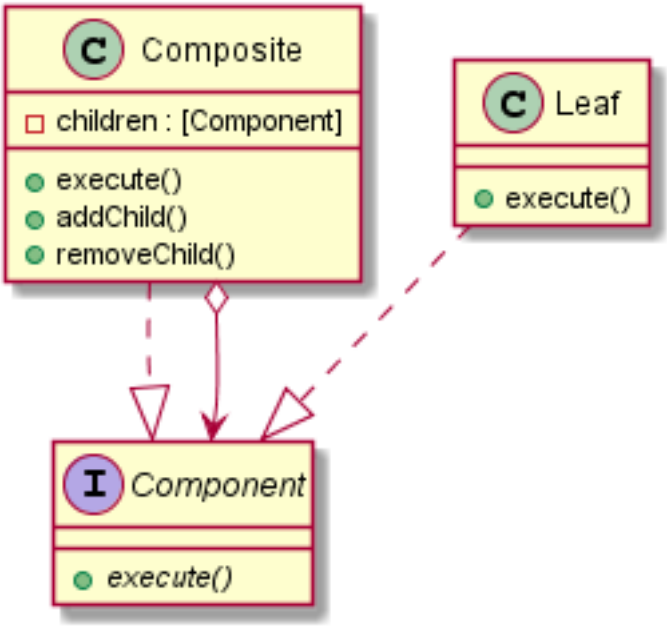


Figure 33: composite

Example

File System

The file system in your computers are described using a tree structure. The entities in your file system are either directories or files.

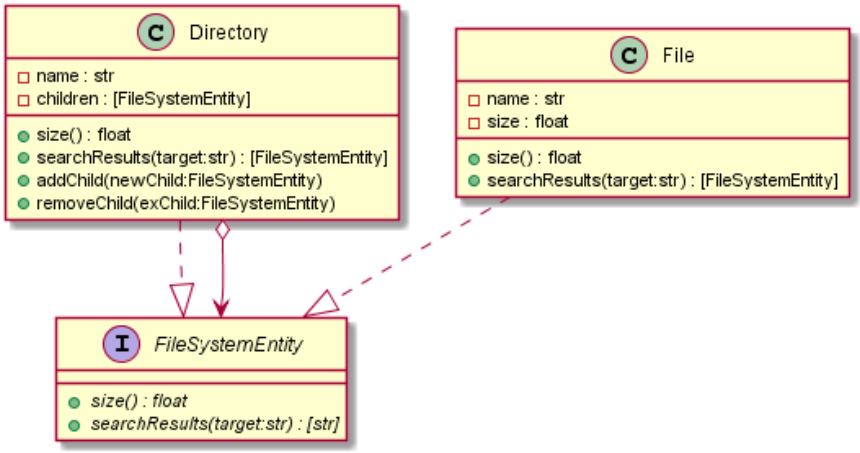


Figure 34: composite pattern

What you need to do is to implement a simulation of a file system. Each node of the file system should be able to call the following methods:

- **`size()`** - the size of a `File` is based on the attribute size which is set during the initialization of the `File` instance. The size of a `Directory` is the size of all the `FileSystemEntities` inside it.
- **`searchResults(target)`** - if used by a `File`, if the name of the file matches the `target` it returns a list containing the `File`, if not it returns an empty list. If used by a `Directory` returns a list containing all the of the instances of `FileSystemEntity` (including itself) that matches the target inside the `Directory`.

Why this is elegant

- **Open/Closed Principle** - You can introduce new component realizations in the system without touching any existing code.
- Working with composite trees are easier because of the polymorphism in the pattern

How to implement it

1. Create an abstraction called `Component` that contains the abstract methods that are supposed to be executed across all components.

2. The `Component` has two realizations, `Composites` and `Leafs`.
3. `Component` contains an attribute `children` which is a list of `Composite` instance references and the methods `addChild()` and `removeChild()` to attach/detach `Composites`. It also has `execute()` which an implemented method from `Component`.
4. `Leaf` contains the method `execute()` as well.
5. When a `Composite`'s execute is invoked, it calls invokes all of its children'sexecute(). When `Leaf`'s execute is invoked it performs, leaf related behavior.

Facade (Optional Read)

Problem

When your system becomes large enough, parts of the system which is responsible for a single operation may involve interactions between multiple classes. Creating flexible and maintainable systems tend to look like this.

Looking from the outside, simple functionality (like borrowing a book or depositing money to an account) will appear complex since it involves multiple lines of object interaction.

Solution

To solve this issue, you create a straightforward interface, that contains methods to encapsulate complicated functionality in your subsystem. Instead of using the internal classes to perform some functionality, you call the facade interface's method instead.

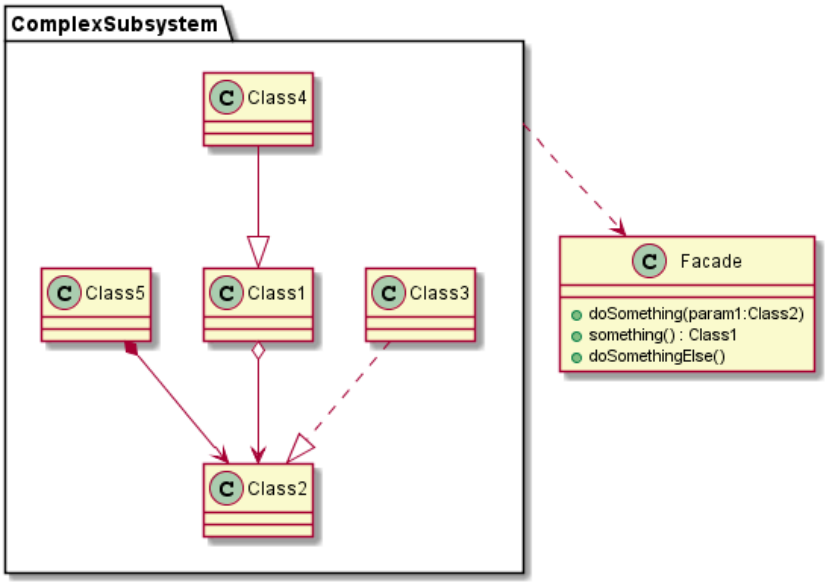


Figure 35: facade

Why this is elegant

- Implementing this pattern encapsulates complicated subsystem behavior into simple straightforward functions.

Optional Readings

Shvets A. (2018) Structural Patterns Accessed August 31, 2020

Behavioral Patterns

Introduction

Some systems require complex and extremely decoupled relationships. Behavioral patterns are used on these tightly interconnected systems so that they are easier to maintain. These patterns separate behavioral responsibilities among the classes in your system in such a way that volatile behaviors are encapsulated deep into your object structure.

Learning Outcomes

1. Design systems that apply the strategy pattern
2. Design systems that apply the state pattern
3. Design systems that apply the command pattern
4. Design systems that apply the observer pattern
5. Design systems that apply the template pattern
6. Design systems that apply the iterator pattern

Strategy pattern

Problem

Some systems require behavior that have to be parametrized for other behavior. This is easily done in a functional programming environment since higher order functions are used to represent these. In programming languages that don't support these features, the strategy pattern is used.

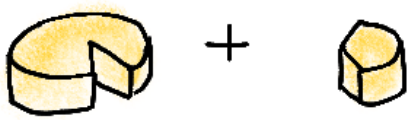


Figure 36: strategy

Solution

Functions that are not first class citizens are encapsulated inside a `Strategy` class. A strategy class simply contains the method `execute(params)`, which represents the behavior that should be passed into a higher order function. Any method that can be passed into the higher order function should realize `Strategy`.

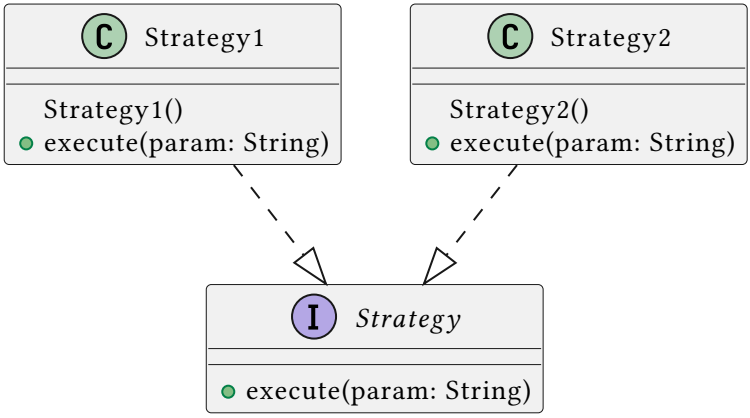


Figure 37: Strategy pattern

The object `params` represent the data that you need to pass into the correct class. In this pattern you pass the the whole `Strategy` realization so that `strategy.execute(params)` perform the desired behavior. You can add other methods in the `Strategy` abstraction, if it makes sense for the system.

Example

Fraction Calculations

You're creating a less sophisticated version of a fraction calculator. This calculator only has arithmetic operations inside it, addition, subtraction, division, and multiplication. Inside this calculator, a calculation is represented in a `Calculation` instance. Every calculation has four parts:

- `left` - represents the left operand fraction
- `right` - represents the right operand fraction
- `operation` - represents the operation ($+,-,\times,\div$)
- `answer` - represents the solution of the operation

Kotlin does indeed support higher order functions but your boss is anti-functional programming so they forbid the use these features. Because of this you decide to implement the strategy pattern.

To do this, you need to create an abstraction called `Operation` to represent the different operations. For each operation, you create a class that realizes `Operation`.

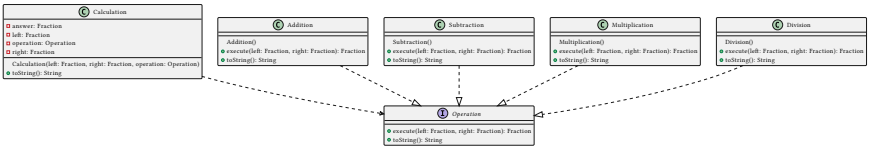


Figure 38: strategy pattern example

Why this is elegant

- **Open/Closed Principle** - If you want to add new strategies, you wouldn't need to touch any existing code.
- The implementation of a strategy is deeply tucked inside multiple layers of encapsulation. Changing these implementations are very easy.
- You can swap strategies during runtime in the same way you do in functional programming.

How to implement it

1. Create an abstract `Strategy` that contains an abstract method called `execute`. This method should be specified to accept all the necessary parameters needed by your parameterized function.
2. For every strategy, the higher order method can accept, you create a realization for `Strategy` and implement the correct behavior in `execute`.
3. The higher order function should now be specified to accept a `strategy` of type `Strategy`.
4. Inside the higher order function, whenever it wants to perform the strategies embedded behavior, call `strategy.execute(...)`.

State Pattern

Problem

Some objects can change into many different states. If different objects behavior is dependent on its current state, it would require, bulky and annoying if-else blocks to handle its dynamic behavior.

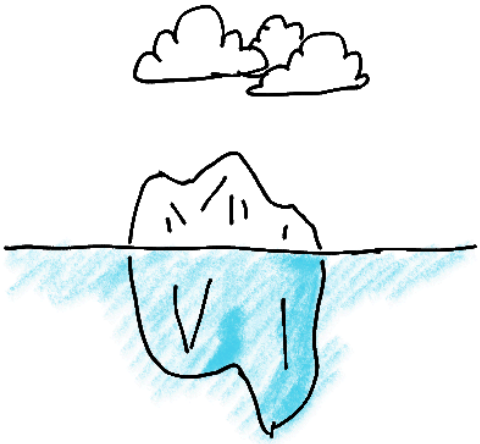


Figure 39: State

Solution

An object that can have many states should contain an attribute representing its state. Instead of performing, state dependent behavior directly inside the object, you delegate this responsibility to its embedded state instead. In this way the object will behave according to its current state.

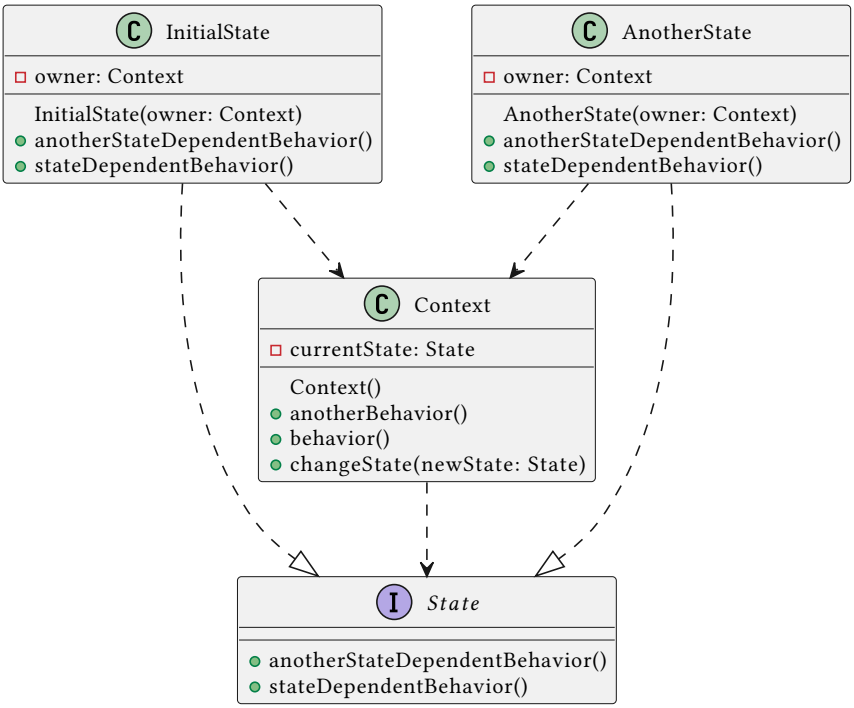


Figure 40: state pattern

The state may be required to contain backreference to the context object that owns it. This is only required if state methods requires to access/ control the context that owns it.

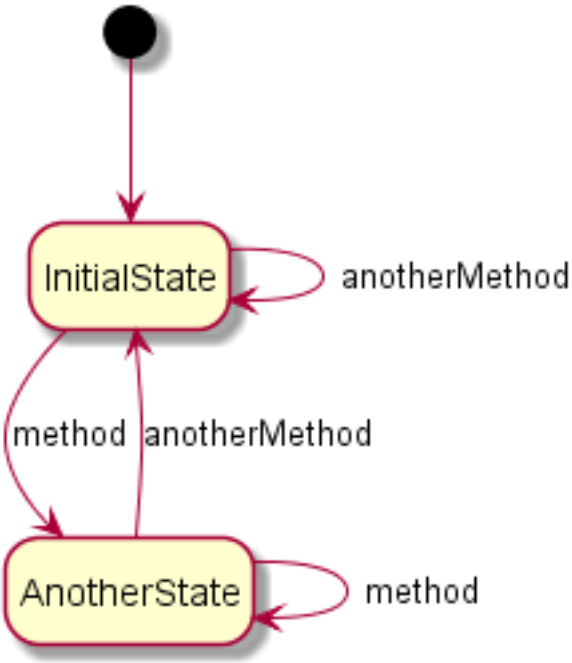


Figure 41: state diagram

Example

States of matter

The state of any given matter is dependent on the pressure and temperature of its environment. If you heat up some liquid enough it will turn to gas, if you compress it enough it will become solid.

The state diagram would look something like this:

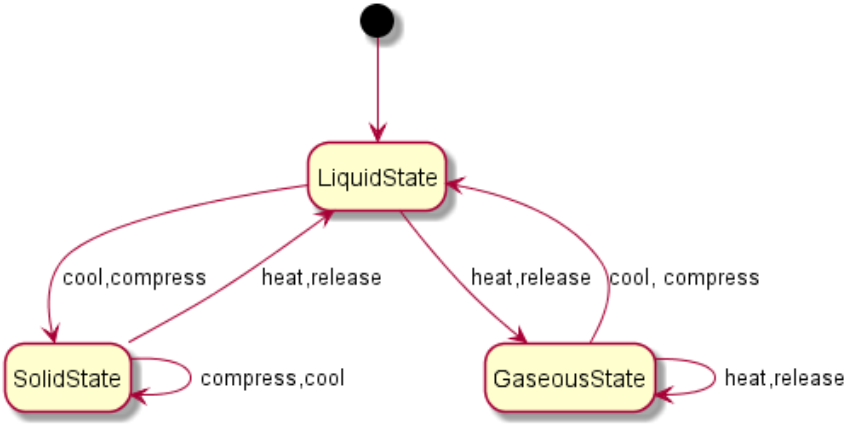


Figure 42: state diagram example

To implement something like this, you would need to create matter which owns an attribute called `state` which represents the matter's current state. Since there are three states, you create three realizations to a common abstraction to state.

When you compress/release/cool/heat the matter, you delegate the appropriate behavior and state change inside `state`'s version of that behavior. Each `State` realization will need a backreference to the `Matter` that owns it. This backreference will allow the `State` instance to change the `Matter`instance that owns said `State` intance.

> Delegating behavior to the composed state means that, when the `Matter` instances invoke, `compress()`, `release()` `heat()`, and `cool()`, the composed `State` owned by the matter calls its own version of `compress()`, `relaease()` `heat()`, and `cool()`.

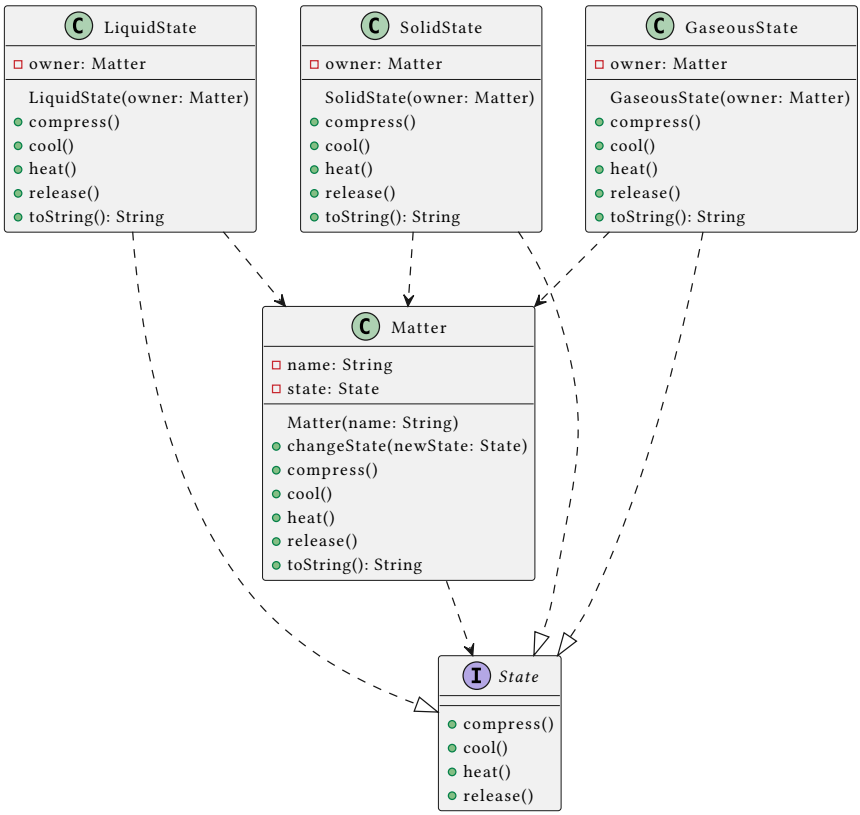


Figure 43: state example

Matter owns an instance of `State`, and that instance has an attribute called `matter`. The attribute `matter` is the reference to the instance of `Matter` that owns it. The `State` instance needs this reference so that it can change the matter's state when it is compressed, released, heated, or cooled.

Why this is elegant

- **Single Responsibility Principle** - behavior related to state is delegated to the state itself.

- **Open/Closed Principle** - You can incorporate new states to the system without touching any existing client code
- Implementing this pattern will remove bulky and annoying state conditionals

How to implement it

1. Create an abstract `State` that contains abstract methods for all state dependent behavior (context related behaviors that are dependent on context's state).
2. For every state the `Context` can have, create a realization of `State`.
3. `Context` owns an attribute that represents the current state (`currentState`) that it owns.
4. If the state needs to control the `Context` instance that owns it, add a backreference to `Context` inside state.
5. Whenever a `Context` instance performs state dependent methods, it calls `currentState.stateDependentBehavior()` instead so that its behavior is dependent on its current state.

Command Pattern

Problem

Sometimes, object behavior contain complicated constraints. Sometimes, the system require the behavior to be invoked by an object but performed by another (this is common in presentation layer/domain layer separation in MVC enterprise systems). Sometimes, the system requires behavior to be undone. Sometimes the system requires a history of the behaviors that were being performed.

Figure 44: Command (What does this strange picture mean?)

Solution

These problems have a common solution, the **command pattern**. A `Command` is a more powerful version of a `Strategy`. While both of them encapsulates behavior, a `Strategy` is just that, a function wrapper. A `Command` on the other hand contains which object performs the behavior, which parameters are needed to perform the behavior, and how to undo the command (if needed).

Creating `Commands`, allow for more flexible behavior responsibility assignments. A separate `Invoker` object triggers the behavior by creating a command. This `Invoker` prepares the command with the appropriate `Reciever` (the object performing the command), and the appropriate parameters. The invoker then executes the prepared command. The command doesn't actually do anything, it just tells the `Reciever` to call the appropriate method.

This separation of responsibility allows for the creation of extra features that may be required for your system:

- If you want to keep a history of the performed commands, the `Invoker` may keep a list of `Commands`. This way the data stored in the list history, is a perfect representation of the previous commands.
- If you want the `Commands` to be undoable, you can store a backup of the receiver (and other affected objects) by the `Command` inside each

instance of `Command` . Undoing a command will be as simple as restoring the receiver to its backup.

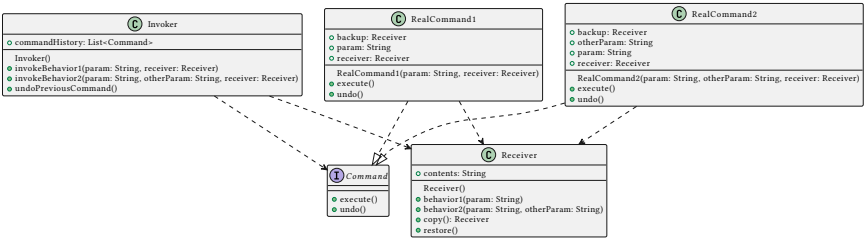


Figure 45: command example

Instead of passing the receiver in the `Invoker` methods, you can create an attribute called receiver inside `Invoker`. But doing this will make it so there is one `Receiver` instance for every `Invoker` instance.

The commands should only affect the receiver. If the behavior that is performed changes a lot of objects, then make a `Receiver` class that encapsulates all of the affected objects. Doing this will make the implementation of undo easier since the backup inside of the command will simply be an older version of `Receiver`.

Different command realizations are not necessarily of the same `Strategy`. That's why the parameters of the behavior are stored as attributes of the command, not passed in the `execute()` function. This is so that no matter what the command is, all `execute()` functions will have the same type signatures.

Example

Zooming through a maze

You're creating a maze navigation game thing. This is what the application currently has right now:

- `Board` - this represents the layout of the maze. The layout is loaded from a file. It has these attributes:
  - `__isSolid` - this is a 2 dimensional grid encoded as a nested list of booleans which represents the solid boundaries of the maze. For example if `__isSolid[row][col]` is true then it means that that cell on (row,col) is a boundary
  - `__start` - a tuple of two integers that represent where the character starts
  - `__end` - tuple of two integers that represent the position of the end of the maze
  - `__cLoc` - tuple of two integers that represents the current location of the character
  - `moveUp()`, `moveDown()`, `moveLeft()`, `moveRight()` - moves the character one space, in the respective direction. The character cannot move to a boundary cell, it will raise an error instead.
  - `canMoveUp()`, `canMoveDown()`, `canMoveLeft()`, `canMoveRight()` - returns true if the cell in the respective direction is not solid.
  - `__str()__` string representation of the board. It shows which are the boundaries and the character location

What's missing right now is controller support. This is how a player controls the character on the maze:

- `dpad_up()`, `dpad_down()`, `dpad_left()`, `dpad_right()` - The character dashes through the maze in the specified direction until it hits a boundary.
- `a_button()` - The character undoes the previous action it did.

To implement controller support you need to create a `Command` abstraction which is realized by all controller commands. The `Controller` (which represents the controller) is the invoker for the commands. Since commands are undoable, this controller needs to keep a command history, represented as a list. Every time a controller button is pressed, it creates the appropriate `Command`, executes it and appends it to the command history. Every time the `a_button()` is pressed to undo,

the controller pops the last command from the command history and undoes it.

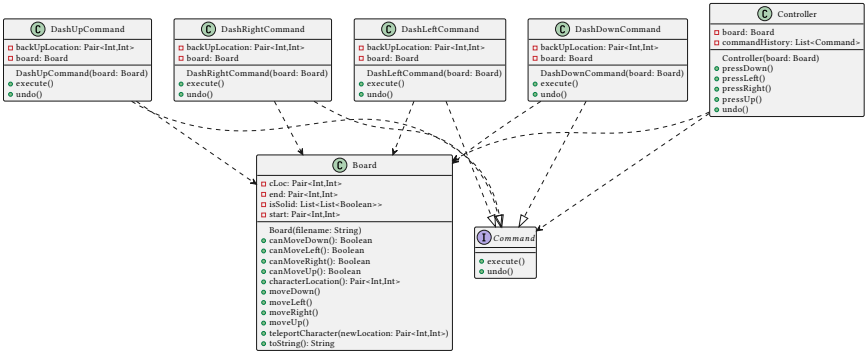


Figure 46: command example

Why this is elegant

- **Single Responsibility Principle** - The behavioral responsibilities in the system are thoroughly separated. One invokes the command, and the other performs the behavior associated with the command.
- **Open/Closed Principle** - If there are more commands you want to add, you don't have to touch any existing code.
- Switching between invokers and receivers is easily done
- You can implement undo (and redo)
- You can defer the execution of behavior
- A command may be made of smaller simpler commands

How to implement it

1. Create an abstraction `Command` that contains abstract method `execute()`, and other command related methods like `undo()`.
2. For every command, create a realization to `Command`. These commands uses a reference to a `Reciever` instance. This instance represents the instance/s that are affected whenever `Command` realizations are executed.
3. Create an `Invoker` class that will be responsible for instantiating, preparing, and executing commands. Inside these class are methods

for invoking each commands. When these methods are called, the invoker does the following:
    1. Instantiate an instance of `Command` called `c` with the correct realization.
    2. select the receiver of the `Command`, including the related parameters.
    3. invoke `c.execute()`.
4. If the system supports undoable commands, the `Invoker` should keep a list of commands called `commandHistory` and each command instance should keep a reference called `backup` to enable restoration of `Reciever` instances.

Observer Pattern

Problem

What if you need to inform a lot of objects about the changes to some interesting data? If you globalize the data and let your client objects poll for changes all the time, this will affect the security and safety of your interesting data. Plus, global data is something that should be avoided as much as possible. Also, forcing your objects to poll for changes all the time will be inefficient if your interesting data has not changed.

Figure 47: observer

Solution

The responsibility of sharing information about the changes to interesting data should not be placed in the clients of the data. You should create a notifier class that encapsulates the interesting data. This class should be responsible of notifying interested clients about changes on the subject.

To do this you need to encapsulate the interesting data (from now on lets call it the `subject`), into a `Publisher` class. An instance of this class will be responsible of notifying the observers for any change in the `subject`. Whenever there are changes to the subject, the `Publisher` instance calls `notifyObservers()` so that all interested observers will be informed of the change. Any class that is potentially interested in the `subject` should realize an `Observer` abstraction, which in the bare minimum contains, the `update(updatedSubject)` function. Inside `Publisher`'s `notifyObservers()` method, every subscriber (an interested observer) is updated (`subscriber.update()`).

Any instance of an `Observer` should be subscribed to the change notifications using `Publisher`'s `subscribe()` function. They can also be unsubscribed using the`unsubscribe()` function.

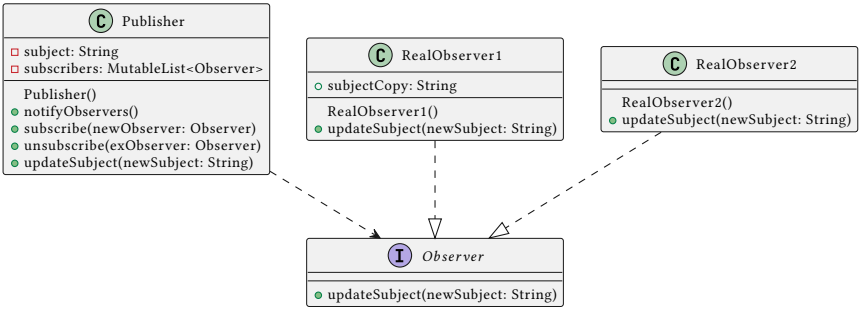


Figure 48: observer

Whenever an observer has updated, the publisher needs to pass all the necessary details in the notification. This is generally done by passing the updated subject in the `update(updatedSubject)` method.

In some cases, the observer needs to keep a copy of the subject as an attribute. Make sure to change the value of this attribute during updates.

Make sure that changes to the subject are only done using the `Publisher` class (`manipulateSubject()`). If you change the subject without using `Publisher`'s methods, your subscribers won't be notified.

Example

Push Notifier for Weather and Headlines

You are creating a push notification system that works for multiple platforms. You want to distribute information about the current weather and news headlines. This system will be potentially used on many

platforms so you have to think about the maintainability issues for adding new platform support.

To implement this, you have to apply the observer pattern. Your subject would be `Weather` data and `Headline` data (which are their own classes). These subjects should be encapsulated into a single publisher class (which will be called `PushNotifier`).

Any platform, that is interested in the changes to the subject should realize a `Subscriber` abstraction (Observer), which contains the abstract method update().

You need to implement th following observer realizations:

- `EmailSubscriber` - every time the `currentWeather` or `currentHeadline` changes you send an email to the specified `email`. (you don't have to actually send an email. You can just simulate sending an email by printing something like "sending and to ").
- `FileLogger` - every time the `currentWeather` or `currentHeadline` changes, you append the updated weather and headline to the file specified in the attribute `filename`. You actually have to update a file via kotlin/python file writing.

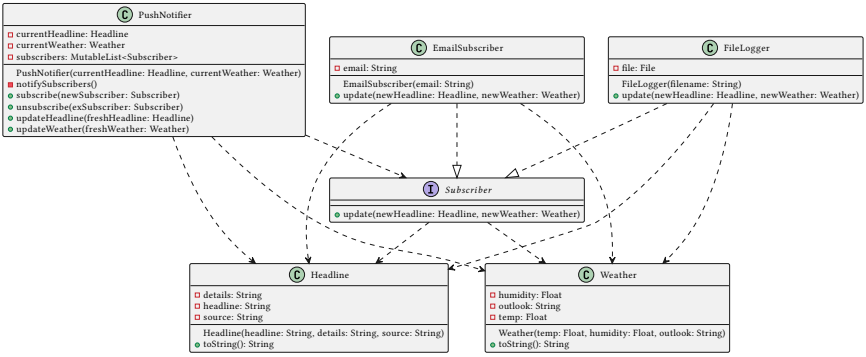


Figure 49: observer example

Why this is elegant

- **Open/Closed Principle** - You can add new `Observer` realizations seamlessly every time there are new objects that are interested in the subject.
- A observer can be subscribed/unsubscribed during runtime

How to implement it

1. Create an `Observer` abstraction that represents all classes that can potentially observe changes to the `Publisher`. `Observer` will contain the abstract method `update()`.
2. All classes that want to be notified about changes to the `subject` should realize `Observer`.
3. `Publisher` will either own/use an instance of the `subject` of interest. It will also use an attribute which is stores the list of `Observers` that are interested in the subject. To attach or detach `Observers`, `Publisher` contains the methods `subscribe()` and `unsubscribe()`.
4. Every time `subject` is manipulated, it should be done through `Publisher` , because `Publisher` needs to notify all `Observers` in its observer list attribute after every manipulation. This notification is done through `notifyObservers()` after the end of every subject manipulation.
5. Inside the `Publishers` `notifyObservers` method, every `Observer` in its list of observers invoke their `update()` method.

Template Method Pattern

Problem

Say you have two or more *almost* identical behaviors from different classes. Rewriting these object behaviors as separate methods for each class duplicates many parts of the code (especially if the behavior has a lot of lines of code).

Figure 50: template

Solution

To avoid code duplication, you break down your code into individual steps. By doing this you can create a superclass that contains the implementation for all common steps. This superclass will also contain the common implementation for the **template method**, the method that combines all steps into the original object behavior. Differences between steps will be resolved under different specializations of this superclass.

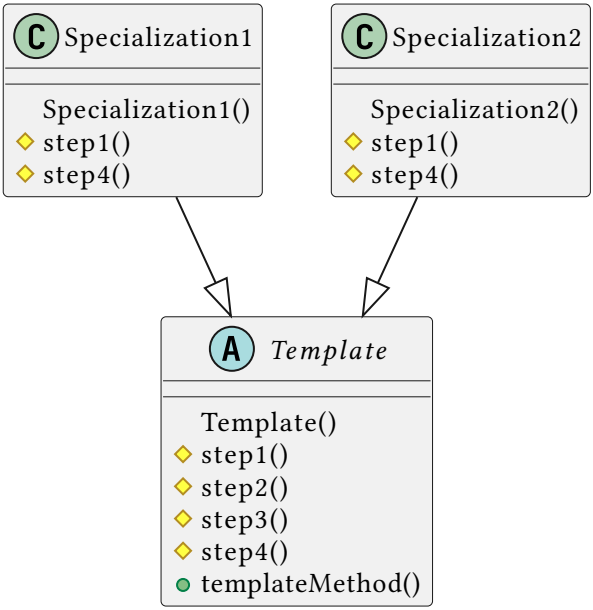


Figure 51: template method

The method `templateMethod()` calls each of the steps defined in `step1()`, `step2()`, `step3()` and `step4()`. The exact type of the instance (either `Specialization1` or `Specialization2`) decides which versions of the steps are executed. For example, if the exact instance of the class is a `Specialization1`, it performs the template method with special versions of `step1()` and `step4()` (since `Specialization1` overrides them) but the other parts are inherited from the `Template`.

> The steps in the superclass can be a mix of abstract methods and concrete methods. Make a method abstract if you want to force all specializations to override these steps. You'll want to do these if some of the steps in your template doesn't have a default implementation.

Example

Brute Force Recipe

If you write brute force algorithms as search problems, they will have a common recipe. This is the reason why it is called the exhaustive search algorithm. It will traverse all of the elements in the search space, trying to check the validity of each element, until it completes the solution

Equality Search

Search for integers equal to the target

```
#searchSpace = [2,3,1,0,6,2,4]
#target = 2

i = 0
solutions = []
candidate = searchSpace[0]
while(i<len(searchSpace)):
    if candidate == target:
        solutions.add(candidate)
    candidate = searchSpace[++i]

#solution = [2,2]
```

Divisibility Search

Search for integers divisible by the target

```
#searchSpace = [2,3,1,0,6,2,4]
#target = 2

i = 0
solutions = []
candidate = searchSpace[0]
while(i<len(searchSpace)):
    if candidate % target == 0:
        solution.add(candidate)
    candidate = searchSpace[++i]

#solution = [2,0,6,2,4]
```

Minimum Search

No target, searches for the smallest integer

```
#searchSpace = [2,3,1,0,6,2,4]
#target = None

i = 1
solutions = [searchSpace[0]]
candidate = searchSpace[1]
while(i<len(searchSpace)):
    if candidate <= solutions[0]
        solutions[0] = candidate
    candidate = searchSpace[++i]

#solution = [0]
```

Common Recipe

```
i = 0
solutions = []
candidate = first()
while(isStillSearching()):
    if valid(candidate):
        updateSolution(candidate)
    candidate = next()
```

Because of this we can write a general brute force template method that would return the solution to brute force problems. To do this you create a superclass `SearchAlgorithm()` that contains the template method for brute force algorithms. If you want to customize this algorithm for special problems, all you have to do is to inherit from `SearchAlgorithm` and override only the necessary steps.

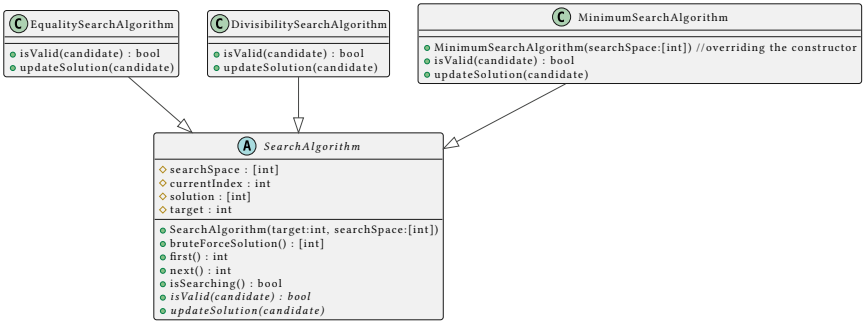


Figure 52: template example

is `isValid()` and `updateSolution(candidate)` is different for each algorithm so it doesn't have a default implementation. It would be best to make these steps abstract.

Why this is elegant

- **Open/Closed Principle** - The `Template` is open for extension but closed for modification
- *Encapsulate what varies* - steps can vary from specialization to specialization, therefore they are encapsulated into step methods.
- Implementing this pattern will remove duplicate code in the common parts of the algorithm.
- Clients may override only certain steps in a large algorithm, making it easier to create specializations

How to implement it

1. Create an abstract class called `Template`. It contains the method `templateMethod()` broken down into separate steps through separate `step1()`, `step2()`, ... and etc. methods. When invoked `templateMethod()` will just call these step methods.
2. Each step method inside `Template` will contain the default implementation of that step. If there is no default implementation, the method should be abstract.

3. For every similar behavior to `templateMethod()` a specialization of `Template` is created. These methods will implement all abstract methods and override all step methods that vary for its behavior.

Iterator

Problem

One of the most common iteration recipes that you'll likely implement is the **for-each** loop. This loop traverses a collection, and performing some kind of operation along the way. Most programming languages implement for each loops on built in collections like arrays, sets, and trees. But what about non-built in collections?

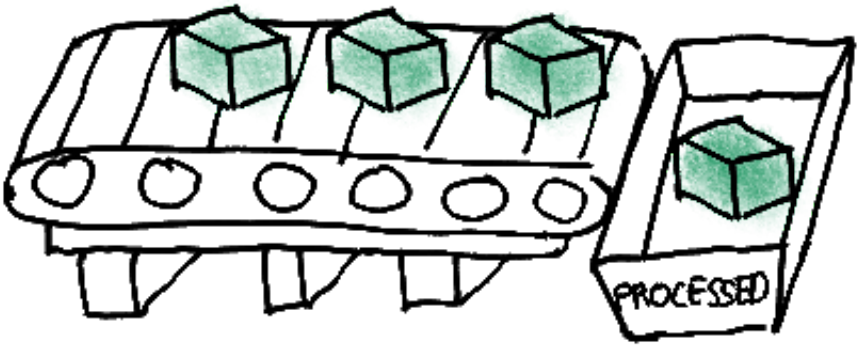


Figure 53: iterator

Solution

For non built-in collections, you can create an iterator that does the traversal for you. On the bare minimum these iterators will realize some `Iterator` abstraction that contains the methods, `next()`, and `hasNext()`. From these methods alone you can easily perform complete traversals without knowing the exact type of the collection:

```
i = collection.newIterator()
while i.hasNext():
    print(i.next())
```

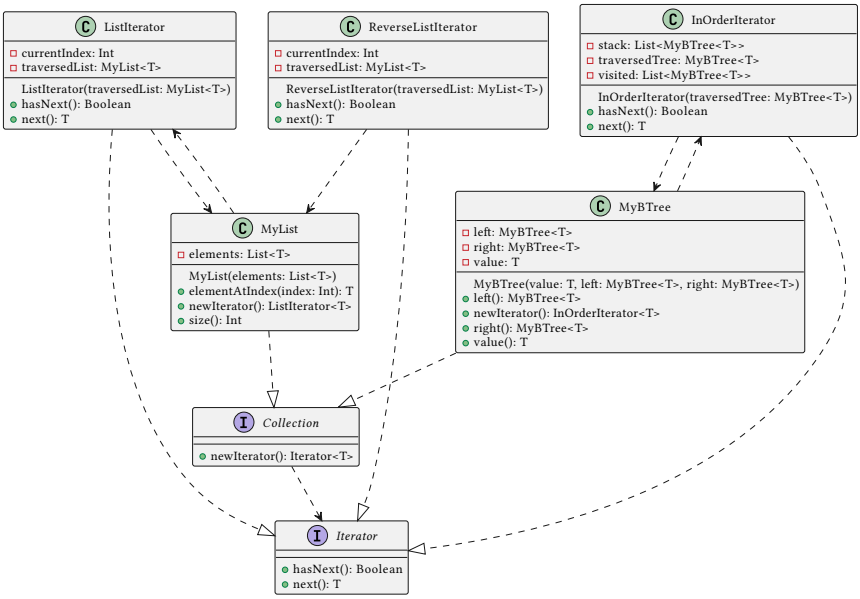


Figure 54: iterator

The `hasNext()` method, returns a boolean value that indicates whether or not there are more elements to be traversed. The `next()` method, returns the next element in the traversal.

A collection can have more than one `Iterators`, if it makes sense for the collection to be travsersed in more than one way. Despite this possibility, a collection must have a default iterator which will be the type of the new instance returned in the factory method, `newIterator()`

Why this is elegant

- **Single Responsibility Principle** - Traversal algorithms can now be placed into separate classes that interact with an iterator instead of the collection itself.
- **Open/Closed Principle** - You can implement new types of collections and iterators without touching any existing code.

- You can traverse all the elements in a collection, even if you don't know the exact type of the said collection.
- Two iterators, can iterate over the same collection without problem as long as the iterators are of different instances.

How to implement it

1. Create an abstraction called `Iterator` which contains the abstract methods `next()` and `hasNext()`.
2. Create an abstraction called `Collection` which contains the abstract method `newIterator()`.
3. For very collection that can be iterated through create a realization to `Collection`. Inside these `Collection` realizations, the `newIterator()` method must be implemented which simply returns a new instance of the default `Iterator`. (for collections that can be iterated through in more than one way, create different methods for creating other iterators as well but always keep `newIterator()` as the one that returns a new instance of the default iteration).
4. For every different way of iterating through a `Collection` realization, a realization to `Iterator` must be created as well.
5. `Iterator` realizations should contain an attribute that refers to the collection instance it is iterating through.

Optional Reading

Shvets A. (2018) Behavioral Patterns Accessed August 31, 2020

Haskell Introduction

Setting up Haskell

To start writing Haskell code, install Haskell through stack. Stack is found in the folder called "Haskell/Stack" inside the provided course pack. Copy the Stack folder and place it in your computer. To be able to use stack anywhere, add your copy of the stack folder in the PATH variable of your computer.

Once stack has been set up using the steps above, you can run the GHC repl using the command

```
> stack ghci
```

The first time you run this code, stack will automatically install the GHC compiler.

After downloading GHC, you will be taken to the Prelude part of your GHC repl. To test if everything is working properly, try the following Haskell expression:

```
Prelude> show (1 + 3)
```

If everything is good to go, the GHC expression will evaluate to:

```
"4"
```

To exit GHC run the following GHC command

```
:quit
```

To load a Haskell program, enter the GHC repl first

```
> stack ghci
```

While inside `Prelude`, use the command `:load <Path to haskell file>`. For example

```
Prelude> :load "Trying Things.hs"
```

If the path to the haskell file contains spaces, you need to enclose the path in quotes.

Binding

To bind a value to some identifier use the = operator

```
identifer = <some expression>
```

The expression to the left of the = operator is evaluated and then bound to the identifier to the right of the = identifier.

Example

the integer 3 bound to `x`

```
x = 3
```

a lambda bound to `f`

```
f = \x -> x + x
```

Type annotations

The last type in the arrow series is the range type or return type. Every type before it are the domain types or the types of the parameters in order

```
function_name :: Type_of_Param1 -> Type_of_Param1 -> ... ->
Type_of_Paramn -> ReturnType
```

Examples

Double of an integer, double : $\mathbb{Z} \to \mathbb{Z}$

```
double :: Int -> Int
```

Sum of the length of two strings (strings in Haskell are arrays of `Char`, an array of specific types are written enclosed in square brackets, `[Type]` represents an array of `Type`)

```
len_sum :: [Char] -> [Char] -> Int
```

Check if some integer is even (boolean values are written capitalized, `True` and `False`)

```
isEven :: Int -> Bool
```

Accept two `(Int -> Int)` functions and produce the composition of those functions (given functions f and g, $f \circ g$)

```
compose :: (Int -> Int) -> (Int -> Int) -> (Int -> Int)
```

Type Variables

Haskell also supports type variables. In order for functions to support multiple types, you can use type variables instead of concrete types like `Int`, `Char`, `[Int]`.

Types written in lower case are considered as type variables. For example, you can define a more general `len_sum` function with the following type annotation:

```
len_sum :: [a] -> [b] -> Int
```

By defining the function this way, it doesn't only accept two Strings (`[Char]`), it now accepts any two lists.

You can also define a more general `compose` function in the following way:

```
compose :: (a -> b) -> (b -> c) -> (a -> c)
```

Since you have defined that the first parameter is of type `(a -> b)` and the second parameter is of type `(b -> a)`, the types labelled `b` should match. In this case, the return type of the first function must match the accepted type of the second function. The returned function is of type `(a -> c)`, this means that this returned function must accept a type that matches the first parameter's accepted type (`a`), and must return a type that matches the second parameters return type, (`c`).

Typeclasses

Sometimes you might want to add some constraints to the allowed type in the type variable. This is where you would use typeclasses.

By declaring the typeclass of a type variable, you will only be able to use types that match the typeclass specified.

For example, when you define a more generalized version of the `double` function as such:

```
double :: a -> a
```

You might want to add an extra restriction to only allow, numerical types. By adding that restriction, you will disallow the usage of double on strings, lists, chars, etc. But you will allow the usage of integers, floats, doubles, etc. To restrict `a` to only numerical types, you can use the `Num` typeclass.

```
double :: Num a => a -> a
```

The typeclass prefix `Num a =>` means that type variables `a` can only be types under the `Num` subclass (i.e. `Int`, `Integer`, `Double`, `Float`).

Function definitions

Every identifier placed in between the function name and = are the parameter names. The expression to the right of = is the expression evaluated when the function is called.

```
function_name param1 param2 ... paramn = <some expression>
```

Example

Double function

```
double x = x + x
```

isEven function

```
isEven n = n % 1
```

Function/Lambda Application

Two expression separated by a space is a function application. The expression on the left side must evaluate to a function or a lambda. This function/lambda is applied to the right expression as its parameter

```
<expression1> <expression2>
```

A series of expressions are curried multiparameter applications. The leftmost expression must evaluate to a function or a lambda. This function/lambda is applied to the right expressions as its parameters.

```
<expression1> <expression2> <expression3> ... <expressionn>
```

Example

```
double 3
```

evaluates to 6

```
compose addThree double 2
```

evaluates to 7

Lists

Lists in haskell can be defined as comma-separated values in between brackets.

```
[element1, element2, element3]
```

Lists have to be homogeneous[26], i.e. all of its elements must match the same type.

Examples

[26] Look into tuples for heterogeneous collections.

```
[1,2,3,4]
['a','b','c','d']
```

You can also create lists using the following pattern. Such as list is valid if `tail` is a list and `head` matches the type of the elements in `tail`

```
head : tail
```

Examples

```
1:[2,3,4]
```

evaluates to `[1,2,3,4]`

```
'a':('b': ['c', 'd'])
```

evaluates to `"abcd"`[27]

```
1:2:3:4:[]
```

The `[]` expression evaluates to an empty list.

evaluates to `[1,2,3,4]`

You can also create regular series as lists by specifying the start and end of the list:

```
[start..end]
```

Examples

```
[1..6]
```

evaluates to `[1,2,3,4,5,6]`

```
[-3..4]
```

evaluates to `[-3,-2,-1,0,1,2,3,4]`

If-then-else expression

One of haskell's condition expressions are if-then-else expression. Because a haskell expression is required to evaluate to something, unlike C, all `if` parts must be followed by a `then` part and `else` part.

[27] A list of `Chars` will evaluate to a string.

```
if <bool-exp> then <exp1> else <exp2>
```

The expression inside the if-clause must be an expression that evaluates into a boolean value. If the expression `<bool-exp>` evaluates to `True`, then the whole if-then-else expression evaluates to whatever `<exp1>` evaluates to. If `<bool-exp>` evaluates to `False`, then the whole if-then-else expression evaluates to whatever `<exp2>` evaluates to. The then clause and else clause cannot be empty, and both clauses must evaluate to the same type

Haskell boolean literals start with uppercase letters, `True` and `False`.

Examples

The expression:

```
if (2 > 1) then 5 else 4
```

evaluates to `5`

The expression:

```
8 * (if (3 <= 2) then 2 else 3)
```

evaluates to `24`

If-then-else statements can be nested by writing if statements inside the `then` part and `else` part

```
if (2 > 1) then (if (0 == 1) then 2 else (1 + 2)) else (if (2
== 2) then 5 else (if (3 == 2) then 6 else 7))
```

evaluates to `3`

If else statements can be written neatly with tabs and newlines like this:

```
f :: Int -> Int
f x = if (x > 1)
  then (if (x == 1)
    then 2
    else (1 + 2))
  else (if (x == 2)
    then 5
```

```
    else (if (3 == x)
      then 6
      else 7))
```

Guards

To avoid long nested `if-then-else` expressions, you can use guards.

```
function param
    | boolean_expression1 = expression1
    | boolean_expression2 = expression2
    | boolean_expression3 = expression3
    ...
    | otherwise = expression4
```

Guard expression are read top to bottom, if it encounters a boolean expression that evaluates to `True`, the function will evaluate to the corresponding expression.

Examples

```
f :: Int -> [Char]
f x
    | x == 1 = "option1"
    | x == 2 = "option2"
    | x == 3 = "option3"
    | otherwise = "invalid option"
```

```
capped :: Num a => a -> a
capped x
    | x > 100 = 100
    | x < 0 = 0
    | otherwise = x
```

```
f :: Int -> [Char]
f x
    | x == 1 = "option1"
    | x == 2 = "option2"
    | x == 3 = "option3"
```

A guard block with no `otherwise` clause is still valid, but it can cause run time errors if none of the boolean expression evaluate to `True`.

Pattern matching

Pattern matching is a powerful tool that can also replace `if-then-else` expressions and guard blocks. For example, we can define the factorial function as a function that evaluates into some `if-then-else` expression, or a guard block. This is because we need to separate the evaluation for the base case and the recursive case.

```
factorial :: Int -> Int
factorial n =
    if (n == 1) then 1
    else n * factorial (n - 1)
```

But with pattern matching, you can declare multiple definitions under the same function name where the arguments match to different patterns of values.

```
factorial :: Int -> Int
factorial 0 = 1
factorial n = n * factorial (n - 1)
```

Since the `factorial` name is defined with distinct patterns of values, these definitions can coexist with one another. Whenever you call the `factorial` function with a given argument, Haskell will try to match it with all the possible patterns starting from top to bottom. For example, if you evaluate `factorial 0`, it will match to the first pattern (`factorial 0`) and evaluate to 1. If you evaluate `factorial 1`, it will match to the second pattern (`factorial n`) and evaluate to the recursive case `n * factorial 0`.

Pattern matching also allows you to break down data structures into different binding. For example, you can define a function called `secondElement` which returns the second element of a list. To achieve this we use the `:` list constructor. In this example we use the specific pattern, `(e1:e2:rest)` Any list that can be deconstructed into such pattern will match. For example, the list `[1,2,3,4]`, can be deconstructed into `1:2:[3,4]`, (`e1 = 1`, `e2 = 2`, `rest = [3,4]`). The list `['a','b']` can also be deconstructed into `'a':'b':[]`, (`e1 = 'a'`, `e2 =`

`'b', rest = []`). This means that the pattern can match any list with length two or greater.

```
secondElement :: [a] -> a
secondElement (e1:e2:rest) = e2
```

From the deconstruction, haskell also creates a local binding that binds the first element into `e1`, the second element into `e2`, and the rest of the list into `rest`. We can use this binding in the functions definition.

Since the `(e1:e2:rest)` pattern only matches lists with 2 elements or more, the pattern is non exhaustive. We can add a definition that matches the wildcard pattern (_). The wildcard pattern matches any expression, making our function exhaustive.

```
secondElement :: [a] -> a
secondElement (e1:e2:rest) = e2
secondElement _ = error "list not long enough"
```

Make sure that the wildcard pattern appears last so that it doesn't subsume the expressions matched by `(e1:e2:rest)`.

You can also use the wildcard pattern to ignore binding unused parts of the data structure. For example, for the first pattern, we only need the binding `e2` so we can replace the others with wildcards.

```
secondElement :: [a] -> a
secondElement (_:e2:_) = e2
secondElement _ = error "list not long enough"
```

Here's another example, we can create a function that returns `True` if the list has exactly one element, and false if otherwise.

```
isSolo :: [a] -> Bool
isSolo [_] = True
isSolo _ = False
```

Alternatively:

```
isSolo :: [a] -> Bool
isSolo (_:[]) = True
isSolo _ = False
```

Let Expressions

To create a local binding within an expression, you can use a let binding. With a `let` expression like below, you can use `identifier` in the context of `expression`, its value will be `bound_expression`.

```
let identifier = bound_expression in expression
```

Examples

```
let x = 12 in 4 + x * x
```

evalauates to 148

```
(let u = 13 in 2 + u) - (let u = 11 in 10 + u)
```

evaluates to −6

Where blocks

You can create multiple local bindings inside the `where` block of functions. All of the bindings inside the `where` block can be used in the scope of the function definition.

```
function arg arg = definition
    where
        identifier1 = bound_expression1
        identifier2 = bound_expression2
        ...
```

Examples

```
fullname :: [Char] -> [Char] -> [Char] -> [Char]
fullname fname mname lname = fname ++ " " ++ [minitial] ++ ".
" ++ lname
    where minitial = head mname
```

You can even use pattern matching in your `where` blocks

```
fullname :: [Char] -> [Char] -> [Char] -> [Char]
fullname fname mname lname = fname ++ " " ++ [minitial] ++ ".
" ++ lname
    where (minitial:_) = mname
```

Lambdas

```
\param1 param2 ... paramn -> <some expression>
```

All identifiers between `\` and `->` are the parameters of the lambda. The expression to the right of `->` evaluates when the lambda is applied.

Example

A lambda that doubles a number

```
\x -> x + x
```

A lambda that adds two numbers

```
\a b -> a + b
```

The same function but written in its verbose uncurried form

```
\a -> (\b -> a + b)
```

Recursion from Fixed Point Combinators

Haskell and other functional programming languages are based on lambda calculus formalism. But how is recursion achieved from lambda calculus?

In lambda calculus you've seen how we sometimes give lambda calculus expressions identifiers. We do this to for convenience. This allows us to define lambda calculus expressions based on other lambda calculus expressions.

For example, we define the successor function[28] as the following.

$$S(\overline{n}) = S(\lambda s.\lambda z.s^n(z)) = \lambda s.\lambda z.s(s^n(z)) = \lambda s.\lambda z.s^{n+1}(z)$$

We can reuse the successor functions definition in the definition for the addition functions.

[28] Refer to Addition and Multiplication.

$$\text{add} = \lambda m.\lambda n.mSn$$

The identifiers S and add are just a means for us to avoid rewriting definitions. If we want to, functions can be truly anonymous. This becomes an issue when it comes to recursive functions. In such cases the function being defined contains a reference to itself.

```
summation :: Int -> Int
summation 0 = 0
summation n = n + summation (n - 1)
```

To simulate recursion in the formalism of lambda calculus, we use special functions, called **fixed point combinators**. A fixed point of a given function, is a value, which has the property of being its own image. Generally, if you have a function f, u is a fixed point of f if $f(u) = u$. In the example below, we have a quadratic function with two fixed points.

$$f(x) = x^2 + 3x - 3$$
$$f(-3) = 9 - 9 - 3$$
$$f(-3) = -3$$

$$f(1) = 1 + 3 - 3$$
$$f(1) = 1$$

In the example below, the horizontal shear transformation has infinitely many fixed points.

$$\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} u \\ 0 \end{bmatrix} = \begin{bmatrix} u \\ 0 \end{bmatrix}$$

All functions in lambda calculus have `fixed points`. This can be proven using a *fixed point combinator*. A fixed point combinator is a special function that can produce the fixed point of any lambda calculus function. There are many fixed point combinators, the example below, known as the Y-combinator is discovered by Haskell Curry.

$$Y = \lambda f.(\lambda x.f(xx))(\lambda x.f(xx))$$

We can show that this is indeed a fixed point combinator by applying it to a general function F.

$$\begin{aligned} YF &= (\lambda f.(\lambda x.f(xx))(\lambda x.f(xx)))F \\ &= (\lambda x.F(xx))(\lambda x.F(xx)) \\ &= F((\lambda x.F(xx))(\lambda x.F(xx))) \end{aligned}$$

$$YF = F(YF)$$

To end up with the fixed point property $YF = F(YF)$, we substitute YF, with the second reduction. This shows any function F has a fixed point YF.

If we apply the fixed point combinator to the successor function S, we end up with an infinitely expanding repeated applications of S.

$$\begin{aligned} YS &= S(YS) \\ YS &= S(S(YS))) \\ YS &= S(S(S(YS))) \\ &\vdots \\ YS &= S(S(S(\cdots S(YS)\cdots))) \end{aligned}$$

You can write a fix point combinator in Haskell by using the fix point property $YF = F(YF)$.

```
y :: (a -> a) -> a
y f = f (y f)
```

As you can see this definition contains a recursive call. We can avoid a recursive call by using the property of all fix points: u is a fixed point if $f(u) = u$[29].

[29]The where clause `u = f u` from `y` and let clause `x = f x` are circularly dependent definitions that might be a problem for other languages. But since Haskell is lazily evaluated, it will not end up being a syntax error.

```
y :: (a -> a) -> a
y f = u
    where u = f u
```

In fact this is very similar to how the `fix` function is defined, Haskell's built-in fixed point combinator[30].

```
fix :: (a -> a) -> a
fix f = let x = f x in x
```

When you use the fixed point combinator on any `(a -> a)` function, it also ends up as repetitive application of a function, repeated an infinite amount of times. For example, given the successor function `s`.

```
s :: Int -> Int
s n = n + 1
```

When applying the `fix` to `s`, the evaluation looks like this:

```
fix s
let x = f x in x
let x = f x in (f x) --substitute value of x with (f x)
let x = f x in (f (f x)) --substitute value of x with (f x)
again
let x = f x in (f (f (f x)))
...
let x = f x in (f (f (f ... (f x) ...)))
```

Note that what actually happens under the hood in Haskell is more complicated than this, but in for the purposes of visualization, let's assume that this is correct.

We can use this repetitive nature of `fix` to show how recursion can be achieved without recursive calls.

Using the summation function as an example:

[30]The where clause `u = f u` from `y` and let clause `x = f x` are circularly dependent definitions that might be a problem for other languages. But since Haskell is lazily evaluated, it will not end up being a syntax error.

```
sum :: Int -> Int
sum 0 = 0
sum n = n + sum (n + 1)
```

First, we convert summation into a higher order function that accepts some `r :: (Int -> Int)` as a first argument and some `n :: Int` as a second argument.

```
sum :: (Int -> Int) -> Int -> Int
sum r 0 = 0
sum r n = n + sum (n - 1)
```

We then replace the recursive call to `sum` with `r`

```
sum :: (Int -> Int) -> Int -> Int
sum r 0 = 0
sum r n = n + r (n - 1)
```

To complete the repetition, we apply `fix` to `sum`. The resulting function will be our completed `summation` function.

```
summation = fix sum
```

To see how this is equivalent to the recursive version, we can simulate its evaluation when `summation` is applied to `3`.

```
summation 3
(fix sum) 3
(let x = sum x in x) 3
(let x = sum x in (sum x)) 3 --evaluate the partial
application, (sum x)
(let x = sum x in (\n -> sum x n)) 3
(let x = sum x in (sum x 3)) --apply lambda to 3
(let x = sum x in (3 + x 2)) --evaluate (sum x 3)
(let x = sum x in (3 + (sum x) 2)) --evaluate x based on x =
sum x
(let x = sum x in (3 + (\n -> sum x n) 2))
(let x = sum x in (3 + (sum x 2)))
(let x = sum x in (3 + (2 + x 1)))
(let x = sum x in (3 + (2 + (sum x) 1)))
(let x = sum x in (3 + (2 + (sum x 1)))) --partial
application and apply to 1
```

```
(let x = sum x in (3 + (2 + (1 + x 0))))
(let x = sum x in (3 + (2 + (1 + (sum x) 0))))
(let x = sum x in (3 + (2 + (1 + (sum x 0)))))
(let x = sum x in (3 + (2 + (1 + 0)))) --evaluate sum x 0 = 0
(base case)
(3 + (2 + (1 + 0))) --evaluate the let binding
6
```

You can achieve the same result in fewer steps using the fix point `y f = f (y f)`.

Actual recursion in haskell is achieved using a cyclical graph, which is not too dissimilar to the circularly dependent fix point `let x = f x in x`.

Prolog Introduction

Setting up prolog

Install the prolog compiler using the installer included in the course pack. To able to use swi-prolog anywhere, add the path to the directory of `swipl` executable in your PATH environment variable. It is generally found in the `bin` folder of the installation.

Once prolog is has been set-up you can run the `swipl` command to start swi-prolog

```
> swipl
```

to exit, use the `halt.` command (swipl will wait for a `.` for every query/ command)

```
?- halt.
```

To load prolog knowledge bases (`*.pl`), use the `swipl` command again but this time include the path to the prolog file as an argument.

```
> swipl knowledegebase.pl
```

Writing Prolog knowledge bases

Knowledge bases are made of facts and rules.

Prolog facts

Every prolog fact ends with a period. A prolog fact can be a constant, or a complex term.

Constant

Constants start with a lowercase letter

```
truth.
```

Complex Term

Complex terms follow the pattern `functorName(arg1,arg2,...,arg3)`. Arguments can be a constants, variables, or complex terms. Here are examples

```
isresistantto(squirtle,X).
vertical(line(point(X,Y),point(X,Z)))
```

> Functors are prologs representations for predicates. Whenever you see `functorName/2` mentioned, it means it is a functor called "functor" that accepts two parameters.

Rules

Rules follow the following pattern `head :- tail1,tail2,...tail3`. `head` is the conclusion of the implication statement, it can be any fact. The tails represent the hypothesis of the implication. Writing more than one tail means that the hypothesis is a conjunction of the tails. Examples:

```
isresistantto(X,Y) :- watertype(X),watertype(Y).
is_digesting(X,Y)  :-  just_ate(X,Y).
```

Writing queries

To ask the knowledge base some questions, you load the knowledge base file using `swipl` and write queries in the `?-` dialog inside `swipl`. Remember to always end the query with a period. Queries are written with the same pattern as prolog facts. For example.

```
?- truth.
```

```
true.
```

If the query has multiple lines of answer, prolog will wait for you to either press semicolon/space to show the next answer or the enter key to stop showing more answers.

Resolution

Rules of inference are just specializations of resolution. To write the inference rule as a resolution, convert all statements to disjunctions.

Modus Ponens

$$\begin{aligned} p \to q \\ p \\ q \end{aligned}$$

$$\begin{aligned} \neg p \lor q \\ p \lor \bot \\ q \lor \bot \end{aligned}$$

Modus Tollens

$$\begin{aligned} p \to q \\ \neg q \\ \neg p \end{aligned}$$

$$\neg p \vee q$$
$$\neg q \vee \bot$$
$$\neg p \vee \bot$$

Hypothetical Syllogism

$$p \to q$$
$$q \to r$$
$$p \to r$$

$$\neg p \vee q$$
$$\neg q \vee r$$
$$\neg p \vee r$$

Disjunctive Syllogism

$$p \vee q$$
$$\neg p$$
$$q$$

$$p \vee q$$
$$\neg p \vee \bot$$
$$q \vee \bot$$

Kotlin Syntax

Kotlin is a high-level language with rich syntax support. Lets start by discussing how statements and expressions in kotlin look like. Unlike C or C++, statements in kotlin do not end with a semicolon. The boundary of a statement is defined by newlines. Below you have two statements

```
3 + 5
9 - 2
```

```
7
```

Kotlin types

Numerical types

Numbers in kotlin can be `Int`s `Float`s, `Long`, `Double` and more. An `Int` literal can be written just like any number in math.

```
3
```

```
3
```

You can expose the specific type of some expression using the following syntax:

```
3::class.simpleName
```

```
Int
```

```
(2 - 3)::class.simpleName
```

```
Int
```

A number with decimal values are automatically interpreted as `Double`

```
3.1::class.simpleName
```

```
Double
```

```
2.0::class.simpleName
```

```
Double
```

If you want to force kotlin to interpret a number literal as `Long`, add the prefix `L` to the number literal. If you want to force kotlin to interpret a decimal literal as `Float`, add the prefix `f`.

```
2L::class.simpleName
```

```
Long
```

```
0xffffffffL::class.simpleName //works with hexadecimal values
too
```

```
Long
```

```
0.0f::class.simpleName
```

```
Float
```

```
(-1f)::class.simpleName
```

```
Float
```

Boolean values

You can write `Boolean` literals using `true` and `false`. Expression that evaluate into either `true` or `false` are also `Boolean` values

```
true::class.simpleName
```

```
Boolean
```

```
false::class.simpleName
```

```
Boolean
```

```
(4 > 5)::class.simpleName
```

```
Boolean
```

Characters

Character literals are created by surrounding single characters with single quotes

```
'a'::class.simpleName
```

```
Char
```

```
'2'::class.simpleName
```

```
Char
```

```
'\n'::class.simpleName
```

```
Char
```

Operations and type compatibility

When combining different types using operations, only some combinations of types are compatible for said operation. Test the following code on your own to see how kotlin handles them:

```
3 + 4.0
```

```
7.0
```

```
'2' + 1
```

```
3
```

```
1 + '4'
```

```
org.jetbrains.kotlinx.jupyter.exceptions.ReplCompilerException:
at Cell In[64], line 1, column 3: None of the following
functions can be called with the arguments supplied:
```

```
true + 0
```

Values and Variables

Just like other languages, you can assign values or into identifiers. There are two types in kotlin, values (denoted by `val`) and variables (denoted by `var`).

Variables are just like any variables in the imperative sense. You can assign values to it and you can reassign new values to it. When declaring `var`s (and `val`s) you must provide it with an assigned value. Without an assigned value, kotlin cannot compile

```
var x = 0
x = 1
```

```
var y
```

```
org.jetbrains.kotlinx.jupyter.exceptions.ReplCompilerException:
at Cell In[43], line 1, column 1: Abstract property 'y' in
non-abstract class 'Line_42_jupyter'
```

The value assigned to a `var` or `val` allows kotlin to **infer** the type of said `var` or `val`.

```
var y = 2
y::class.simpleName
```

```
Int
```

You can also explicitly declare the type of a `var` or `val` using the following syntax.

In some cases kotlin cannot infer the type of a `var` or `val`. This happens when the assigned value is ambiguous or a value cannot be provided (function declarations and abstractions). For these cases, an explicit type definition is required.

```
var y: Int = 2
```

Kotlin `val`s are just like `var`s except the values assigned during declaration are final. Kotlin `val`s cannot be reassigned with new values.

```
val z = 5
z = 4
```

```
org.jetbrains.kotlinx.jupyter.exceptions.ReplCompilerException:
at Cell In[49], line 2, column 1: Val cannot be reassigned
```

Compound Types

Compound types are types that are made of other types.

Pair

A `Pair` in kotlin is a pair of values. You can create a pair literal using the following syntax:

```
1 to 5
```

```
(1, 5)
```

You can access the first value and second value using `first` and `second`

```
val point = 2 to 5
point.first
```

```
2
```

```
point.second
```

```
5
```

```
val u = true to 'a' // it can be a mix of any two types
u.first
```

```
true
```

you can also use destructuring to separate a pair into two `vars` or `vals` using the following syntax:

```
val (booleanValue, charValue) = u
```

```
booleanValue
```

```
true
```

```
charValue
```

```
a
```

Collections

Collections in kotlin are like specialized arrays. They can hold a series of values and include useful member functions and attributes.

List

You can create a list using the builtin `listOf` constructor. The code below declares a variable called number assigned with the list containing the integers `1`, `2`, `3`, and `4` as its elements.

```
var numbers = listOf(1,2,3,4)
numbers
```

```
[1, 2, 3, 4]
```

When creating an empty list `var` or `val`, kotlin cannot infer the type of the elements of the said empty list. This is a case where you must explicitly declare the type of the of the variable. The syntax below declares that you are creating an empty list of `Ints`

```
var emptylist: List<Int> = listOf()
emptylist
```

```
[]
```

You can check if an element is a member of a Collection using the `in` operator

```
3 in numbers
```

```
true
```

```
5 in numbers
```

```
false
```

You can concatenate two lists using the + operator (as long as the types match)

```
listOf(1,2,3,4) + listOf(5,6,7)
```

```
[1, 2, 3, 4, 5, 6, 7]
```

The following are useful builtin functions and attributes for lists:

```
numbers.size //attribute where the size (number of elements)
of the list is stored
```

```
4
```

```
numbers.elementAt(2) //returns the element at index 2 (third
element)
```

```
3
```

You can also achieve the behavior of `elementAt` at using list indexing:

```
numbers[2]
```

```
3
```

Lists in kotlin are immutable. This means that you cannot change the elements inside it.

```
numbers[2] = 4
```

```
org.jetbrains.kotlinx.jupyter.exceptions.ReplCompilerException:
at Cell In[9], line 1, column 1: Unresolved reference. None
of the following candidates is applicable because of receiver
type mismatch:
```

But since `number` is a `var`, even if the current list stored inside it is immutable, `number` itself is a variable, meaning `number` itself can be changed.

```
numbers = listOf(5,6,7,8)
numbers
```

```
[5, 6, 7, 8]
```

MutableList

Mutable lists can be created using the `mutableList()` constructor. Like the name suggests, the elements of this type of collection can be changed.

```
var series = mutableListOf(-1,-2,-3,-4)
series
```

You can change the elements by reassigning elements accessed through element indexing:

```
series[1] = 5
series
```

```
[-1, 5, -3, -4]
```

You can append elements to the list using the function `add()`

```
series.add(-5)
series
```

```
[-1, 5, -3, -4, -5]
```

You can remove elements using `remove()` (removes the fires occurence of the reference passed in the function) or `removeAt()` (given an index, it removes the lement at said index)

```
series.remove(-3) //removes the first occurence of -3
series
```

```
[-1, 5, -4, -5]
```

```
series.removeAt(1) //removes the element at index 1
series
```

```
[-1, -5]
```

```
numbers.slice(0,1)
```

```
org.jetbrains.kotlinx.jupyter.exceptions.ReplCompilerException:
at Cell In[49], line 1, column 15: The integer literal does
not conform to the expected type IntRange
```

IntRange and IntProgression

`IntRanges` are collections of series of `Ints`. Using `IntRanges` you can generate a finite series of integers by simply defining the start and finish,

```
var range = 0..10
range
```

```
0..10
```

```
range.elementAt(0)
```

```
0
```

```
range.elementAt(2)
```

```
2
```

```
range.toList() // We reveal the elements of the range by
generating a list out of it
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

When specifying a step size you will create an `IntProgression` instead. This will allow you to skip some integers

```
(-3..4 step 2).toList()
```

```
[-3, -1, 1, 3]
```

```
(10..45 step 4).toList()
```

```
[10, 14, 18, 22, 26, 30, 34, 38, 42]
```

You can use `IntRanges` to create sublists out of `Lists` and `MutableLists`. Using the builtin `slice()` function you can generate a new list taking only elements specified in the `IntRange` instance passed in `slice()`.

```
val aList = listOf(101,102,103,104,105,106,107,108,109)
aList.slice(0..3) // creates a sublist from index 0 until
index 3
```

```
[101, 102, 103, 104]
```

```
aList.slice(5..(aList.size-1)) //creates a sublist from index
5 until the last available index
```

```
[106, 107, 108, 109]
```

```
aList.slice(8 downTo 0 step 2) // to create a decreasing
progression use `downTo` instead of `..`
```

```
[109, 107, 105, 103, 101]
```

Strings

`Strings` are special collections where the elements are exclusively `Chars`. Just like in C, `Strings` are used to represent text. You can create a `String` by surrounding characters with double quotes.

```
var str = "this is a string"
str
```

```
this is a string
```

```
str::class.simpleName
```

```
String
```

You can use a lot of the existing builtin list functions and operations for strings as well

```
str.length // str uses the attribute length as the number of elements
```

```
16
```

```
str[2] // accessing individual elements
```

```
i
```

```
str + " too." // concatenation
```

```
this is a string too.
```

```
'h' in str
```

```
true
```

```
"is a" in str
```

```
true
```

Just like `Lists`, `Strings` are also immutable

```
str[0] = 'b'
```

```
org.jetbrains.kotlinx.jupyter.exceptions.ReplCompilerException: at Cell In[123], line 1, column 1: Unresolved reference. None of the following candidates is applicable because of receiver type mismatch:
```

You can print strings to the standard output stream using the buitlin functions `print()` and `println()`, the only difference between these two is that `println()` automatically adds a linebreak (`\n`)

```
print("this ")
print("is")
println(" a")
println("string")
```

```
this is a
string
```

Using string templates you can easily insert non-`Strings` expressions into `String` literals. You can use the `${}` syntax to achieve this:

```
"Sum is: ${3 + 2}"
```

```
Sum is: 5
```

Expressions enclosed inside `${}` are evaluated, converted into `String` representations, and inserted into the string. When inserting `vars` or `vals` you can omit the curly braces:

```
val list = listOf(1,2,3)

"List elements:$list, size:${list.size}"
```

```
List elements:[1, 2, 3], size:3
```

Map

`Maps` are collections of `Pairs`. This datatype models the mapping between two values. In each `Pair` entry, the first value is called the key. You can think of a map as an array but instead of using integers as indices you can use any type as indices. The keys of a map are basically its indices.

```
val m = mapOf('a' to 1, 'b' to 2, 'x' to -1)
m
```

```
{a=1, b=2, x=-1}
```

```
m::class.simpleName
```

```
LinkedHashMap
```

When dereferencing using the index operator you can access the each key's associated value.

```
m['a']
```

```
1
```

```
m['x']
```

```
-1
```

Just like lists you can concatenate two maps using the + operator (as long as the types of match). You can also use `size` to find the number of pairs, and `in` to check if a key exists in the map

```
m + mapOf('y' to -4)
```

```
{a=1, b=2, x=-1, y=-4}
```

```
m.size
```

```
3
```

```
'x' in m
```

```
true
```

`Maps` are immutable. The entries cannot be changed. If you want a mutable version you can instead use a `MutableMap`:

```
val foodtype = mutableMapOf(
    "apple" to "fruit",
    "tomato" to "vegetable",
    "carrot" to "vegetable",
)
// the formatting above is optional, it helps with
readability

val food = "apple"
foodtype[food]
```

```
fruit
```

Since `foodtype` is mutable. You can add change entries using assignment:

```
foodtype["tomato"] = "fruit"
foodtype
```

```
{apple=fruit, tomato=fruit, carrot=vegetable}
```

If you assign on an non-existent key, kotlin will create a new instance with said key:

```
foodtype["soy sauce"] = "condiment"
foodtype
```

```
{apple=fruit, tomato=fruit, carrot=vegetable, soy
sauce=condiment}
```

you can delete using `remove()`

```
foodtype.remove("carrot")
foodtype
```

```
{apple=fruit, tomato=fruit, soy sauce=condiment}
```

Selection and Repetition

Selection and repetition in kotlin is written similar to c's selection and repetition.

if-else

```
if (1 > 3)
    print("foo")
else
    print("bar")
```

```
bar
```

```
if (1 > 2) {
    print("branch 1")
}
else if (0 == 0 && 'a' == 'a') {
    print("branch 2")
}
else {
    print("branch 3")
}
```

```
branch 2
```

Ternary

Kotlin also supports a ternary expression. A ternary expression can evaluate to either one of two possible expressions. If the condition on

the ternary expression evaluates into `true` then the entire ternary expression will evaluate into the first expression, and the second expression otherwise.

```
val number = -2

3 + (if (number > 0) number else (-number))
```

```
5
```

You can also use other `selection` patterns in kotlin using `when` clauses: kotlin control flow

```
var sum = 0
var i = 0
while (i < 5) {
    sum += i
    i++
}
sum
```

```
10
```

```
do {
    print("this")
} while (false)
```

```
this
```

`for` loop

A `for` loop in kotlin is a different from for loops in c. Instead of supplying it with the usual initialization-condition-increment, you instead supply it with the membership check operation, `in`, specifically checking if a given `var` (`iter` in the example below) is a member of a given collection (`listOf(1,2,3,4,5)`). If true then kotlin executes enclosed block. The loop initializes `iter` with the first element of the collection (`1` in the example below). After executing one iteration it then reassigns the `iter` to the next element (`2` in the example below) and loops back. It does this until it goes through every element of the collection exactly once.

```
for (iter in listOf(1,2,3,4,5))
    print("$iter,")
println()
```

```
1,2,3,4,5,
```

For loops work with any collection (and other types that have iterators), including `Maps`:

```
val entries : Map<Char,Int> = mapOf('a' to 20, 'b' to 5, 'c'
to 30)

var cumulativeSum = 0
for ((key,value) in entries) { // since elements of a map are
pairs, it can be destructured using the syntax here
    cumulativeSum += value
    println("$key : $value, $cumulativeSum")
}
```

```
a : 20, 20
b : 5, 25
c : 30, 55
```

You can achieve iteration over a range of numbers using when combining for loops with `IntRange` and `IntProgression` collections:

```
for (i in 0..6)
    print("$i,")
```

```
0,1,2,3,4,5,6,
```

```
for (i in 10 downTo -10 step 3)
    print("$i,")
```

```
10,7,4,1,-2,-5,-8,
```

Functions

You can define functions in haskell using the `fun` declaration:

```
fun printSquare(x: Int) {
    print("square of $x is ${x*x}")
}

printSquare(25)
```

```
square of 25 is 625
```

As seen above, when defining functions, kotlin cannot infer the type of the parameters. You must declare parameters with an explicit type.

The `printSquare()` function does not return anything. But internally it actually returns a type known as a `Unit`. It's a type that can only have one value. When you omit the return type of a function, kotlin automatically assumes that the return type is `Unit`.

When creating functions that return something, you must declare the return type:

```
fun square(x: Int): Int {
    return x * x
}

square(25)
```

```
625
```

Optional parameters

Kotlin functions can be declared with optional parameters. For a parameter to be designated optional, it must be supplied with a default value. In the example `introduce()`, we designate the parameter `isCensored` as optional by appending `= true`.

```
fun introduce(name: String, age: Int, isCensored: Boolean =
false) {
    if (!isCensored)
        println("$name, $age")
    else
        println("redacted")
}
```

When invoking `introduce()` you can omit `isCensored`, when doing so you `isCensored` will be automatically assigned with the its default value, `true`

```
introduce("Rub", 75)
```

```
Rub
```

```
introduce("Rub", 75, true)
```

```
redacted
```

Named parameters

When calling functions, you can rearrange the parameter order by explicitly naming parameters using the following syntax

```
introduce(age = 90, name = "Rub")
```

```
Rub, 90
```

Null Safety

By default, all types in kotlin cannot have a `null` value. We call these types non-nullable. Kotlin can infer if a specific `var`, `val`, parameter, or function return has the potential to be `null`. In the example below, kotlin infers the value `nullable` as a nullable integers. This is because if database is indexed with a nonexistent key (like `"invalid"`), then it cannot supply a value. As a result `database[q]` has a potential to evaluate into `null`

```
val database = mapOf("this" to 1, "that" to 2)


var q = "invalid"
val nullable = database[q]
```

```
nullable
```

```
null
```

On cases where you need to declare that a type is nullable suchas the function return value below. You can append the type with a `?`:

```
fun value(map: Map<String,Int>, key: String): Int? { //
removing the `?` will result in a compilation error
    return map[key]
}
```

A nullable type will cause type mismatch errors on some operations. In the example below, + wont work with nullable values. Even if the expression does not evaluate into null. The code below wont even compile:

```
value(database,"this") + 3
```

```
org.jetbrains.kotlinx.jupyter.exceptions.ReplCompilerException:
at Cell In[296], line 1, column 24: Operator call corresponds
to a dot-qualified call 'value(database,"this").plus(3)'
which is not allowed on a nullable receiver
'value(database,"this")'.
```

To remedy this you can do a null check using selection statements:

```
val v = value(database, "this")
if (v == null)
    print(v + 3)
else
    print(0 + 3)
```

```
3
```

You can also use the specialized safe elvis operator `?:` that works like a ternary. Using this operator if the left operand evaluates into `null` the elvis operation evaluates into the left operand. Otherwise, it evalutes into the right operand.

```
(value(database, "this") ?: 0) + 3
```

```
4
```

```
(value(database, "invalid") ?: 0) + 3
```

```
3
```

You can also avoid returning nullable types by ensuring null safety in the function call.

```
fun value(map: Map<String,Int>, key: String): Int { //
removing the `?` will not result in a compilation error
```

```
    return map[key] ?: 0
}
```

Imports and Packages

To import definitions from external packages you can use the import statement. The line below imports the class `File` found in the package `java.io`.

```
import java.io.File

val f = File("test.in")
```

You can also give imported definitions aliases. This will help when importing definitions from different packages that happene to have the same name:

```
import java.io.File as FileClass

val g = FileClass("test.in")
```

If you want to import everythin on the package `java.io`, you can use the following syntax

```
import java.io.*

val h: InputStream? = null // able import InputStream by
importing everything
```

One way to make definitions available across different files is by using a `package` declaration above your .kt files

```
package library
...
```

All of the definitions (classes, functions global variables) in every `.kt` file with the package header `library` will be included. This allows you to use defintions across different files as long as they are in the same package

File writing and reading

You can read and write files using the the class `File` in the `java.io*` package.

```
import java.io.File

val inputFile = File("input.in")
```

You can read the entire file using `readText()`

```
inputFile.readText()
```

```
content
more content
foo
bar
```

You can read the entire file line by line using `readLines()` which stores the contents into a `List<String>` of lines:

```
for (line in inputFile.readLines())
    println("line: $line")
```

```
line: content
line: more content
line: foo
line: bar
```

You can write to a file using `writeText()`

```
val output = File("output.out")

output.writeText("foo")
```

Calling `writeText()` again will replace the current contents.

```
output.writeText("bar")
```

To append, you can use `appendText()` instead

```
output.appendText(" extra content")
```

Other Things

LocalDate

Kotlin has plenty of classes related to date, one of them is imported from `java.time.LocalDate` which is available in the kotlin standard library.

You can create date instances using the following:

```
import java.time.LocalDate

LocalDate.of(2023, 5, 11)
```

```
2023-05-11
```

`LocalDates` can be compared. Using the < comparison checks if the left operand comes before the right operand

```
LocalDate.of(2023, 5, 11) < LocalDate.of(2023, 4, 30)
```

```
false
```

You can add or subtract days/weeks/months/years to a `LocalDate`

```
LocalDate.of(2023, 5, 11).plusDays(44L)
```

```
2023-06-24
```

```
LocalDate.of(2024, 5, 11).minusMonths(3L)
```

```
2024-02-11
```

You can subtract `LocalDates` to obtain the time period between them using `Period`

```
import java.time.Period

val period = Period.between(LocalDate.of(2023, 5, 11),
LocalDate.of(2023, 2, 28))

println(period.years)
println(period.months)
println(period.days)
```

```
0
-2
-11
```

OOP kotlin

While kotlin is a modern multiparadaigm language. Its supports rich OOP support. In fact kotlin is very strongly OOP flavored. Every type in kotlin is a class and every value is an object.

Kotlin Classes

Creating classes in kotlin is very easy, here's an empty class called `EmptyClass`:

```
class EmptyClass
```

> It is conventional to name your classes as nouns that start with capital letters like `EmptyClass`

To make classes more interesting, lets put something inside it. You can put methods and attributes inside classes. Everything found inside the `{ }` scope of a class, belongs to that class. Below we have a more interesting class called `VoiceBox`

```
class VoiceBox {
    fun speak() {
        println("Hi I'm a voicebox that speaks")
    }
}
```

Inside the scope of the class `VoiceBox` we have the function `speak()`. Since this function is a member of the class `VoiceBox` we also call it a **method**. We say `speak()` is a method of `VoiceBox`

To use this class we create an **instance** of `VoiceBox`. We invoke what is know as the **default constructor** of `VoiceBox` using the syntax `VoiceBox()`. The constructor `VoiceBox()` will evaluate into an instance of `VoiceBox`.

This constructed instance is assign to the `val v`. We call `v` an instance of `VoiceBox` or a `VoiceBox` **object**. To interact with this voicebox object we can call its method `speak()`. We use the dot-reference operator as shown below. This tells kotlin that we are invloking the method `speak()` specifically owned by the instance `v`.

```
val v = VoiceBox()
v.speak()
```

```
Hi I'm a voicebox that speaks
```

Classes vs objects. Classes and objects are similar things. We refer to `v` as a `VoiceBox` object or instance. This is different from the `VoiceBox` class itself. The `VoiceBox` class refers to the definition of what `VoiceBox` objects like `v` can be. There can only be one `VoiceBox` class. On the other hand a `VoiceBox` instance like `v` refers to one of the many possible `VoiceBox` instances that can be created. In the example below we create another `VoiceBox` instance `w`. Right now there is no way to differentiate between these two because all `VoiceBoxes` are created the same way.

> The distinction between classes may not be important right now (especially in kotlin). But in some language you can create methods that are specifically owned by the class itself rather than the

instances of the class. These methods are known as static methods. Kotlin does not happen to support static methods.

```
val w = VoiceBox()
v.speak()
```

```
Hi I'm a voicebox that speaks
```

Customizing the Constructor

All class definitions automatically come with a default constructor. But to add nuance to how instances are created we can change the constructor.

```
class Robot constructor (var name: String) {
    fun talk() {
        println("Howdy it's me $name")
    }

    fun communicate(partner: Robot) {
        println("Howdy ${partner.name} it's me $name")
    }

}
```

In the example above we change our constructor so that when invoked it accepts a string called `name` as a parameter. By adding `name` in the constructor header kotlin automatically creates an **attribute** called `name`. This attribute will recieve the value passed during the construction of `Robot`.

The attributes of a specific instance is owned by that specific instance. Using the code below as n example, the `Robot` instance `r1` has its own version of the attribute `name` (accessed through `r1.name()`) which is distinct from `r2`s version of `name`.

Note how inside the function communicate, you refer to an external instance's name attribute using dot-reference

(`partner.name`) while referencing an internal attribute is done directly.

```
val r1 = Robot("Bonk")
val r2 = Robot("Chonk")

println(r1.name)
println(r2.name)
```

```
Bonk
Chonk
```

Going back to the class definition of `Robot` you can see the attribute `name` referenced inside the scope of `Robot`. All attributes owned by `Robot` can referenced by any of `Robots` methods. In fact it can be referenced anywhere inside the `Robot` scope.

When you call the methods `talk()` or `communicate()`, the reference `name` can be used without any problem.

```
r1.talk()
r2.communicate(r1)
```

```
Howdy it's me Bonk
Howdy Bonk it's me Chonk
```

Since `name` is declared as `var` in the constructor, it can be changed like any variable

```
r1.name = "Donk"
r1.talk()
```

```
Howdy it's me Donk
```

```
class RoboticArm (val type: Char) { //if there are no
visibility modifiers, the `constructor` keyword can be
ommitted

    var position = 0.0f

    fun move(amount: Float) {
        position = position + amount
```

```
    }


    fun reset() {
        position = 0.0f
    }

}

val arm = RoboticArm('A')
println(arm.position)
arm.move(0.2f)
arm.move(0.5f)
println(arm.position)
```

```
0.0
0.7
```

In some cases you might want to add attributes to your classes without making said attributes as constructor parameters. To do this you simply declare the attribute somewhere inside the class but outside the method bodies. Just like the attribute `position` in the class `RoboticArm` above.

You can only place declarations and definitions in the class defintion scope. You cannot put runnable code. If you want to run some code during object construction you can create an `init` block. All of the code inside an `init` block will be executed during the constructor invocation.

```
class Camera {

    init {
        println("startup.")
        cleanSensor()
    }

    fun cleanSensor() {
        println("cleaning sensor...")
        println("cleaning complete")
    }
```

```
}
val cam = Camera()
```

```
startup.
cleaning sensor...
cleaning complete
```

You can also **overload** the class's constructor in kotlin. To do this you simply define a new constructor block with a different set of parameters. The constructor in the class header (beside the class name) is called the primary constructor while all other constructors are know as secondary constructors.

You can read more aboth secondary constructors here: Constructors

```
class Person (val name: String) {
    constructor (firstname: String, lastname: String) :
this("$firstname $lastname")
}
```

Kotlin Specializations and Realizations

All classes in kotlin inherit from the `Any` generalization. The `Any` class is a class that doesn't have any generalization.

```
println(r1 is Any) // r1 is an instance of Robot
```

```
true
```

To establish your a specialization relationship in kotlin you must first mark the general class to be `open`:

```
open class Robot constructor (var name: String) {
...
```

```
open class Robot constructor (open var name: String) { //open
is a modifier that allows Robot to be specialized

    open fun talk() {
        println("Howdy it's me $name")
    }
```

```
    open fun communicate(partner: Robot) {
        println("Howdy ${partner.name} it's me $name")
    }

}
```

To establish that the class `Skybot` below is a specialization of `Robot`, it must be specified in class header using `:` syntax. In this particular example, `Robot` has a primary constructor (i.e. it is not using a default constructor), so `Skybot` must have a primary constructor with the same pattern as `Robot`. The primary constructor of `Robot` accepts one `String` called name, so `Skybots` primary constructor should also accept one `String`. This must be done so that all of the attribute assignments in `Robot` are also inherited by `Skybot`.

```
class SkyBot (name: String) : Robot(name) {
    fun fly(height: Int) {
        println("I'm flying ${height}m high in the air")
    }
}
```

Since `Skybot` is now a specialized `Robot`, it will inherit all of the inheritable definitions in `Robot`. This includes `talk()` and `communicate()`

```
val r = Robot("Donk")
val s1 = SkyBot("Zonk")

s1.talk()
s1.communicate(r)
```

```
Howdy it's me Zonk
Howdy Donk it's me Zonk
```

Due to polymorphism all specializations of `Robot` are also considered as `Robots`. This means that `Skybot` instances can be used as `Robots` without any type mismatch. The method `communicate()` which accepts `Robot` instances as parameters can also accept `Skybot` instances.

```
val s2 = SkyBot("Wonk")

s1.communicate(s2)
```

```
Howdy Wonk it's me Zonk
```

Instances of `Skybot` can of course use it's specialized method `fly()`

```
s1.fly(4)
```

```
I'm flying 4m high in the air
```

Specializations of classes can also override the definitions of their generalization.

```
class ShadeBot(name: String, val visorOpacity: Float) :
Robot(name) {

    override fun communicate(partner: Robot) {
        if (visorOpacity >= 1)
            println("Howdy, it's me $name. Sorry I cant see
you my shades are too dark")
        else
            println("Howdy ${partner.name} it's me $name")
    }

}
```

To override a method or an attribute you must explicitly, use the `override` modifier. This tells kotlin that instead of inheriting `communicate()` you will replace it with the definition found in `ShadeBot`. Take note that methods are final by default so you can only override methods that are declared to be `open` in the generalization. Without the modifiers `override` and `open` the override of `communicate()` will not compile.

```
open class Robot constructor (var name: String) {

    open fun talk() {... //talk() is actually not overriden
right now but we can declare it to be open for future
specializations
```

```
    open fun communicate(partner: Robot) {...

}
```

Also in the class header of `ShadeBot` we are overriding the primary constructor to accept two parameters instead of just one. Since `visorOpacity` is a special attribute of `ShadeBot` it has not been declared yet. Therefore we have to explicitly declare it as `val` or `var` in the in the primary constructor declaration. The attribute `name`, on the other hand, is inherited so it doesn't need to be redeclared. If you do redeclare it, then it will be considered as an override, requiring the `override` modifier.

```
val sh1 = ShadeBot("Tonk", 1.0f)
sh1.communicate(s1)
```

```
Howdy, it's me Tonk. Sorry I cant see you my shades are too
dark
```

Realization

To create an abstraction in kotlin you simply use the keyword `interface`. Any abstraction's methods must be declared to be `abstract`. The `abstract` modifier allows these functions to have no body. Note that abstract functions are open by default so you dont need the `open` modifier

```
interface BorrowableItem {
    abstract fun borrow()
    abstract fun name(): String
}
```

To realize an existing abstraction, you also use the `:` syntax

```
class Book(val title: String) : BorrowableItem {

    override fun borrow() {
        println("I am a book called ${name()} and I'm being
borrowed")
    }
```

```
    override fun name(): String{
        return title
    }

}

class IMacUnit(val id: Int) : BorrowableItem {

    override fun borrow() {
        println("I am an imac called ${name()} and I'm being
borrowed")
    }

    override fun name(): String {
        return "iMac$id"
    }
}
```

Due to polymorphism, you can declare a `Book` or `IMacUnit` instance to be of type `BorrowableItem`.

```
val b: BorrowableItem = Book("Necronomicon")
val imac: BorrowableItem = IMacUnit(5)

b.borrow()
imac.borrow()
```

```
I am a book called Necronomicon and I'm being borrowed
I am an imac called iMac5 and I'm being borrowed
```

```
val binder: List<BorrowableItem> = listOf(b,imac)

for (bi in binder)
    println(bi.name())
```

```
Necronomicon
iMac5
```

```
binder::class.simpleName
```

```
ArrayList
```

Any class that realizes the `BorrowableItem` abstraction will be forced to override all of its abstract functions. If you miss overriding even one function, it will lead to an error

```
class Research: BorrowableItem{
    override fun name(): String {
        return "Research"
    }
}
```

```
org.jetbrains.kotlinx.jupyter.exceptions.ReplCompilerException:
at Cell In[25], line 1, column 1: Class 'Research' is not
abstract and does not implement abstract member public
abstract fun borrow(): Unit defined in
Line_19_jupyter.BorrowableItem
```

Visibility Control

To implement proper data hiding principle sin OOP, kotlin has the visibility modifers `public`, `private`, and `protected`.

```
open class ClandestineClass (
    public var v1: Int,
    protected var v2: Int,
    private var v3: Int
) { //the syntax above is just a way to make the formatted
constructor look neater
    public open fun method1() {
        println("Hey, these are my values")
        println("$v1, $v2, $v3")
    }

    protected open fun method2() {
        println("Hey!")
    }

    private fun method3() {
        println("...")
    }
```

```
}
```

```
val secretiveObject = ClandestineClass(1,2,3)
```

```
secretiveObject.v1
```

```
1
```

```
secretiveObject.v2
```

```
org.jetbrains.kotlinx.jupyter.exceptions.ReplCompilerException:
at Cell In[53], line 1, column 17: Cannot access 'v2': it is
protected in 'ClandestineClass'
```

```
secretiveObject.v3
```

```
org.jetbrains.kotlinx.jupyter.exceptions.ReplCompilerException:
at Cell In[54], line 1, column 17: Cannot access 'v3': it is
private in 'ClandestineClass'
```

As you see in the example above, accessing the attribute `v1` outside the scope of `ClandestineClass` works. On the other hand accessing the attributes `v2`, and `v3` will cause an error. This is because only `public` attributes can be accessed outside the scope of any give class.

```
secretiveObject.method1()
```

```
Hey, these are my values
1, 2, 3
```

The function body of `method1()` is inside the scope of the class, so there are no issues with using `v2`, and `v3` inside `method1()`.

Just like attributes, `protected` and `private` methods cannot be accessed outside the scope of the class.

```
secretiveObject.method2()
```

```
org.jetbrains.kotlinx.jupyter.exceptions.ReplCompilerException:
at Cell In[56], line 1, column 17: Cannot access 'method2':
it is protected in 'ClandestineClass'
```

```
secretiveObject.method3()
```

```
org.jetbrains.kotlinx.jupyter.exceptions.ReplCompilerException:
at Cell In[57], line 1, column 17: Cannot access 'method3':
it is private in 'ClandestineClass'
```

```
open class SpecialClandestineClass1(
    v1: Int,
    v2: Int,
    v3: Int
) : ClandestineClass(v1,v2,v3) {

    override fun method2() {
        println("Hey! Attempting to display values")
        println("$v1, $v2, $v3")
    }

}
```

```
org.jetbrains.kotlinx.jupyter.exceptions.ReplCompilerException:
at Cell In[58], line 9, column 29: Cannot access 'v3': it is
invisible (private in a supertype) in
'SpecialClandestineClass1'
```

The specializations of a class will have access to `public` and `protected` attributes. The code above will not compile because `SpecialClandestineClass1` doesn't have access to the private attribute `v3`.

Omitting `v3` inside the body of `method2`, the code now works:

```
open class SpecialClandestineClass2(
    v1: Int,
    v2: Int,
    v3: Int
) : ClandestineClass(v1,v2,v3) {

    override fun method2() {
        println("Hey! Attempting to display values")
        println("$v1, $v2")
```

```
    }

}
```

Note that although private attributes and methods of a general class cannot be accessed by its specializations, said attributes and methods are still inherited. For example, instances of `SpecialClandestineClass2` can still display the private attribute `v3` since it still exists. This works because the definition of `method1()` being used here is inherited from `ClandestineClass`.

```
val secret2 = SpecialClandestineClass2(1,2,3)
secret2.method1()
```

```
Hey, these are my values
1, 2, 3
```

Note that since private methods cannot be accessed by specializations, they cannot be have the `open` modifier as well.

Here is a summary of visibility modifiers and where they can be accessed

visibility	accessed by specializations	accessed by clients (outside)
public	yes	yes
protected	yes	no
private	no	no

Abstract Classes

Abstract classes are hybrids of interfaces and concrete classes. Abstract classes, unlike interfaces, can have attributes and contructors. Also, unlike concrete classes, it can be defined with abstract methods.

In the example below `AbsClass` has the modifier `abstract` making it an abstract class. The method `printSomethingA()` is an abstract function so all realizations of `AbsClass` must override it. While the method `printSomethingB()` is concrete, so it can be inherited or overridden.

```
abstract class AbsClass(val state: String) {

    abstract fun printSomethingA()

    open fun printSomethingB() {
        println("I'm inherited, you can also override me if
you want")
    }

}

class ConcreteClass1(state: String) : AbsClass(state) {

    override fun printSomethingA() {
        println("I'm implemented by ConcreteClass1")
    }

}

class ConcreteClass2(state: String) : AbsClass(state) {

    override fun printSomethingA() {
        println("I'm implemented by ConcreteClass2")
    }

    override fun printSomethingB() {
        println("I'm overriden by ConcreteClass2")
    }

}
val c1 = ConcreteClass1("initial state")
val c2 = ConcreteClass2("initial state")

c1.printSomethingA()
c1.printSomethingB()
println()
c2.printSomethingA()
c2.printSomethingB()
```

```
I'm implemented by ConcreteClass1
I'm inherited, you can also override me if you want

I'm implemented by ConcreteClass2
I'm overriden by ConcreteClass2
```

Exceptions (Kotlin)

Introduction

One of the features added to imperative programming was the elegant handling of errors. Exception handling provided OOP a mechanism to control what exactly the system does when parts of the system fail.

Learning Outcomes

1. Create methods that throw errors in kotlin
2. Create a try-except clause to properly react to errors in kotlin
3. Design systems that correctly assign error handling responsibilities

Let's explore this with an example. In the snippet below, the last line is not reached because the system fails during `println(quotient(3f, 0f))`.

```
fun quotientUnsafe(a: Int, b:Int): Int {
    return a/b
}
```

```
fun main() {
    println(quotientUnsafe(5, 2))
    println(quotientUnsafe(3, 0))
    println("continuation")
}
```

```
2
Exception in thread "main" java.lang.ArithmeticException: /
by zero
    at ExceptionsKt.quotientUnsafe(exceptions.kt:7)
    at ExceptionsKt.main(exceptions.kt:48)
    at ExceptionsKt.main(exceptions.kt)
```

[31]

Problematic functions and methods like `quotientUnsafe` don't always return a float. The problem for this `quotientUnsafe` function is that there is a possibility you'll end up dividing with zero. This introduces the concept of exceptions. Where the quotient function works **except** when the denominator is zero.

To implement this kind of behavior. You can create an if-else check (or any control statement) to make sure the denominator is not zero. If you do encounter a zero denominator you **`throw`** an error. Here you are throwing a user defined error object called `DivisionByZeroError`. This user defined exception must be defined to be a specialization of the built in class `Exception`.

```
class DivisionByZeroError(message: String) :
Exception(message)

fun quotient(a: Float, b: Float): Float {
    when (b) {
        0f -> throw DivisionByZeroError("Division By Zero:
$a/$b")
        else -> return a/b
```

[31]Kotlin has its own exception thrown when it encounters division by zero (`ArithmeticException`) but we'll create our own for the sake of learning.

```
    }
}
```

When quotient is called where the denominator passed is zero, python will inform you that the function call has resulted in a `DivisionByZeroError`

```
fun main() {
    println(quotient(5f, 2f))
    try {
        println(quotient(3f, 0f))
    } catch (ex: DivisionByZeroError) {
        println("division by zero")
    }
    println("continuation")
}
```

```
2.5
division by zero
continuation
```

By enclosing the lines of code that could potentially throw errors with a `block` you create a safety net for the system. The system **tries** to execute these lines, and if there are no errors thrown inside the `try` block then the system runs normally. Nothing abnormal would occur **except** when the problematic lines of code throws an error. In this specific case, the system is **catching**, a specific type of error called `DivisionByZeroError`. If a specific type of `Exception` is not specified in the exception block, the system will catch any `Exception` specialization.

If a method with potential to throw errors like `quotient()`, is invoked inside another method without using the try except block, the caller method becomes a method with potential to throw errors as well.

```
fun mixedFraction(wholePart: Int, numerator: Int,
denominator: Int): Float {
    return wholePart + quotient(numerator.toFloat(),
denominator.toFloat())
}
```

```
fun main() {
    println(mixedFraction(1,1,0))
    println("continuation")
}
```

```
Exception in thread "main" DivisionByZeroError: Division By
Zero: 1.0/0.0
    at ExceptionsKt.quotient(exceptions.kt:12)
    at ExceptionsKt.mixedFraction(exceptions.kt:18)
    at ExceptionsKt.main(exceptions.kt:47)
    at ExceptionsKt.main(exceptions.kt)
```

Here, the `quotient()`'s caller,`mixedFraction()`, does not enclose the problematic line with a try-except block. This means that `mixedFraction` is basically the ignoring any potential error, shifting the responsibility of dealing with the error to wherever `mixedFraction()` is called.

On the example below, `quotient()` is invoked inside `quotientString()`, but instead of ignoring the error, `quotientString()` deals with it using a try-except block. When quotient fails, the function returns "undefined number" instead of a fraction string.

```
fun quotientString(a: Float, b: Float): String {
    try {
        val quotient = quotient(a,b)
        val wholePart = floor(quotient(a,b))
        return "whole part: $wholePart, decimal part:
${quotient - wholePart}"
    } catch (ex: DivisionByZeroError) {
        return "undefined number"
    }
}

fun main() {
    println(quotientString(17f,7f))
    println(quotientString(1f,0f))
    println(quotientString(1f,3f))
}
```

```
2 and 0.4285714285714284
undefined number
0 and 0.3333333333333333
```

By dealing with potential errors, `quotientString()` becomes a safe function that has no potential of throwing `DivisionByZeroError`.

You can catch multiple kinds of exceptions if you want to handle different exceptions in different ways. Here, you see different kind of `Exception` called `IndexOutOfBoundsException`. This exception is thrown when an indexed collection like `List` accesses a non-existent member. Here the function `quotientList` wants to append `Float.POSITIVE_INFINITY` if you're dividing by zero and not append anything if you're dividing with non-existent list members.

```
fun quotientList(l: List<Float>, m: List<Float>): List<Float>
{
    val r: MutableList<Float> = mutableListOf()
    for (i in 0..(l.size-1)) {
        try {
            r.add(quotient(l[i], m[i]))
        } catch (ex: DivisionByZeroError) {
            r.add(Float.POSITIVE_INFINITY)
        } catch (ex: IndexOutOfBoundsException) {
            //do not append anything to the list
        }
    }
    return r.toList()
}

fun main() {
    println(quotientList(listOf(1f,2f,3f,4f,5f,6f),
listOf(3f,0f,2f,2f,1f)))
}
```

```
[0.33333334, Infinity, 1.5, 2.0, 5.0]
```

To ignore errors or to deal with errors?

So you've seen two ways to react with potential errors, to ignore them like `mixedFraction()` or to deal with them like `quotientString()`.

Which is the proper way? You might think that dealing with errors is better since it's this way doesn't break the system. But actually the choice to ignore or to deal with errors depends on who has the correct responsibility in fixing the error.

If you immediately deal with the error as quickly as possible, you'll end up missing the importance of raising errors. The method `quotient()` for example, is responsible for providing the caller with a quotient, that must be this method's only responsibility. You should not give `quotient` the responsibility of fixing division by zeros. That responsibility lies on its caller, because different caller's may have different ways to deal with the error. The method `quotientString()` deals with division by zero with "undefined number" while the method `quotientList()` deals with division by zero with ∞. These two callers have different ways of interpreting division by zero, so they should be the one's responsible for dealing with the error.

References

Abelson, Harold, Gerald Jay Sussman, and Julie Sussman. 2002. “Linear Recursion and Iteration.” In *Structure and Interpretation of Computer Programs*, 2. ed., 7. [pr.]. Electrical Engineering and Computer Science Series. MIT Press [u.a.].

Alexander, Christopher, Sara Ishikawa, Murray Silverstein, and Max Jacobson. 2017. *A Pattern Language: Towns, Buildings, Construction.* 41. print. Center for Environmental Structure Series 2. Oxford Univ. Press.

Blackburn, Patrick, Johan Bos, and Kristina Striegnitz. 2006. *Learn Prolog Now!* Texts in Computing 7. College Publications.

Böhm, Corrado, and Giuseppe Jacopini. 1966. “Flow Diagrams, Turing Machines and Languages with Only Two Formation Rules.” *Communications of the ACM* 9 (5): 366–71. https://doi.org/10.1145/355592.365646.

Church, Alonzo. 1932. “A Set of Postulates for the Foundation of Logic.” *The Annals of Mathematics* 33 (2): 346. https://doi.org/10.2307/1968337.

Cote, Joe. 2025. “What Is Computer Programming.” In *Southern New Hampshire University*.

Gamma, Erich, ed. 2011. *Design Patterns: Elements of Reusable Object-Oriented Software*. 39. printing. Addison-Wesley Professional Computing Series. Addison-Wesley.

Hosch, William. 2010. “Peano Axioms.” In *Encyclopaedia Britannica*. Encylopaedia Britannica, Inc. https://www.britannica.com/science/Peano-axioms.

MovGP0. 2013. *Overview of the Various Programming Paradigms According to Peter Van Roy*.

PARADIGM Definition & Meaning Dictionary.com. n.d. Accessed August 10, 2026. https://www.dictionary.com/browse/paradigm.

Steele, Guy L., and Richard P. Gabriel. 1996. “The Evolution of Lisp.” In *History of Programming Languages—II*, edited by Thomas J. Bergin and Richard G. Gibson. ACM. https://doi.org/10.1145/234286.1057818.

Sturm, Oliver. 2011. “Accumulator Passing Style.” In *Functional Programming in C#: Classic Programming Techniques for Modern Projects*. Wiley Pub.

Triska, Markus. 2026. *The Power of Prolog*. Metalevel.at.